HieroTSS: Updatable Weighted Threshold Signing System for Stake-based Blockchains

Hashgraph

May 16, 2026

Abstract

HieroTSS is a weighted threshold signing system for the Hedera distributed ledger that supports validator-set rotation without sacrificing verifier efficiency. At its core, HieroTSS composes three components: (i) the hinTS silent threshold signature [GJM+24] for constant-size BLS aggregate signatures; (ii) Schnorr multi-signatures for authorized key update, establishing a root-of-trust; and (iii) NOVA-based Incrementally Verifiable Computation [KST22] with a Groth16 decider [Gro16] to compress the entire rotation history.

A validator quorum signs messages using hinTS, producing a single $\approx$ 96-byte BLS aggregate signature verifiable in a constant number of pairings, regardless of how many validators participated. When the validator set rotates, the outgoing quorum signs the new address book under Schnorr; the prover folds this authorization into a running NOVA folding scheme. Finally the folded proof is compressed by a Groth16 decider into a $\approx$ 192-byte proof. An external verifier—a light client, a smart contract, a cross-chain bridge—that holds only the genesis state hash (as a root-of-trust) can verify the current epoch's signatures with (i) a single hinTS verification (ii) plus a single Groth16 check (3 pairings).

Contents

1 Introduction

The Hedera decentralized ledger network relies on a weighted validator set to collectively sign consensus outputs. Two fundamental requirements drive the design of the signing system.

- **Efficiency.** The network has many validators with heterogeneous weights (represented by 64 bits). A naïve approach—collect all partial BLS signatures and verify each one—scales linearly in the validator count, both in signature size and in verifier work. One can use aggregatable multi-signatures based on BLS to make the signature compact, yet the verifier's work still scales linearly in the validator count. This is incompatible with on-chain verification (EVM gas limits) and with lightweight clients. Using threshold signatures typically require a distributed key-generation protocol in the setup, in that all parties must participate. Furthermore, they do not scale for the heteregenous weighted setting we are interested in.
- **Updatability.** We have a dynamic setting, in that the validator set changes over time: nodes join, leave, and change weight. Any external verifier (smart-contract, cross-chain bridge, light client etc) be able to verify signatures from the current epoch, which has its own verification key. However, putting trust on any such public keys put the network in severe risk. A natural trust mitigation strategy is to trust the genesis epoch only. Therefore, the signature from the current epoch is only meaningful if the verifier can confirm that the signing key was legitimately derived from the genesis state.

What HieroTSS does. HieroTSS is a threshold signing system that solves both problems simultaneously. In particular, HieroTSS consists of the following two parts:

1. **Weighted Silent Threshold Signing (hinTS).** hinTS [GJM+24] is a *silent* threshold signature: each validator registers a BLS key and a set of KZG-based *hints* once, then participates in any number of signing sessions without further interaction. An aggregator (which maybe one of the signers) collects partial BLS signatures from any quorum meeting the weight threshold and compresses them into a single aggregate signature, verifiable with an combined verification key VK. The verifier checks using a constant number of pairings: it does not need to know which validators participated, what their individual keys are, or what the quorum weights were—yet the soundness is maintained by the use of succinct (zero-knowledge) proofs.[1] Signature size and verification time are constant regardless of the number of validators or quorum size.
2. **Committee Rotation and Key Updates (Wraps)** This part, called Wraps [2] can further be split into two parts:
   (a) **Authorized key rotation (Schnorr multi-signature).** When the network rotates from address book AB_t to AB_{t+1},[3] a quorum from AB_t signs the rotation message $\mathsf{msg}_t = \mathcal{H}(\mathsf{VK}_t \| \mathsf{AB}_{t+1})$ under a Schnorr multi-signature, where VK_t is the combined hinTS verification key for epoch-t. The rotation message explicitly binds the current hinTS verification key and the incoming address book, preventing replay and substitution attacks. Moreover, a proof-of-possession at key registration (Section 4.1) prevents rogue-key attacks for Schnorr multi-signature.
   (b) **Compressed rotation history (NOVA IVC + Groth16 decider).** Each rotation authorization is folded into a running NOVA incrementally verifiable computation (IVC) [KST22]. Folding is cheap: the prover performs two MSMs per step, with no FFTs. The accumulated proof is compressed by a Groth16 decider circuit [Gro16] into a $\approx$ 192-byte proof. An external verifier that trusts only the genesis state $\psi_0 = (\mathcal{H}(\mathsf{AB}_0), \mathcal{H}(\mathsf{VK}_0))$ can verify the entire rotation chain with a single Groth16 check—3 pairings, 1 MSM over public inputs, regardless of the current epoch number.

[1] We actually do not need the zero-knowledge property itself; the succinctness and the soundness suffice.
[2] The acronym for Weighted Robust Asynchronous Proactive Signing.
[3] Looking ahead, each address book consists of identities of the parties active in a specific epoch.

Verification flow. An external verifier, in epoch-t, proceeds in two steps:

1. Verify the hints signature, generated in the current epoch, with respect to VK_t. This costs a constant number of pairings independent of validator count.
2. Verify the compressed $\mathsf{Groth16}$ proof that $\boldsymbol{\psi}_t = (\mathcal{H}(\mathsf{AB}_t), \mathcal{H}(\mathsf{VK}_t))$ is the legitimate t-step successor of $\boldsymbol{\psi}_0$. This costs 3 pairings.

No intermediate address books, no partial signatures, and no quorum membership information are needed at verification time.

Security Intuition. Security argument for the soundness of the HieroTSS system essentially stems from the security of underlying components (which are well established in the literature): (i) unforgeability of hinTS; (ii) Schnorr unforgeability with proof-of-possession; (iii), NOVA-IVC folding soundness; (iv) $\mathsf{Groth16}$ soundness. Recall that any verifier's root of trust is the genesis hash $\mathcal{H}(\mathsf{AB}_0)$, which is published at network launch.

Document Structure. Section 2 fixes notation. Section 3 specifies hinTS in full detail, including the optimized aggregation and verification algorithms. Sections 4 specify the key update / committee rotation building blocks: Schnorr multi-signatures, $\mathsf{R1CS}$systems, $\mathsf{Groth16}$ proof systems, Pedersen commitments, NOVA folding, IVC, $\mathsf{CycleFold}$ etc. Finally, Section 5 assembles the complete HieroTSS protocol, works through a 5-party example across two epoch rotations, and tabulates resource requirements.

2 Notation and Preliminaries

We use κ for the security parameter; typically set to 128. We use $\mathsf{negl}(\kappa)$ for a negligible function, which means that for all polynomial $f(\kappa)$, $\mathsf{negl}(\kappa) < 1/f(\kappa)$ for large enough κ—intuitively, this means for a standard security parameter, such as 128, this quantity is negligibly small. All algorithms take κ as input implicitly. We write $[n]$ for the set $\{1, 2, \ldots, n\}$. For a set S, we say $s \leftarrow_\$ S$ to denote a uniform random sampling. For multiple values $s_1, s_2, \ldots$, we use the same notation $s_1, s_2, \ldots, \leftarrow_\$ S$. We use $x := a$ to denote assignments. $(x \stackrel{?}{=} y)$ returns a decision bit, which is 1 if and only if $x = y$, and 0 otherwise. For algorithms Alg and functions $\mathcal{F}$, we write $\mathsf{Alg}^{\mathcal{F}}$ to denote that the algorithm Alg uses $\mathcal{F}$ as a sub-routine.

Vectors are denoted by boldface, e.g. $\boldsymbol{v}$ and by default vectors are columns. For row vectors $\boldsymbol{v}^T$ we shall be using the transpose notation. The i-th element of vector $\boldsymbol{v}$ is denoted as v_i. Matrices are typically denoted in upper case, M, and the corresponding (i, j)-th element is denoted M_{ij}. Also, the i-th column is denoted by M_i, and the j-th row can be denoted similarly using the transpose notation M_j^{T}. Let us also define the Hadamard product between two matrices.

Hadamard Product Let M_1 and M_2 be two matrices both of dimensions $m \times n$, with entries in a field $\mathbb{F}$:

$$M_1 = \big(a_{ij}\big)_{\substack{1 \le i \le m \\ 1 \le j \le n}} \in \mathbb{F}^{m \times n}, \qquad M_2 = \big(b_{ij}\big)_{\substack{1 \le i \le m \\ 1 \le j \le n}} \in \mathbb{F}^{m \times n}.$$

The **Hadamard product** (also called the *element-wise* or *Schur product*) of M_1 and M_2, denoted $M_1 \circ M_2$, is the $m \times n$ matrix whose (i, j)-th entry is the product of the corresponding entries of M_1 and M_2:

$$\boxed{M_1 \circ M_2 \;=\; \big(a_{ij}\, b_{ij}\big)_{\substack{1 \le i \le m \\ 1 \le j \le n}} \;\in\; \mathbb{F}^{m \times n}.}$$

Explicitly, the full matrix reads

$$M_1 \circ M_2 = \begin{pmatrix} a_{11}b_{11} & a_{12}b_{12} & \cdots & a_{1n}b_{1n} \\ a_{21}b_{21} & a_{22}b_{22} & \cdots & a_{2n}b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1}b_{m1} & a_{m2}b_{m2} & \cdots & a_{mn}b_{mn} \end{pmatrix}.$$

2.1 Bilinear Pairing

We use $(\mathbb{F}, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2, g_T)$ for an asymmetric pairing group parameters, where $\mathbb{F}$ is a prime field of order $p = \exp(\Omega(\kappa))$ (e.g. 512 bit prime); $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ are multiplicative groups of order p with a standard pairing operation $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$; $g_1 \in \mathbb{G}_1$ and $g_2 \in \mathbb{G}_2$ are uniformly sampled generators such that $g_T = e(g_1, g_2)$ is a generator of $\mathbb{G}_T$. For a field element $x \in \mathbb{F}$, we shall write $[x]_1$, $[x]_2$, and $[x]_T$ for g_1^x, g_2^x and g_T^x, respectively. For any group element of the form $g^x = [x]$, one can simply compute $[ax] = (g^x)^a$ for any given field element $a \in \mathbb{F}$ just by simple exponentiation. This notation is slightly overloaded as $[n]$ also denotes the set of integers from 1 to n; the two uses are distinguished by context—group elements always carry a subscript ($[x]_1$, $[x]_2$, $[x]_T$), while integer ranges are written without one ($[n]$, $[m]$, etc.).

Essentially, a pairing equation of the form $e(a, b) = e(c, d)$ enables checking the bilinear equation $ab = cd$ in the exponent, that is, when these elements are not available in the clear, but only the exponent (for security/privacy reasons).

Optimizing Number of Pairings with Batch Verification using Random Linear Combination Since pairing is computational expensive, we attempt to optimize it. Essentially for multiple pairing equations with a *common element in both sides* can be merged into one single pairing equation via random linear combination. For example, if we have the following equations:

$$\begin{aligned} e(a_1, b) &= e(c, d_1) \\ e(a_2, b) &= e(c, d_2) \\ &\cdots \\ e(a_n, b) &= e(c, d_n) \end{aligned}$$

This means we are checking the following bilinear equations in the exponent:

$$\begin{aligned} a_1 \cdot b &= c \cdot d_1 \\ a_2 \cdot b &= c \cdot d_2 \\ &\cdots \\ a_n \cdot b &= c \cdot d_n \end{aligned}$$

One can argue that with very high probability, these equations are equivalent to the following equations for *uniform random* $\chi_1, \ldots, \chi_n$

$$\chi_1 \cdot a_1 \cdot b + \chi_2 \cdot a_2 \cdot b + \ldots \chi_n \cdot a_n \cdot b = \chi_1 \cdot c \cdot d_1 + \chi_2 \cdot c \cdot d_2 + \ldots + \chi_n \cdot c \cdot d_n$$

which is the same as:

$$(\chi_1 \cdot a_1 + \chi_2 \cdot a_2 + \ldots \chi_n \cdot a_n) \cdot b = c \cdot (\chi_1 \cdot d_1 + \chi_2 \cdot d_2 + \ldots + \chi_n \cdot d_n)$$

Therefore, using a single pairing equation we can verify this in the exponent:

$$e(a_1^{\chi_1} \cdot \ldots \cdot a_n^{\chi_n}, b) = e(c, d_1^{\chi_1} \cdot \ldots \cdot d_n^{\chi_n})$$

which is equilavent to:

$$e(a_1, b)^{\chi_1} \cdot e(a_2, b)^{\chi_2} \cdot \ldots \cdot e(a_n, b)^{\chi_n} = e(c, d_2)^{\chi_1} \cdot e(c, d_2)^{\chi_2} \cdot \ldots \cdot e(c, d_n)^{\chi_n}$$

It is important to note that this is only possible because there are common elements (b and c in this example) on both sides.

2.2 Succinct (Zero-knowledge) Proof Systems

A succinct proof system for any NP relation $\mathfrak{R}$ allows a prover who possesses a statement-witness pair $(\mathsf{stmt}, \mathsf{wit}) \in \mathfrak{R}$—where wit is long—to produce a short proof π, such that a verifier can verify only with (stmt, π) that there exists a long witness wit such that $(\mathsf{stmt}, \mathsf{wit}) \in \mathfrak{R}$. Not only the proof size, but the verifier's computation does not grow with the size of wit. Soundness guarantees that, it is computationally hard for a cheating prover to produce a false proof π' such that the verifier accepts (stmt, π') yet $(\mathsf{stmt}, \cdot) \notin \mathfrak{R}$. If the proof system is also zero-knowledge, then π should not reveal any additional information about wit, which is not already known from stmt itself. We use proof systems which are not required to be zero-knowledge, though upgrading them to zero-knowledge can virtually be done for fee. A proof system has the following syntax:

- $\mathsf{SP.Setup} \to \mathsf{ProofPar}$. The setup outputs a set of parameters $\mathsf{ProofPar}$.
- $\mathsf{SP.Prove}(\mathsf{ProofPar}, \mathsf{stmt}, \mathsf{wit}) \to \pi$. The prover's algorithm produces a short proof π using the statement and a witness.
- $\mathsf{SP.Verify}(\mathsf{ProofPar}, \mathsf{stmt}, \pi) \to \mathsf{dec\_sp}$. The verifier's algorithm checks the proof with respect to the statement and outputs a decision bit.

We remark that, stmt can also be empty, in which case we denote $\mathsf{stmt} = \varepsilon$.

2.3 Multiplicative Subgroup in a Field

We will be using polynomial arithmetic over a given finite field $\mathbb{F}$. For technical reason, we use a multiplicative sub-group $\mathbb{H} \subset \mathbb{F}$ of size $|\mathbb{H}| = n + 1$ for some integer n generated by the $(n+1)$-th root of unity $\omega \in \mathbb{F}$, such that $\omega^{|\mathbb{H}|} = \omega^{n+1} = 1$. We use $\mathbb{H}$ as the evaluation points for a n-degree polynomial $P(x)$, and define it as $P(\omega^i) = y_i$ for all $i \in [n+1]$. Alternatively, we can write the polynomial in the Lagrange bases as $P(x) = \sum_{i\in[n+1]} y_i \cdot L_i(x)$ where $L_i(x)$ is the i-th Lagrange polynomial[4] over $\mathbb{H}$ and is defined as:

$$L_i(x) := \frac{\omega^i}{n+1} \cdot \frac{x^{n+1} - 1}{x - \omega^i}$$

Also, we will be using the vanishing polynomial $Z(x) = \prod_{i\in[n+1]}(x - \omega^i)$, which is 0 for any element in $\mathbb{H}$.

2.4 Generalized Sumcheck

We use generalized sumcheck theorem from [RZ21]. It says that, for $n+1$-degree polynomials $A(x)$ and $B(x)$ over field $\mathbb{F}$ (with corresponding subgroup $\mathbb{H}$), where $A(x) = \sum_{i\in[n+1]} a_i \cdot L_i(x)$ and $B(x) = \sum_{i\in[n+1]} b_i \cdot L_i(x)$ it holds that:

$$A(x) \cdot B(x) = \frac{\sum_i a_i \cdot b_i}{n+1} + Q_x(x) \cdot x + Q_Z(x) \cdot Z(x),$$

where both Q_x and Q_Z are polynomials, such *only*[5] Q_x has degree $\leqslant n-1$ defined as

$$Q_x(x) = \sum_i a_i \cdot b_i \cdot \frac{L_i(x) - L_i(0)}{x},$$

$$Q_Z(x) = \sum_i a_i \cdot b_i \cdot \frac{L_i^2(x) - L_i(x)}{Z(x)} + \sum_{i\neq j} a_i \cdot b_j \cdot \frac{L_i(x) \cdot L_j(x)}{Z(x)}.$$

[4] Note that $L_i(x)$ is defined in a way such that it is 1 only when $x = \omega^i$ and is 0 for all other ω^j.

[5] In hinTS [GJM+24], this was wrongly mentioned as both Q_x and Q_z to have degree $n-1$. Looking ahead, imposing the degree constraint only on Q_x helps reducing the number of pairings in the verification.

3 hinTS: A Silent Threshold Signature [GJM+24]

3.1 BLS Signature

The hinTS STS is built on BLS signature [BLS01], which we describe below:

BLS Signature: Ingredients, Parameters, Notations, and Setting

- Asymmetric pairing group parameters $\mathsf{PairPar} := (\mathbb{F}, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2, g_T)$ with pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$;
- Hash function $\mathcal{H} : \{0,1\}^* \to \mathbb{G}_2$;

BLS Signature: Construction

- $\mathsf{BLS.Gen} \to (\mathsf{sk}, \mathsf{pk})$:
  - $\mathsf{sk} \leftarrow_{\$} \mathbb{F}$
  - $\mathsf{pk} := [\mathsf{sk}]_1$
- $\mathsf{BLS.Sign}(\mathsf{sk}, \mathsf{msg}) \to \sigma$:
  - $\sigma := \mathcal{H}(\mathsf{msg})^{\mathsf{sk}}$
- $\mathsf{BLS.Verify}(\mathsf{pk}, \mathsf{msg}, \sigma) \to \mathsf{dec\_bit}$:
  - $\mathsf{dec\_bit} := (e(\mathsf{pk}, \mathcal{H}(\mathsf{msg})) \stackrel{?}{=} e([1]_1, \sigma))$

Security. The security of BLS guarantees that no practical adversarsary can forge a signature without knowing the secret key. More formally, BLS is *existentially unforgeable* under chosen-message attacks (EU-CMA) in the random oracle model, under the co-computational Diffie–Hellman (co-CDH) assumption in the bilinear group [BLS01]. Concretely, the verification equation $e(\mathsf{pk}, \mathcal{H}(\mathsf{msg})) = e([1]_1, \sigma)$ is sound because producing a valid σ for a fresh message without knowing sk requires computing $\mathcal{H}(\mathsf{msg})^{\mathsf{sk}}$ from $[\mathsf{sk}]_1$ and $\mathcal{H}(\mathsf{msg})$—an instance of co-CDH.

Efficiency. Key generation costs one scalar multiplication in $\mathbb{G}_1$. Signing costs one hash-to-curve evaluation in $\mathbb{G}_2$ plus one scalar multiplication. Verification costs exactly two pairings. The signature is a single $\mathbb{G}_2$ element (48 bytes on BLS12-381, 32 bytes on BN254). A critical property exploited by hinTS (similar to BLS multi-signatures) is *aggregation*: k BLS signatures on the same message can be multiplied into a single aggregate $\sigma = \prod_i \sigma_i$, which verifies against the aggregated key with the same two pairings.

3.2 KZG Polynomial Commitment

Notation We require a sub-routine $\mathsf{ExpEval}$ which takes d many group elements $([\alpha], [\alpha^2], \ldots, [\alpha^d])$ for some field element $\alpha \in \mathbb{F}$, powers of which are in the exponents, and then execute an MSM computation to output any polynomial evaluation $[P(\alpha)]$ in the exponent for a given polynomial $P(x)$ of degree $\leq d$ over field $\mathbb{F}$. The sytanx is as follows: $[P(\alpha)] \leftarrow \mathsf{ExpEval}(P(x), ([\alpha], [\alpha^2], \ldots, [\alpha^d]))$.

The KZG polynomial commitment scheme given below works for all polynomials of degree upto d over $\mathbb{F}$.

KZG Polynomial Commitment: Ingredients, Parameters, Notations, and Setting

- Asymmetric pairing group parameters $\mathsf{PairPar} := (\mathbb{F}, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2, g_T)$ with pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$;
- Hash function $\mathcal{H} : \{0,1\}^* \to \mathbb{G}_2$;

- $\mathsf{src\_grp} \in \{1, 2\}$ indicates over which group the commitment is being computed.

KZG Polynomial Commitment: Construction

- $\mathsf{KZG.Gen} \to \mathsf{KZGPar}$:
  - $\tau \leftarrow_{\$} \mathbb{F}$
  - $\boldsymbol{s}_1 := \{[\tau]_1, [\tau^2]_1, \ldots, [\tau^d]_1\}$
  - $\boldsymbol{s}_2 := \{[\tau]_2, [\tau^2]_2, \ldots, [\tau^d]_2\}$
  - $\mathsf{KZGPar} := (\boldsymbol{s}_1, \boldsymbol{s}_2)$
- $\mathsf{KZG.Com}(\mathsf{KZGPar}, P(x), \mathsf{src\_grp}) \to [P(\tau)]_{\mathsf{src\_grp}}$:
  - $(\boldsymbol{s}_1, \boldsymbol{s}_2) := \mathsf{KZGPar}$
  - $[P(\tau)]_{\mathsf{src\_grp}} := \mathsf{ExpEval}(P(x), \boldsymbol{s}_{\mathsf{src\_grp}})$
    (Note: ExpEval and KZG.Com can be used interchangeably)
- $\mathsf{KZG.Open}(\mathsf{KZGPar}, P(x), \alpha, \mathsf{src\_grp}) \to [Q^{\alpha}(\tau)]_{\mathsf{src\_grp}}$:
  - $(\boldsymbol{s}_1, \boldsymbol{s}_2) := \mathsf{KZGPar}$
  - $O^{\alpha}(x) := \frac{P(x)-P(\alpha)}{x-\alpha}$
  - $[O^{\alpha}(\tau)]_{\mathsf{src\_grp}} := \mathsf{ExpEval}(O^{\alpha}(x), \boldsymbol{s}_{\mathsf{src\_grp}})$
- $\mathsf{KZG.Ver}(\mathsf{KZGPar}, [P(\tau)]_{\mathsf{src\_grp}}, \alpha, \beta, [O^{\alpha}(\tau)]_{\mathsf{src\_grp}}, \mathsf{src\_grp}) \to \mathsf{dec\_bit}$:
  - If $\mathsf{src\_grp} = 1$, then $\mathsf{dec\_bit} := \left(e([P(\tau)]_1/[\beta]_1, [1]_2) \stackrel{?}{=} e([O^{\alpha}(\tau)]_1, \boldsymbol{s}_2{}^{(1)}/[\alpha]_2)\right)$
  - If $\mathsf{src\_grp} = 2$, then $\mathsf{dec\_bit} := \left(e([1]_1, [P(\tau)]_2/[\beta]_2) \stackrel{?}{=} e(\boldsymbol{s}_1{}^{(1)}/[\alpha]_1, [O^{\alpha}(\tau)]_2)\right)$

Security. KZG is computationally *binding* under the d-Strong Diffie–Hellman (d-SDH) assumption: given the SRS $([1]_1, [\tau]_1, \ldots, [\tau^d]_1)$, no efficient adversary can find $(P, \alpha, \beta, \beta', O, O')$ with $\beta \neq \beta'$ such that both $\mathsf{KZG.Ver}$ calls accept for the same commitment $[P(\tau)]_1$ at the same point α [KZG10]. KZG, as presented here, is not *hiding*—commitments are deterministic—but this is irrelevant for hinTS, where the committed polynomials ($B(x)$, $W(x)$, $Q_x(x)$, $Q_Z(x)$) are public or become public upon aggregation.[6] The polynomial evaluation point τ must remain hidden; it is destroyed after the trusted setup ceremony.

Efficiency. Committing to a polynomial of degree d costs one MSM of size d in $\mathbb{G}_{\mathsf{src\_grp}}$. Generating an opening proof costs one polynomial division (to compute $O^{\alpha}(x)$) plus one MSM of size d. Verifying an opening costs exactly two pairings, regardless of d. Both a commitment and an opening proof are a single group element—constant size in d. This is the core property that makes hinTS scalable: the hints $[\mathsf{sk}_i \cdot H_{i,k}(\tau)]_1$ are constant-size group elements that encode degree-n polynomial information, yet each opens in two pairings, which can further be aggregated as discussed below.

Homomorphism and optimization It is easy to observe that KZG commitments are homomorphic: given two commitments $[P_1(\tau)]_1$ and $[P_2(\tau)]_1$, one can compute the commitment of $P(x) = \chi_1 \cdot P_1(x) + \chi_2 \cdot P_2(x)$ as $[P(\tau)]_1 := [P_1(\tau)]_1^{\chi_1} \cdot [P_2(\tau)]_1^{\chi_2}$. Now, this yields a useful optimization technique if both these polynomials are opened to the same input α, and the corresponding openings are $[O_1^{\alpha}(\tau)]_1$ and $[O_2^{\alpha}(\tau)]_1$, then instead of generating separate openings, it is possible to just generate a single opening $\boldsymbol{O} := [O_1^{\alpha}(\tau)]_1^{\chi_1} \cdot [O_2^{\alpha}(\tau)]_1^{\chi_2}$. This not only reduces the size from two elements to one, but also improves the verification time, as instead of checking two pairing equations, one equation suffices:

$$\left(e([P(\tau)]_1/[\beta]_1, [1]_2) \stackrel{?}{=} e(\boldsymbol{O}, \boldsymbol{s}_2{}^{(1)}/[\alpha]_2)\right)$$

[6] One can easily make it hiding with a randomized construction—we refer to the original paper [KZG10].

where $\beta = P(\alpha)$. Furthermore, it is also possible to just create a single opening for the combined polynomial $P(x)$, say $[O^P_\tau(\tau)]_1$, where

$$O^\alpha(x) = \frac{P(x) - P(\alpha)}{x - \alpha} = \frac{\chi_1(P_1(x) - P_1(\alpha)) + \chi_2(P_2(x) - P_2(\alpha))}{x - \alpha} = \chi_1 \cdot O^\alpha_1(x) + \chi_2 \cdot O^\alpha_2(x)$$

Clearly, due to homomorphism, the opening of $P(x)$ at α is exactly the same as the value homomorphically computed from the two separate openings.

Finally, the random values χ_1, χ_2 can be related as $\chi_2 = {\chi_1}^2$ and so on, as this preserves linear independence.

3.3 Plonkish Zero Test and Sumcheck

The hinTS construction uses a Plonkish zero-test and sumcheck [GWC19] with KZG commitments. Consider a field $\mathbb{F}$, and the corresponding multipliactive subgroup $\mathbb{H}$ of size $n+1$ generated by $n+1$-th root of unity ω (cf. Sec 2.3). Now, Plonkish zero-test allows a prover to succinctly prove that for all $x \in \mathbb{H}$ $P(x) = 0$. Clearly, this means $P(x)$ is divisible by $Z(x)$. Hence there exists a quotient polynomial $Q(x)$, such that $P(x) = Z(x)Q(x)$. To prove the statement succinctly, the prover uses a polynomial commitment. For example, KZG commitments $[P(\tau)]_1, [Z(\tau)]_1, [Q(\tau)]_1$ (assume a KZG setup beforehand) maybe used. The steps are as follows:

Plonkish Zero Test: Ingredients, Parameters, Notations, and Setting

- Asymmetric pairing group parameters $\mathsf{PairPar} := (\mathbb{F}, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2, g_T)$ with pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$;
- Hash function $\mathcal{H} : \{0,1\}^* \to \mathbb{F}$
- The vanishing polynomial for $\mathbb{H}$: $Z(x)$.

Plonkish Zero Test: Construction

- $\mathsf{ZT.SP.Setup} \to \mathsf{ProofPar}$:
  - $\mathsf{KZGPar} \leftarrow \mathsf{KZG.Gen}$
  - $\mathsf{ProofPar} := \mathsf{KZGPar}$
- $\mathsf{ZT.SP.Prove}(\mathsf{ProofPar}, \varepsilon, P(x)) \to \pi$:

  $(\forall\ x \in \mathbb{H} :\ \ P(x) = 0)$

  (Computing the quotient polynomial)
  - $Q(x) := P(x)/Z(x)$

  (Generating KZG commitments of $P(x), Q(x)$)
  - $([P(\tau)]_1) := \mathsf{KZG.Com}(\mathsf{KZGPar}, P(x), 1)$
  - $([Q(\tau)]_1) := \mathsf{KZG.Com}(\mathsf{KZGPar}, Q(x), 1)$

  (Generating the Fiat-Shamir challenge point)
  - $r := \mathcal{H}([P(\tau)]_1, [Q(\tau)]_1)$

  (Generate the openings at r for both the polynomials)
  - $[O^r_P(\tau)]_1 := \mathsf{KZG.Open}(\mathsf{KZGPar}, P(x), r, 1)$
  - $[O^r_Q(\tau)]_1 := \mathsf{KZG.Open}(\mathsf{KZGPar}, Q(x), r, 1)$
  - $\pi := \big(([P(\tau)]_1, [Q(\tau)]_1), ([O^r_P(\tau)]_1, [O^r_Q(\tau)]_1), (P(r), Q(r))\big)$
- $\mathsf{ZT.SP.Verify}(\mathsf{ProofPar}, \varepsilon, \pi) \to \mathsf{dec\_sp}$:

- $([P(\tau)]_1, [Q(\tau)]_1), ([O_P^r(\tau)]_1, [O_Q^r(\tau)]_1), (P(r), Q(r)) := \pi$

  (Generating the Fiat-Shamir challenge point)
- $r := \mathcal{H}([P(\tau)]_1, [Q(\tau)]_1)$

  (Verifying the quotient)
- $\mathsf{dec\_q} := (P(r) \stackrel{?}{=} Q(r)Z(r))$

  (Verifying the openings)
- $\mathsf{dec\_op^P} := \mathsf{KZG.Ver}(\mathsf{KZGPar}, [P(\tau)]_1, r, P(r), [O_P^r(\tau)]_1, 1)$
- $\mathsf{dec\_op^Q} := \mathsf{KZG.Ver}(\mathsf{KZGPar}, [Q(\tau)]_1, r, Q(r), [O_Q^r(\tau)]_1, 1)$

  (Computing the final decision bit)
- $\mathsf{dec\_sp} := \mathsf{dec\_op^P} \ \wedge \ \mathsf{dec\_op^Q}$

Next, this can be extended to support a more advanced check, named as sumcheck. The prover wants to succinctly prove that $\sum_{x\in\mathbb{H}} P(x) = 0$. The sumcheck protocol uses ideas from the zero test. The main idea is to essentially reduce sum-check to zero-test recursively. In particular, consider another polynomial $PS(x)$ (PS stands for partial sum) defined as follows:

- For all $i \in \{2, \ldots, n+2\}$: $PS(\omega^i) = \sum_{j=1}^{i-1} P(\omega^j)$

Evidently, we have that:

Condition-1 For all $x \in \mathbb{H}$: $PS(x\omega) - PS(x) - P(x) = 0$

Condition-2 $PS(\omega^{n+2}) = PS(\omega) = \sum_{x\in\mathbb{H}} P(x) = 0$

So the verification must check these two conditions. The proof system works as follows:

Plonkish Sumcheck: Ingredients, Parameters, Notations, and Setting

- Asymmetric pairing group parameters $\mathsf{PairPar} := (\mathbb{F}, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2, g_T)$ with pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$;
- Hash function $\mathcal{H} : \{0,1\}^* \to \mathbb{F}$
- The vanishing polynomial for $\mathbb{H}$: $Z(x)$.
- The Lagrange bases polynomials over $\mathbb{H}$: $L_1(x), \ldots, L_{n+1}(x)$.

Plonkish Sumcheck: Construction

- $\mathsf{SCH.SP.Setup} \to \mathsf{ProofPar}$:
  - $\mathsf{KZGPar} \leftarrow \mathsf{KZG.Gen}$
  - $\mathsf{ProofPar} := \mathsf{KZGPar}$
- $\mathsf{SCH.SP.Prove}(\mathsf{ProofPar}, \varepsilon, P(x)) \to \pi$:

  $(\sum_{x\in\mathbb{H}} P(x) = 0)$

  (Computing $PS(x)$ and $Q(x)$)
  - $PS(x) := \sum_{i\in\{2,\ldots,n+1\}} (\sum_{j=1}^{i-1} P(\omega^j)) L_i(x)$
  - $Q(x) := \frac{PS(x\omega) - PS(x) - P(x)}{Z(x)}$

  (Generating KZG commitments of $P(x), PS(x), Q(x)$)
  - $([P(\tau)]_1) := \mathsf{KZG.Com}(\mathsf{KZGPar}, P(x), 1)$

- $([PS(\tau)]_1) := \mathsf{KZG.Com}(\mathsf{KZGPar}, PS(x), 1)$
- $([Q(\tau)]_1) := \mathsf{KZG.Com}(\mathsf{KZGPar}, Q(x), 1)$

  (Generating the Fiat-Shamir challenge point)
- $r := \mathcal{H}([P(\tau)]_1, [PS(\tau)]_1, [Q(\tau)]_1)$

  (Generate the openings at required points $P(r), PS(r), PS(r\omega), PS(\omega), Q(r)$)
- $[O^r_P(\tau)]_1 := \mathsf{KZG.Open}(\mathsf{KZGPar}, P(x), r, 1)$
- $[O^r_{PS}(\tau)]_1 := \mathsf{KZG.Open}(\mathsf{KZGPar}, PS(x), r, 1)$
- $[O^{r\omega}_{PS}(\tau)]_1 := \mathsf{KZG.Open}(\mathsf{KZGPar}, PS(x), r\omega, 1)$
- $[O^{\omega}_{PS}(\tau)]_1 := \mathsf{KZG.Open}(\mathsf{KZGPar}, PS(x), \omega, 1)$
- $[O^r_Q(\tau)]_1 := \mathsf{KZG.Open}(\mathsf{KZGPar}, Q(x), r, 1)$
- $\pi := \begin{pmatrix} ([P(\tau)]_1, [PS(\tau)]_1, [Q(\tau)]_1), \\ ([O^r_P(\tau)]_1, [O^r_{PS}(\tau)]_1, [O^{r\omega}_{PS}(\tau)]_1, [O^{\omega}_{PS}(\tau)]_1, [O^r_Q(\tau)]_1), \\ (P(r), PS(r), PS(r\omega), Q(r) \end{pmatrix}$

- $\mathsf{SCH.SP.Verify}(\mathsf{ProofPar}, \varepsilon, \pi) \to \mathsf{dec\_sp}$:
  - $\begin{pmatrix} ([P(\tau)]_1, [PS(\tau)]_1, [Q(\tau)]_1), \\ ([O^r_P(\tau)]_1, [O^r_{PS}(\tau)]_1, [O^{r\omega}_{PS}(\tau)]_1, [O^{\omega}_{PS}(\tau)]_1, [O^r_Q(\tau)]_1), \\ (P(r), PS(r), PS(r\omega), Q(r) \end{pmatrix} := \pi$

    (Generating the Fiat-Shamir challenge point)
  - $r := \mathcal{H}([P(\tau)]_1, [PS(\tau)]_1, [Q(\tau)]_1)$

    (Verifying the quotient)
  - $\mathsf{dec\_q} := \left(Q(r) \stackrel{?}{=} \frac{PS(r\omega) - PS(r) - P(r)}{Z(r)}\right)$

    (Verifying the openings)
  - $\mathsf{dec\_op}^{\mathsf{P}} := \mathsf{KZG.Ver}(\mathsf{KZGPar}, [P(\tau)]_1, r, P(r), [O^r_P(\tau)]_1, 1)$
  - $\mathsf{dec\_op}^{\mathsf{PS_r}} := \mathsf{KZG.Ver}(\mathsf{KZGPar}, [PS(\tau)]_1, r, PS(r), [O^r_{PS}(\tau)]_1, 1)$
  - $\mathsf{dec\_op}^{\mathsf{PS_{r\omega}}} := \mathsf{KZG.Ver}(\mathsf{KZGPar}, [PS(\tau)]_1, r\omega, PS(r\omega), [O^{r\omega}_{PS}(\tau)]_1, 1)$
  - $\mathsf{dec\_op}^{\mathsf{PS}_\omega} := \mathsf{KZG.Ver}(\mathsf{KZGPar}, [PS(\tau)]_1, \omega, 0, [O^{\omega}_{PS}(\tau)]_1, 1)$
  - $\mathsf{dec\_op}^{\mathsf{Q}} := \mathsf{KZG.Ver}(\mathsf{KZGPar}, [Q(\tau)]_1, r, Q(r), [O^r_Q(\tau)]_1, 1)$

    (Computing the final decision bit)
  - $\mathsf{dec\_sp} := \mathsf{dec\_op}^{\mathsf{P}} \ \wedge \ \mathsf{dec\_op}^{\mathsf{PS_r}} \ \wedge \ \mathsf{dec\_op}^{\mathsf{PS_{r\omega}}} \ \mathsf{dec\_op}^{\mathsf{PS}_\omega} \ \wedge \ \mathsf{dec\_op}^{\mathsf{Q}}$

Note that the above protocols easily extend to a **product of polynomials** $P(x) = \prod_k P_k(x)$. Furthermore, though we use KZG commitments over $\mathbb{G}_1$, any of them can work with either $\mathbb{G}_1$ or $\mathbb{G}_2$. For products, we may use commitments of individual $P_i(x)$ in different groups too. Looking ahead, we need to use one of the commitment (for $B(x)$) in $\mathbb{G}_2$, while everything else stays in $\mathbb{G}_1$.

Security (zero-test). Soundness follows from two facts. First, if $P \not\equiv 0 \pmod{Z(x)}$ then $P(r) \neq Q(r)Z(r)$ at a uniformly random $r \in \mathbb{F}$ except with probability $\deg(P)/|\mathbb{F}| = \mathsf{negl}(\kappa)$ by the Schwartz–Zippel lemma. Second, KZG binding ensures the prover cannot open $[P(\tau)]_1$ or $[Q(\tau)]_1$ to values inconsistent with the committed polynomials at the challenge r. Together, a cheating prover succeeds with probability at most $\deg(P)/|\mathbb{F}|$, which is negligible.

Security (sumcheck). The partial-sum polynomial $PS(x)$ is uniquely determined by the defining conditions $PS(\omega) = 0$ and $PS(\omega^i) = \sum_{j=1}^{i-1} P(\omega^j)$ for $i \geq 2$. The check $PS(\omega) = 0$ (Condition 2) anchors the sum; the quotient check at a random r (Condition 1) is again sound by Schwartz–Zippel plus KZG binding, with error $O(n/|\mathbb{F}|) = \mathsf{negl}(\kappa)(\kappa)$.

Efficiency. For both protocols, the prover computes $O(n)$ field operations to evaluate and divide polynomials, plus $O(1)$ MSMs of size n (for KZG commitments and openings). The *verifier* performs only a constant number of field evaluations and KZG verifications, each costing two pairings—independent of n. Concretely:

	Prover MSMs	Verifier pairings	Proof size
Zero-test	2 (size n)	4	$2\ \mathbb{G}_1 + 2\ \mathbb{G}_1 + 2\ \mathbb{F}$
Sumcheck	3 (size n)	10	$3\ \mathbb{G}_1 + 5\ \mathbb{G}_1 + 4\ \mathbb{F}$

Note that, after the KZG opening merging optimization (Section 3), the sumcheck prover opening cost collapses from 5 separate openings to 1 merged opening, reducing the verifier pairing count correspondingly.

3.4 The hinTS construction

The protocol is split into a few parts.

1. **SRS Generation**, which is executed by a *ceremony*—we refrain from describing it here, and is merged with HieroTSS construction in Section 5.
2. **Local Setup**, which is executed once by each party after SRS Generation. It can be further split into two parts:
   (a) **Key-independent** Local Setup—this is common for all parties, but depends on the SRS.
   (b) **Key-dependent** Local Setup—this involves each party's secret key, and hints are generated in this step.
3. **Partial / Verification** happens using each party's individual secret/verification key and the message (and signature)—this is exactly similar to BLS.
4. **Aggregated Key Generation** procedure, publicly executed by any party, parses the hints, verifies them, and then generates the aggregated public key. This is a public procedure and can be executed by anyone.
5. **Signature Aggregation** is also done publicly by any party, and it takes place for every signing.
6. **Verification** Finally, the compact signature is verified publicly by anyone.

3.4.1 Technical Overview : hinTS Silents Threshold Signatures

For n parties $P_1, \ldots, P_n$ and a threshold t_{th} Let us denote the weight vector $\boldsymbol{w} := (w_1 \ \cdots \ w_n)$, whereas the total weight is denoted $\widehat{w} = \sum_{i=1}^{n} w_i$. The construction uses bilinear pairing, with parameters $\mathsf{PairPar} := (\mathbb{F}, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2, g_T)$ with map $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$. Now, let us briefly recall the strawman approach of constructing a weighted silent threshold signature from BLS multi-signatures in this setting. Each party P_i has a key pair $(\mathsf{sk}_i, [\mathsf{sk}_i]_1)$. We denote the secret-key vector $\mathbf{sk} = (\mathsf{sk}_1 \ \cdots \ \mathsf{sk}_n)$ and public key vector $[\mathbf{sk}]_\mathbf{1} = ([\mathsf{sk}_1]_1 \ \cdots \ [\mathsf{sk}_n]_1)$. Each party P_i who wants to participate (let us denote the participating set by $\mathcal{S} \subseteq [n]$) produces their own partial signatures $\sigma_i := \mathcal{H}(\mathsf{msg})^{\mathsf{sk}_i}$. The aggregator (any party among the signers, or from outside, can play this role, as no trust assumption is needed on the aggregator) computes (i) aggregated signature $\sigma := \prod_{i \in \mathcal{S}} \sigma_i = \mathcal{H}(\mathsf{msg})^{\sum_{i \in \mathcal{S}} \mathsf{sk}_i}$ and (ii) the aggregated verification key $\mathsf{AVK} := \prod_{i \in \mathcal{S}} [\mathsf{sk}_i]_1$.[7] The final signature is $\mathsf{ASIG} := (\sigma, \mathsf{AVK}, \boldsymbol{b})$ where $\boldsymbol{b}$ is a binary vector of size n which indicates the participating set—it has 1 in the i-th place if and only if $i \in \mathcal{S}$ and 0 otherwise. The verifier, given the signature $\mathsf{ASIG} = (\sigma, \mathsf{AVK}, \boldsymbol{b})$ and all verification keys $[\mathbf{sk}]_\mathbf{1}$ performs the following checks:

1. Computes the aggregated verification keys of the participating sets using $\boldsymbol{b}$ as $\prod_{i:b_i=1} [\mathsf{sk}_i]_1$ and then checks if it matches with AVK from ASIG.

[7] We note that the aggregator doesn't need to compute the aggregated verification key, as the verifier would anyway compute it. However, for the ease of exposition, we make the aggregator compute it as well. Looking ahead, in hinTS the verifier would *not* compute it, so the aggregator will have to execute this step, and prove that this is correctly computed.

2. Verify σ by computing two pairings as: $e(\sigma, [1]_2) = e(\mathcal{H}(\mathsf{msg}), \mathsf{AVK})$.

3. Check $w := \langle \boldsymbol{b}, \boldsymbol{w} \rangle \geq t_{\mathsf{th}}$.

Now, we note that:

- The verifier needs to compute AVK, which requires computation proportional to the size of set $\mathcal{S}$.
- The verifier needs to remember all n verification keys—in the blockchain setting, this means the state has to store all of them, increasing gas cost.
- The signature must contain the information about the participating set, that is $\boldsymbol{b}$ and hence grows with n.

hinTS resolves all of these. The idea is to use succinct proofs to ensure that all three checks can be performed by the verifier while its computation and signature size are all constant. The final signature ASIG would look like $(\sigma, \mathsf{AVK}, \boldsymbol{\Pi})$ where $\boldsymbol{\Pi}$ is a succinct proof for a statement witness pair $(\mathsf{stmt}, \mathsf{wit})$ that ensures that the Step 1 and Step 3 of the verification are correct, even when $\boldsymbol{b}, \boldsymbol{w}$ or $[\mathsf{sk}]_1$ are not given. In particular, in hinTS, the aggregator produces a succinct proof $\boldsymbol{\Pi}$ for $\mathsf{stmt} = (\sigma, \mathsf{AVK}, \mathsf{msg}, t)$ and $\mathsf{wit} = (\boldsymbol{b}, [\mathsf{sk}]_1, \boldsymbol{w})$ where $(\mathsf{stmt}, \mathsf{wit}) \in \mathfrak{R}$ implies all three checks succeed.

Using KZG Now, we do need to somehow communicate the witnesses to the verifier succinctly. This is done using KZG commitments, which can produce a succinct commitment of any vector. We consider the polynomials over the field $\mathbb{F}$ corresponding to the vectors $\boldsymbol{b}$, $\boldsymbol{w}$, $\boldsymbol{sk}$.[8] Following the standard methods, we consider a multiplicative subgroup $\mathbb{H} \subset \mathbb{F}$, generated by a root ω of unity, such that $\omega^{n+1} = 1$ (which implies $|\mathbb{H}| = n + 1$). We will work over the $n + 1$ evaluation points $\{\omega, \omega^2, \ldots, \omega^n, \omega^{n+1} = 1\}$. So, to convert the vectors to polynomials of degree n first we extend the vectors $\boldsymbol{b}$, $\boldsymbol{sk}$ and $\boldsymbol{w}$ to contain $n + 1$ elements, where $b_{n+1} := 1$ $\mathsf{sk}_{n+1} := 0$ and $w_{n+1} := -\langle \boldsymbol{b}, \boldsymbol{w} \rangle = -w$.[9] Then we define the corresponding polynomials over $\mathbb{H}$:

- $B(\omega^i) = b_i$ for $i \in [n + 1]$
- $SK(\omega^i) = \mathsf{sk}_i$ for $i \in [n + 1]$
- $W(\omega^i) = w_i$ for $i \in [n + 1]$

Which can equivalently be written in Lagrange bases:

- $B(x) = \sum_{i \in [n+1]} b_i \cdot L_i(x)$
- $SK(x) = \sum_{i \in [n+1]} \mathsf{sk}_i \cdot L_i(x)$
- $W(x) = \sum_{i \in [n+1]} w_i \cdot L_i(x)$

Plonkish Sumcheck for Step 3 Now, since $[SK(\tau)]_2$ is not available to the aggregator in the clear, Step 1 is trickier to prove. Let us first look into Step 3, which boils down to proving the following equation:

$$\sum_{x \in \mathbb{H}} B(x) \cdot W(x) = 0 \tag{1}$$

and as long as w is included in the proof $\boldsymbol{\Pi}$—given which the verifier can check $w \geq t_{\mathsf{th}}$. For this, we use the Plonkish sumcheck, described in Section 3.3 for the product polynomial $P(x) := B(x)W(x)$. Using $\mathsf{SCH.SP.Prove}$ on $P(x) = B(x)W(x)$ the aggreagtor generates a proof π_3 which has constant size.

[8] We consider vectors over the field. Looking ahead, since the vector $\boldsymbol{sk}$ is not available in the clear to the aggregator, (but only as the corresponding group elements $[\mathsf{sk}]_1$) this would require more involved techniques, leading to release of "hints".

[9] We note an issue that w is not known when the weight vector is committed. However, for ease of exposition here we assume that w is already known. In the actual construction, we handle it in a slightly different manner, by first committing to $\boldsymbol{w}$ with $w_{n+1} = 0$, and later adjusting it with $-w$.

Generalized Sumcheck for Step 1 Finally, we look into Step 1. First we notice that if all computations are correct, then the following holds:

$$\left[\sum SK(x)B(x)\right]_1 = \mathsf{AVK} \tag{2}$$

which the aggregator ought to prove succinctly. Nevertheless, the main issue here is that the aggregator does not have access to $SK(x)$. For this part, we use the generalized sumcheck (cf. Section 2.4), which ensures that, there exists two polynomials $Q_x(x)$ and $Q_Z(x)$, among them $Q_x(x)$ must be of degree at most $n-1$ such that:

$$SK(x)B(x) = \frac{\mathsf{ask}}{n+1} + Q_x(x)\cdot x + Q_Z(x)\cdot Z(x) \tag{3}$$

where $\mathsf{AVK} = [\mathsf{ask}]_1$ ($Z(x)$ is the vanishing polynomial over $\mathbb{H}$). Furthermore, $Q_x(x)$ and $Q_Z(x)$ are computed as:

$$Q_x(x) = \sum_{i\in[n+1]} \mathsf{sk}_i \cdot b_i \cdot \frac{L_i(x) - L_i(0)}{x},$$

$$Q_Z(x) = \sum_{i\in[n+1]} \mathsf{sk}_i \cdot b_i \cdot \frac{L_i^2(x) - L_i(x)}{Z(x)} + \sum_{i,j\in[n+1];i\neq j} \mathsf{sk}_i \cdot b_j \cdot \frac{L_i(x)\cdot L_j(x)}{Z(x)}.$$

For notational convenience we define:

- $H_{i,1}(x) := \frac{L_i^2(x)-L_i(x)}{Z(x)}$
- $H_{i,2}(x) := L_i(x) - L_i(0)$
- $H_{i,3}(x) := \frac{H_{i,2}(x)}{x}$
- For $j \in [n] \setminus \{i\}$:
  - $G_{i,j}(x) := \frac{L_i(x)L_j(x)}{Z(x)}$

We rewrite $Q_x(x)$ and $Q_Z(x)$ using these notations:

$$Q_x(x) = \sum_{i\in[n+1]} \mathsf{sk}_i \cdot b_i \cdot H_{i,3}(x), \tag{4}$$

$$Q_Z(x) = \sum_{i\in[n+1]} \mathsf{sk}_i \cdot b_i \cdot H_{i,1}(x) + \sum_{i,j\in[n+1];i\neq j} \mathsf{sk}_i \cdot b_j \cdot G_{i,j}(x). \tag{5}$$

Now, see that the verification of Eq 3 can be done in the exponent using pairing as:

$$e([B(\tau)]_1, [SK(\tau)]_2) = e(\mathsf{AVK}^{1/(n+1)}, [1]_2)\cdot e([Q_x(\tau)]_1, [\tau]_2)\cdot e([Q_Z(\tau)]_1, [Z(\tau)]_2) \tag{6}$$

Among these values, $[SK(\tau)]_2, [1]_2, [\tau]_2, [Z(\tau)]_2$ can be computed before the aggregation phase, as they are independent of the participating sets.[10] The aggregator can compute $[B(\tau)]_1$, AVK easily once it knows the participating set. So, it boils down to computing $[Q_x(\tau)]_1$ and $[Q_Z(\tau)]_1$.

Again, we note that the values $[L_i(\tau)]_1, [H_{i,1}(\tau)]_1, [H_{i,2}(\tau)]_1, [H_{i,3}(\tau)]_1$ and $[G_{i,j}(\tau)]_1\}_i$ can be computed by anyone and at any time. Each party P_i owns sk_i. So, they can each compute the following values as hints:

- $[\mathsf{sk}_i \cdot L_i(\tau)]_2, [\mathsf{sk}_i \cdot H_{i,1}(\tau)]_1, [\mathsf{sk}_i \cdot H_{i,2}(\tau)]_1, [\mathsf{sk}_i \cdot H_{i,3}(\tau)]_1, \{[\mathsf{sk}_j \cdot G_{i,j}(\tau)]_1\}_{j\in[n]\setminus\{i\}}$

Now, in the pre-processing phase, which takes place before aggregation:

- Given $[\mathsf{sk}_i \cdot L_i(\tau)]_2$ for all $i \in [n]$ anyone can compute $[SK(\tau)]_2 = \prod_{i\in[n+1]}[\mathsf{sk}_i \cdot L_i(\tau)]_2$ (recall that $\mathsf{sk}_{n+1} = 0$)

[10] It is important to note that, within a pairing, exactly one value is pre-known before the aggregation.

- Given $[\mathsf{sk}_j \cdot G_{i,j}(\tau)]_1$ for all $i, j \in [n]$ and $i \neq j$, anyone can compute $\left[\sum_{j \neq i} \mathsf{sk}_j \cdot G_{i,j}(\tau)\right]_1 := \prod_{j \in \mathcal{V} \setminus \{i\}} [\mathsf{sk}_j \cdot G_{i,j}(\tau)]_1$ for all $i \in [n+1]$.

During aggregation, when $\boldsymbol{b}$ is known, then the aggregator can compute:

- $[Q_x(\tau)]_1 := \prod_{i \in [n+1]} \left([\mathsf{sk}_i \cdot H_{i,3}(\tau)]_1\right)^{b_i}$
- $[Q_Z(\tau)]_1 := \prod_{i \in [n+1]} \left([\mathsf{sk}_i \cdot H_{i,1}(\tau)]_1 \cdot \left[\sum_{j \neq i} \mathsf{sk}_j \cdot G_{i,j}(\tau)\right]_1\right)^{b_i}$

So, Step-1 proof π_1 would consist of:[11]

$$\pi_1 := ([sk(\tau)]_2, [B(\tau)]_1, [Q_x(\tau)]_1, [Q_Z(\tau)]_1)$$

Additional checks Finally, a few additional checks are required:

1. $\boldsymbol{b}$ is indeed a binary vector. Equivalently, $B(x)$'s evaluation on $\mathbb{H}$ (which are b_i) are binary values. It suffices to use the Plonkish zero-test (cf. Section 3.3) for this on the polynomial $B(x)(1 - B(x))$. Let the proof generated using $\mathsf{ZT.SP.Prove}$ on $B(x)(1 - B(x))$ be π_{zt}.
2. $b_{n+1} = 1$, or equivalently $B(\omega^{n+1}) = B(1) = 1$. This can be done easily by using a KZG opening of $[B(\tau)]_1$ on ω^{n+1}. Let us call this opening π_{n+1}
3. The degree of $Q_x(x)$ is indeed bounded by $n - 1$. This can be proved by using the identity $Q_t = x \cdot Q_x(x)$:[12]
   - Note that $Q_t = \sum_{i \in [n+1]} \mathsf{sk}_i \cdot b_i \cdot H_{i,2}(x)$
   - The aggregator produces a KZG commitment of Q_t by computing $[Q_t(\tau)]_1 := \prod_{i \in [n+1]} [\mathsf{sk}_i \cdot H_{i,2}]_1^{b_i}$, where $[\mathsf{sk}_i \cdot H_{i,2}]_1$ are also part of the hints. Let us call this part of the proof $\pi_{deg} := [Q_t(\tau)]_1$.
   - The verifier then checks $e([Q_t(\tau)]_1, [1]_2) = e([Q_x(\tau)]_1, [\tau]_2)$.
4. Finally, the same $B(x)$ must be used to generate all proofs $\pi_1, \pi_3, \pi_{zt}, \pi_{n+1}$. This is ensured by working over the same commitment $[B(\tau)]_1$. However, one may note that, the sum-check proof (π_3) needs to be modified slightly for this, which can be easily done.

Recap We recap the hinTS construction with the following points briefly.

1. **SRS Generation**: this is essentially the same as KZG SRS generation.
2. **Local Setup**: it has two parts:
   (a) **Key-independent**: this is essentially using the KZG SRS to produce the KZG commitments for $L_i(x), H_{i,1}(x), H_{i,2}(x), H_{i,3}(x), G_{i,j}(x)$. In fact, for hints verification (to be performed in the pre-processing step), KZG commitments are generated in both $\mathbb{G}_1$ and $\mathbb{G}_2$.
   (b) **Key-dependent**: this has two parts: (i) BLS key-generation; (ii) hints generation, which combines the KZG commitments of the polynomials with the corresponding sk_i by each party (this takes place only in one group, say $\mathbb{G}_1$).
3. **Partial Signing / Verification**: this is exactly similar to BLS.
4. **Aggregated Key Generation**: this takes place as soon as all the hints are known (but partial signatures are not known). In this step, hints are verified, and bad hints are discarded. Also, in this step, KZG commitments to $SK(x)$ and $W(x)$ are computed and are included in the verification key, along with the aggregated verification key AVK. They serve as the statements to the proof to be generated later. Also, by summing over j, commitment of $\sum_{j \neq i} \mathsf{sk}_j \cdot G_{i,j}$ is computed.

[11] Currently, we are assuming the verifier computes $[Z(\tau)]_1$, which increases the verifier's cost slightly (though one-time). This can be removed by making the aggregator compute this, and one more check at the verifier's end.

[12] Since $Q_x(x)$ is separately being checked by generalized sumcheck (Equation 3 above), it is clear that the degree can not exceed n, as it must be computed using n hints. Therefore, multiplying with x to generate $Q_t(x)$ suffices.

5. **Signature Aggregation**, which takes place for every signing, when the participating set $\boldsymbol{b}$ is known. In this step the following elements are computed:
   (a) The weight w.
   (b) The aggregated verification key AVK and the aggregated signature ASIG.
   (c) The polynomial $B(x)$ (and its KZG commitment $[B(\tau)]_1$) which corresponds to the participating set $\boldsymbol{b}$.
   (d) KZG commitments to $Q_x(x)$ and $Q_Z(x)$ are computed combining the hints along with the elements of $\boldsymbol{b}$ for π_1, and using $[B(\tau)]_1$.
   (e) The Plonkish sumcheck proof π_3 using the $[B(\tau)]_1$.
   (f) The proof π_{zt}, using Plonkish zero-test to prove that $\boldsymbol{b}$ is binary using $[B(\tau)]_1$.
   (g) The KZG opening proof π_{n+1} for $B(\omega^{n+1}) = 1$ using $[B(\tau)]_1$.
   (h) The degree-test proof π_{deg} is computed using part of the hints.
6. The final signature for message msg and threshold t_{th} consists of $(\mathsf{ASIG}, \mathsf{AVK}, w, \boldsymbol{\Pi} = (\pi_1, \pi_3, \pi_{zt}, \pi_{n+1}, \pi_{deg}))$
7. **Verification** It computes KZG commitment of vanishing $Z(x)$, and verifies the following.
   (a) The BLS aggregated signature using AVK, ASIG and msg.
   (b) The generalized sumcheck using pairing on π_1, $[B(\tau)]_1$, $[[SK(\tau)]_2(\tau)]_1$ and AVK.
   (c) The Plonkish sumcheck on π_3 that involves $[B(\tau)]_1$ and commitments of $W(x)$.
   (d) The Plonkish zero-test on π_{zt} and $[B(\tau)]_1$.
   (e) The KZG opening verification on π_{n+1} with 1 and $[B(\tau)]_1$.
   (f) The degree-test verification using pairing on π_{deg}, commitments of $Q_x(x)$ and $Q_t(x)$.
   (g) $w \geq t_{\mathsf{th}}$.

We note that hint generation is a one-time procedure as long as the keys remain same—an unlimited number of signatures can be produced with the same hints. Since, the **SRS generation** is independent of the participants (and is executed by a ceremony to be discussed in later sections), when participants come online, they execute the **local setup** for the first time, along with **partial signing**.[13] For the next signature, a party that has already done the entire local setup before only executes the partial signings. Of course, when keys are refreshed, the local setup should be executed again. Similarly, the **hints aggregation** part may be executed only once, while **signature aggregation** is to be executed separately for each signing session.

3.4.2 Full Construction

We now describe the algorithms for hinTS, broadly following the syntax from [GJM+24], though we directly present the weighted version. We provide plenty of `(comments)`
to connect it to the technical overview.

hinTS: Ingredients, Parameters, Notations, and Setting

- We assume that each party P_i's identity is denoted by integer i, which is given as an input to the algorithms P_i runs.
- Fix a large integer M which is the maximum number of parties who can ever participate. For example $M = 2^{20}$.
- Positive integers: Total number of parties n; party-i's weight w_i; total weight $\widehat{w} = \sum_{i=1}^{n} w_i$; threshold $t_{\mathsf{th}} \leq \widehat{w}$; a maximum universe size parameter M;
- Asymmetric pairing group parameters $\mathsf{PairPar} := (\mathbb{F}, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2, g_T)$ with pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$;

[13] We remark that, for a party P_i, the key-independent local setup to generate hints may be executed by another party or outsourced to save computation.

(Notations for polynomial arithmetic)

- A multiplicative sub-group $\mathbb{H} \subset \mathbb{F}$ of size $|\mathbb{H}| = n+1$ generated by a $(n+1)$-th root of unity $\omega \in \mathbb{F}$, such that $\omega^{|\mathbb{H}|} = \omega^{n+1} = 1$.
- Vanishing polynomial over $\mathbb{H}$: $Z(x) = \prod_{i=1}^{n+1}(x - \omega^i)$
- For all $i \in [n+1]$:

  (Lagrange basis polynomials)

  - $L_i(x) := \frac{\omega^i}{n+1} \cdot \frac{x^{n+1}-1}{x-\omega^i}$

    (Special polynomials $H_{i,1}, H_{i,3}, G_{i,j}$ for Generalized Sumcheck (cf. Eq 3) and $H_{i,2}$ for degree-check (cf. additional checks, Step 3))
  - $H_{i,1}(x) := \frac{L_i^2(x)-L_i(x)}{Z(x)}$
  - $H_{i,2}(x) := L_i(x) - L_i(0)$
  - $H_{i,3}(x) := \frac{H_{i,2}(x)}{x}$
  - For $j \in [n] \setminus \{i\}$:
    - $G_{i,j}(x) := \frac{L_i(x)L_j(x)}{Z(x)}$
- Let $\mathsf{AllPar} := \big(\mathsf{PairPar}, n, t, \{w_i\}_{i\in[n]}, M, \omega, n+1, Z(x), \{L_i(x), H_{1,i}, H_{i,2}, H_{i,3}, \{G_{i,j}\}_{j\in[n]\setminus\{i\}}\}_{i\in n+1}\big)$;
- Hash functions $\mathcal{H} : \{0,1\}^* \to \mathbb{G}_2$; $\mathcal{H}_{\mathbb{F}} : \{0,1\}^* \to \mathbb{F}$

hinTS: SRS Generation

- $\mathsf{hinTS.Setup}(\mathsf{AllPar}) \to \mathsf{SRS}$ (Same as KZG setup.)
  - $\tau \leftarrow_{\$} \mathbb{F}$ (τ must not be known to attacker)
  - $\boldsymbol{s}_1 := ([\tau]_1, [\tau^2]_1, \ldots, [\tau^M]_1)$
  - $\boldsymbol{s}_2 := ([\tau]_2, [\tau^2]_2, \ldots, [\tau^M]_2)$
  - $\mathsf{KZGPar} := (\mathsf{PairPar}, \boldsymbol{s}_1, \boldsymbol{s}_2)$
  - $\mathsf{SRS} := (\boldsymbol{s}_1, \boldsymbol{s}_2, \mathsf{AllPar})$

hinTS: Local Setup (Key Independent)

- $\mathsf{hinTS.PreHintGen}(\mathsf{SRS}, i) \to \mathsf{loc\_st}_i$:
  - $(\boldsymbol{s}_1, \boldsymbol{s}_2, \cdots) := \mathsf{SRS}$

    (KZG commitments of the Lagrange and special polynomials - we choose to use ExpEval notation here instead of KZG.Com.)
  - $[L_i(\tau)]_1 := \mathsf{ExpEval}(L_i(x), \boldsymbol{s}_1)$
  - $[H_{i,1}(\tau)]_1 := \mathsf{ExpEval}(H_{i,1}(x), \boldsymbol{s}_1)$
  - $[H_{i,2}(\tau)]_1 := \mathsf{ExpEval}(H_{i,2}(x), \boldsymbol{s}_1)$
  - $[H_{i,3}(\tau)]_1 := \mathsf{ExpEval}(H_{i,3}(x), \boldsymbol{s}_1)$
  - For $j \in [n] \setminus \{i\}$:
    - $[G_{i,j}(\tau)]_1 := \mathsf{ExpEval}(G_{i,j}(x), \boldsymbol{s}_1)$

    (KZG commitments for the same in $\mathbb{G}_2$ - this is done for hints verification later.)

- $[L_i(\tau)]_2 := \mathsf{ExpEval}(L_i(x), \boldsymbol{s}_2)$
- $[H_{i,1}(\tau)]_2 := \mathsf{ExpEval}(H_{i,1}(x), \boldsymbol{s}_2)$
- $[H_{i,2}(\tau)]_2 := \mathsf{ExpEval}(H_{i,2}(x), \boldsymbol{s}_2)$
- $[H_{i,3}(\tau)]_2 := \mathsf{ExpEval}(H_{i,3}(x), \boldsymbol{s}_2)$
- For $j \in [n] \setminus \{i\}$:
  - $[G_{i,j}(\tau)]_2 := \mathsf{ExpEval}(G_{i,j}(x), \boldsymbol{s}_2)$
- $\mathsf{loc\_st}_i := \begin{pmatrix} [L_i(\tau)]_1, [H_{i,1}(\tau)]_1, [, H_{i,2}(\tau)]_1, [H_{i,3}(\tau)]_1, \{[G_{i,j}(\tau)]_1\}_{j\in[n]\setminus\{i\}} \\ [L_i(\tau)]_2, [H_{i,1}(\tau)]_2, [H_{i,2}(\tau)]_2, [H_{i,3}(\tau)]_2, \{[G_{i,j}(\tau)]_2\}_{j\in[n]\setminus\{i\}} \end{pmatrix}$

  (Note: the commitments in $\mathbb{G}_1$ would be combined with individual secret keys later for hints generation.)

hinTS: Local Setup (Key Dependent)

- $\mathsf{hinTS.KGen}(\mathsf{AllPar}, i) \to (\mathsf{sk}_i, \mathsf{vk}_i)$:(Same as BLS key-generation)
  - $(\mathsf{PairPar}, \cdots) := \mathsf{AllPar}$
  - $(\mathsf{sk}, \mathsf{vk}) \leftarrow \mathsf{BLS.Gen}(\mathsf{PairPar})$
  - $(\mathsf{sk}_i, \mathsf{vk}_i) := (\mathsf{sk}, \mathsf{vk})$
- $\mathsf{hinTS.HintGen}(\mathsf{sk}_i, \mathsf{loc\_st}_i) \to \mathsf{hint}_i$
  - $\begin{pmatrix} [L_i(\tau)]_1, [H_{i,1}(\tau)]_1, [, H_{i,2}(\tau)]_1, [H_{i,3}(\tau)]_1, \{[G_{i,j}(\tau)]_1\}_{j\in[n]\setminus\{i\}} \\ [L_i(\tau)]_2, [H_{i,1}(\tau)]_2, [H_{i,2}(\tau)]_2, [H_{i,3}(\tau)]_2, \{[G_{i,j}(\tau)]_2\}_{j\in[n]\setminus\{i\}} \end{pmatrix} := \mathsf{loc\_st}_i$
  - $\mathsf{hint}_i := \begin{pmatrix} [\mathsf{sk}_i \cdot L_i(\tau)]_2, [\mathsf{sk}_i \cdot H_{i,1}(\tau)]_1, [\mathsf{sk}_i \cdot H_{i,2}(\tau)]_1, [\mathsf{sk}_i \cdot H_{i,3}(\tau)]_1, \{[\mathsf{sk}_i \cdot G_{i,j}(\tau)]_1\}_{j\in[n]\setminus\{i\}} \\ [L_i(\tau)]_1, [L_i(\tau)]_2, [H_{i,1}(\tau)]_2, [H_{i,2}(\tau)]_2, [H_{i,3}(\tau)]_2, \{[G_{i,j}(\tau)]_2\}_{j\in[n]\setminus\{i\}} \end{pmatrix}$

    (Note: These are individual hints for party P_i)

hinTS: Partial Signing

- $\mathsf{hinTS.PartSign}(\mathsf{sk}_i, \mathsf{msg}) \to \sigma_i$:(Same as BLS multi-sig)
  - $\sigma_i := \mathsf{BLS.Sign}(\mathsf{sk}, \mathsf{msg})$

hinTS: Partial Verification

- $\mathsf{hinTS.PartVerify}(\mathsf{msg}, \sigma_i, \mathsf{vk}_i) \to \mathsf{dec\_bit}_i$:(Same as BLS multi-sig)
  - $\mathsf{dec\_bit}_i := \mathsf{BLS.Verify}(\mathsf{msg}, \sigma_i, \mathsf{vk}_i)$

hinTS: Aggregated Key Generation

- $\mathsf{hinTS.PreProcess}(\mathsf{SRS}, \{\mathsf{hint}_i, \mathsf{vk}_i\}_{i\in[n]}) \to (\mathsf{AK}, \mathsf{VK})$
  - For $i \in [n]$:
    - $\begin{pmatrix} [\mathsf{sk}_i \cdot L_i(\tau)]_2, [\mathsf{sk}_i \cdot H_{i,1}(\tau)]_1, [\mathsf{sk}_i \cdot H_{i,2}(\tau)]_1, [\mathsf{sk}_i \cdot H_{i,3}(\tau)]_1, \{[\mathsf{sk}_i \cdot G_{i,j}(\tau)]_1\}_{j\in[n]\setminus\{i\}} \\ [L_i(\tau)]_1, [L_i(\tau)]_2, [H_{i,1}(\tau)]_2, [H_{i,2}(\tau)]_2, [H_{i,3}(\tau)]_2, \{[G_{i,j}(\tau)]_2\}_{j\in[n]\setminus\{i\}} \end{pmatrix} := \mathsf{hint}_i$
    (Optimized/batched verification using random linear combination--possible as each equation here is of the form $e([x]_1, [1]_2) = e([\mathsf{sk}_i]_1, [y]_2)$)
    - $\chi_{H1}, \chi_{H2}, \chi_{H3}, \{\chi_{Gij}\}_{j\in[n]\setminus\{i\}} \leftarrow_{\$} \mathbb{F}$
    - $\mathsf{AS}_i^1 := ([\mathsf{sk}_i \cdot H_{i,1}(\tau)]_1)^{\chi_{H1}} \cdot ([\mathsf{sk}_i \cdot H_{i,2}(\tau)]_1)^{\chi_{H2}} \cdot ([\mathsf{sk}_i \cdot H_{i,3}(\tau)]_1)^{\chi_{H3}} \cdot \prod_{j\in[n]\setminus\{i\}} ([\mathsf{sk}_i \cdot G_{i,j}(\tau)]_1)^{\chi_{Gij}}$

* $\mathsf{A}_i^2 := ([H_{i,1}(\tau)]_2)^{\chi_{H1}} \cdot ([H_{i,2}(\tau)]_2)^{\chi_{H2}} \cdot ([H_{i,3}(\tau)]_2)^{\chi_{H3}} \cdot \prod_{j\in[n]\setminus\{i\}} ([G_{i,j}(\tau)]_2)^{\chi_{Gij}}$
* $\mathsf{hver\_bit}_i := \left(e(\mathsf{AS}_i^1, [1]_2) \stackrel{?}{=} e(\mathsf{vk}_i, \mathsf{A}_i^2)\right) \wedge \left(e([1]_1, [\mathsf{sk}_i \cdot L_i(\tau)]_2) \stackrel{?}{=} e(\mathsf{vk}_i, [L_i(\tau)]_2)\right)$

  (Note: Verifying combined values in batch for all hints except $[\mathsf{sk}_i \cdot L_i(\tau)]_2$, as it is in $\mathbb{G}_2$; this costs only 4 pairings.)

(Excluding the parties who provided wrong hints)

- $\mathcal{V} := \{i \ : \ \mathsf{hver\_bit}_i = 1\}$
- For all $i \in [n+1] \setminus \{\mathcal{V}\}$:
  * $\mathsf{vk}_i := [0]_1$
  * $w_i := 0$

    (w_{n+1} is set to 0 for the time being, as the aggregated weight w is not available.)

(KZG commitment of the polynomial $SK(x) := \sum_{i\in[n+1]} \mathsf{sk}_i \cdot L_i(x)$)

- $[SK(\tau)]_2 := \prod_{i\in\mathcal{V}} [\mathsf{sk}_i \cdot L_i(\tau)]_2$

(KZG commitment of the polynomial $W(x) := \sum_{i\in[n+1]} w_i \cdot L_i(x)$)

- $[W(\tau)]_1 := \prod_{i\in\mathcal{V}} ([L_i(\tau)]_1)^{w_i}$

(KZG commitment of the polynomial $\sum_{j\neq i} \mathsf{sk}_j \cdot G_{i,j}$)

- For $i \in [n]$: $\left[\sum_{j\neq i} \mathsf{sk}_j \cdot G_{i,j}(\tau)\right]_1 := \prod_{j\in\mathcal{V}\setminus\{i\}} [\mathsf{sk}_j \cdot G_{i,j}(\tau)]_1$

(Aggregation key AK and verification key VK - these can be used for multiple signing sessions.)

- $\mathsf{AK} := \left(\mathcal{V}, \left\{w_i, \mathsf{vk}_i, [\mathsf{sk}_i \cdot H_{i,1}(\tau)]_1, [\mathsf{sk}_i \cdot H_{i,2}(\tau)]_1, [\mathsf{sk}_i \cdot H_{i,3}(\tau)]_1, \left[\sum_{j\neq i} \mathsf{sk}_j \cdot G_{i,j}(\tau)\right]_1\right\}_{i\in\mathcal{V}}\right)$
- $\mathsf{VK} := ([SK(\tau)]_2, [W(\tau)]_1)$

hinTS: Signature Aggregation

- $\mathsf{hinTS.SignAggr}(\mathsf{SRS}, \mathsf{AK}, \mathsf{msg}, \{i, \sigma_i\}_{i\in\mathcal{S}}) \to \mathsf{ASIG}$
  - $\left(\mathcal{V}, \left\{w_i, \mathsf{vk}_i, [\mathsf{sk}_i \cdot H_{i,1}(\tau)]_1, [\mathsf{sk}_i \cdot H_{i,2}(\tau)]_1, [\mathsf{sk}_i \cdot H_{i,3}(\tau)]_1, \left[\sum_{j\neq i} \mathsf{sk}_j \cdot G_{i,j}(\tau)\right]_1\right\}_{i\in\mathcal{V}}\right) := \mathsf{AK}$

    (Participating set, and corresponding vector $\boldsymbol{b}$, polynomial $B(x)$, its KZG commitments in $\mathbb{G}_2$ are computed)
  - $\mathcal{S} := \{i \ : \ i \in \mathcal{V} \wedge \mathsf{hinTS.PartVerify}(\mathsf{msg}, \sigma_i, \mathsf{vk}_i)\}$
  - For $i \in [n+1]$:

    $$b_i := \begin{cases} 1 & \text{if } i \in \mathcal{S} \cup \{n+1\} \\ 0 & \text{otherwise} \end{cases}$$
  - $\boldsymbol{b} := (b_1 b_2 \ldots b_{n+1})$

    (Note: b_{n+1} is set to 1 for Plonkish sumcheck (cf. Sec 3.4.1))
  - $B(x) := \sum_{i\in[n+1]} b_i \cdot L_i(x)$
  - $[B(\tau)]_1 := \mathsf{ExpEval}(B(x), \boldsymbol{s}_1)$

    (The aggregated weight w, and also $W(x)$ is computed)
  - $w = \sum_{i\in\mathcal{S}} w_i$
  - $W(x) := \sum_{i\in[n+1]} w_i \cdot L_i(x)$

(Note: Since $w_{n+1} = 0$, we will use $W(x) - w \cdot L_{n+1}(x)$ below)

(Computing the aggregated signature and aggregated verification key)

- $\mathsf{AVK} := (\prod_{i \in \mathcal{S}} \mathsf{vk}_i)$
- $\sigma := (\prod_{i \in \mathcal{S}} \sigma_i)$

(Computing the proof π_1 for Generalized Sumcheck (Eqn. 3) using the hints; the proof consists of KZG commitments of $Q_x(x)$ and $Q_Z(x)$)

- $[Q_Z(\tau)]_1 := \prod_{i \in [n+1]} \left([\mathsf{sk}_i \cdot H_{i,1}(\tau)]_1 \cdot \left[\sum_{j \neq i} \mathsf{sk}_j \cdot G_{i,j}(\tau) \right]_1 \right)^{b_i}$
- $[Q_x(\tau)]_1 := \prod_{i \in [n+1]} \left([\mathsf{sk}_i \cdot H_{i,3}(\tau)]_1 \right)^{b_i}$

(Computing the proof π_3 for Plonkish sumcheck; zero-test proof π_{zt} for proving b is binary; π_{deg} degree of $Q_x(x)$ is bounded by $n-1$ together)

(Next (Plonkish Sumcheck): Computing $PS(x)$ and $Q_{\mathsf{ps}}(x)$)

- $PS(x) := \sum_{i \in \{2,\ldots,n+1\}} \left(\sum_{j=1}^{i-1} b_j \cdot w_j \right) \cdot L_i(x)$

(Note: This implicitly sets $PS(\omega^{n+2}) = PS(\omega) = 0$)

- $Q_{\mathsf{ps}}(x) := \frac{PS(x\omega) - PS(x) - (W(x) - w \cdot L_{n+1}(x)) \cdot B(x)}{Z(x)}$

(Note: $W(x)$ is offset by $-w \cdot L_{n+1}(x)$)

(Next (Plonkish Sumcheck): Generating KZG commitments of $PS(x), Q_{\mathsf{ps}}(x)$)

- $[PS(\tau)]_1 := \mathsf{ExpEval}(PS(x), \boldsymbol{s}_1)$
- $[Q_{\mathsf{ps}}(\tau)]_1 := \mathsf{ExpEval}(Q_{\mathsf{ps}}(x), \boldsymbol{s}_1)$

(Next (Zero-test): Computing $Q_B(x)$ and its KZG commitment)

(Next (Degree-check): Compute proof π_{deg} for degree-check, which is just a KZG commitment)

- $[Q_t(\tau)]_1 := \prod_{i \in \mathcal{S}} [\mathsf{sk}_i \cdot H_{i,2}(\tau)]_1$
- $Q_B(x) := \frac{B(x) \cdot (1 - B(x))}{Z(x)}$
- $[Q_B(\tau)]_1 := \mathsf{ExpEval}(Q_B(x), \boldsymbol{s}_1)$

(Computing the common Fiat-Shamir challenge)

- $r := \mathcal{H}_{\mathbb{F}}(w, \mathsf{AVK}, \mathsf{SRS}, [B(\tau)]_1, [Q_Z(\tau)]_1, [Q_x(\tau)]_1, [Q_t(\tau)]_1, [PS(\tau)]_1, [Q_{\mathsf{ps}}(\tau)]_1, [Q_B(\tau)]_1)$

(Note: The input to the hash contains more elements than described in Plonkish tests for merger of several proofs)

(Next (Plonkish Sumcheck and Zero-test): Computing the required KZG openings)

- $[O^{\omega}_{\mathsf{ps}}(\tau)]_1 := \mathsf{KZG.Open}(\mathsf{KZGPar}, PS(x), \omega, 1)$
- $[O^{r}_{\mathsf{ps}}(\tau)]_1 := \mathsf{KZG.Open}(\mathsf{KZGPar}, PS(x), r, 1)$
- $[O^{r\omega}_{\mathsf{ps}}(\tau)]_1 := \mathsf{KZG.Open}(\mathsf{KZGPar}, PS(x), r\omega, 1)$
- $[O^{r}_{B}(\tau)]_1 := \mathsf{KZG.Open}(\mathsf{KZGPar}, B(x), r, 1)$
- $[O^{r}_{W}(\tau)]_1 := \mathsf{KZG.Open}(\mathsf{KZGPar}, W(x), r, 1)$
- $[O^{r}_{Q_{\mathsf{ps}}}(\tau)]_1 := \mathsf{KZG.Open}(\mathsf{KZGPar}, Q_{\mathsf{ps}}(x), r, 1)$
- $[O^{r}_{Q_B}(\tau)]_1 := \mathsf{KZG.Open}(\mathsf{KZGPar}, Q_B(x), r, 1)$

(Computing the proof π_{n+1}, which is just a KZG opening)

- $[O^{\omega^{n+1}}_{B}(\tau)]_1 := \mathsf{KZG.Open}(\mathsf{KZGPar}, B(x), \omega^{n+1}, 1)$

– $\mathbf{\Pi} := \begin{pmatrix} w, PS(r), PS(r\omega), W(r), B(r), Q_{\mathsf{ps}}(r), Q_B(r), \\ [B(\tau)]_1, [Q_x(\tau)]_1, [Q_Z(\tau)]_1, [Q_t(\tau)]_1, [PS(\tau)]_1, [Q_{\mathsf{ps}}(\tau)]_1, [Q_B(\tau)]_1, \\ [O^{\omega}_{\mathsf{ps}}(\tau)]_1, [O^{r}_{\mathsf{ps}}(\tau)]_1, [O^{r\omega}_{\mathsf{ps}}(\tau)]_1, [O^{r}_{B}(\tau)]_1, [O^{r}_{W}(\tau)]_1, [O^{r}_{Q_{\mathsf{ps}}}(\tau)]_1, [O^{r}_{Q_B}(\tau)]_1, [O^{\omega^{n+1}}_{B}(\tau)]_1 \end{pmatrix}$

(Note: the proof also consists of the evaluations $PS(r), PS(r\omega), W(r), B(r), Q_{\mathsf{ps}}(r), Q_B(r)$)

(Computing the final aggregated signature)

– $\mathsf{ASIG} := (\sigma, \mathsf{AVK}, \mathbf{\Pi})$

hinTS: Verification

- $\mathsf{hinTS.Verify}(\mathsf{SRS}, \mathsf{msg}, \mathsf{ASIG}, t_{\mathsf{th}}, \mathsf{VK}) \to \mathsf{dec\_bit}$
  - $(\sigma, \mathsf{AVK}, \mathbf{\Pi}) := \mathsf{ASIG}$
  - $\begin{pmatrix} w, PS(r), PS(r\omega), W(r), B(r), Q_{\mathsf{ps}}(r), Q_B(r), \\ [B(\tau)]_1, [Q_x(\tau)]_1, [Q_Z(\tau)]_1, [Q_t(\tau)]_1, [PS(\tau)]_1, [Q_{\mathsf{ps}}(\tau)]_1, [Q_B(\tau)]_1, \\ [O^{\omega}_{\mathsf{ps}}(\tau)]_1, [O^{r}_{\mathsf{ps}}(\tau)]_1, [O^{r\omega}_{\mathsf{ps}}(\tau)]_1, [O^{r}_{B}(\tau)]_1, [O^{r}_{W}(\tau)]_1, [O^{r}_{Q_{\mathsf{ps}}}(\tau)]_1, [O^{r}_{Q_B}(\tau)]_1, [O^{\omega^{n+1}}_{B}(\tau)]_1 \end{pmatrix} := \mathbf{\Pi}$
  item $([SK(\tau)]_2, [W(\tau)]_1) := \mathsf{VK}$
  - $(\cdots, \mathsf{KZGPar}, \cdots) := \mathsf{SRS}$

    (KZG commitment of the vanishing polynomial)
  - $[Z(\tau)]_2 := \mathsf{ExpEval}(Z(x), \boldsymbol{s}_2)$

    (Note: This can be done ahead of time to save verifier's cost, and the same $[Z(\tau)]_2$ can be used for all verification.)

    (Re-generating the Fiat-Shamir challenge)
  - $r := \mathcal{H}_{\mathbb{F}}(w, \mathsf{AVK}, \mathsf{SRS}, [B(\tau)]_1, [Q_Z(\tau)]_1, [Q_x(\tau)]_1, [Q_t(\tau)]_1, [PS(\tau)]_1, [Q_{\mathsf{ps}}(\tau)]_1, [Q_B(\tau)]_1)$

    (Verify BLS aggregated signature)
  - $\mathsf{dec\_sigchk} := (e(\mathsf{AVK}, \mathcal{H}(\mathsf{msg})) = e([1]_1, \sigma))$

    (Verify π_1, the Generalized sumcheck in the exponent)
  - $\mathsf{dec\_sumchk} := (e([B(\tau)]_1, [SK(\tau)]_2) = e(\mathsf{AVK}^{\frac{1}{n+1}}, [1]_2) \cdot e([Q_x(\tau)]_1, [\tau]_2) \cdot e([Q_Z(\tau)]_1, [Z(\tau)]_2))$

    (Verify Plonkish Sumcheck π_3)

    (Next (Plonkish Sumcheck): Verifying the quotient at r)
  - $\mathsf{dec\_scq} := \left(PS(r\omega) - PS(r) - (W(r) - w \cdot L_{n+1}(r)) \cdot B(r) \stackrel{?}{=} Q_{\mathsf{ps}}(r) Z(r)\right)$

    (Next (Plonkish Sumcheck): Verifying the openings)
  - $\mathsf{dec\_opnchk}^{r}_{B} := \mathsf{KZG.Ver}(\mathsf{KZGPar}, [B(\tau)]_1, r, B(r), [O^{r}_{B}(\tau)]_1, 1)$
  - $\mathsf{dec\_opnchk}^{r}_{W} := \mathsf{KZG.Ver}(\mathsf{KZGPar}, [W(\tau)]_1, r, W(r), [O^{r}_{W}(\tau)]_1, 1)$
  - $\mathsf{dec\_opnchk}^{\omega}_{\mathsf{ps}} := \mathsf{KZG.Ver}(\mathsf{KZGPar}, [PS(\tau)]_1, \omega, 0, [O^{\omega}_{\mathsf{ps}}(\tau)]_1, 1)$
  - $\mathsf{dec\_opnchk}^{r}_{\mathsf{ps}} := \mathsf{KZG.Ver}(\mathsf{KZGPar}, [PS(\tau)]_1, r, PS(r), [O^{r}_{\mathsf{ps}}(\tau)]_1, 1)$
  - $\mathsf{dec\_opnchk}^{r\omega}_{\mathsf{ps}} := \mathsf{KZG.Ver}(\mathsf{KZGPar}, [PS(\tau)]_1, r\omega, PS(r\omega), [O^{r\omega}_{\mathsf{ps}}(\tau)]_1, 1)$
  - $\mathsf{dec\_opnchk}^{r}_{Q_{\mathsf{ps}}} := \mathsf{KZG.Ver}(\mathsf{KZGPar}, [Q_{\mathsf{ps}}(\tau)]_1, r, Q_{\mathsf{ps}}(r), [O^{r}_{Q_{\mathsf{ps}}}(\tau)]_1, 1)$

    (Verify Plonkish Zero-test π_{zt} (for binary b))

    (Next (Plonkish Zero-test): Verifying the quotient at r)
  - $\mathsf{dec\_ztq} := (B(r) \cdot (1 - B(r)) = Q_B(r) Z(r))$

    (Next (Plonkish Zero-test): Verifying the remaining opening)
  - $\mathsf{dec\_opnchk}^{r}_{Q_B} := \mathsf{KZG.Ver}(\mathsf{KZGPar}, [Q_B(\tau)]_1, r, Q_B(r), [O^{r}_{Q_B}(\tau)]_1, 1)$

    (Verify π_{n+1}, i.e. KZG opening)

- $\mathsf{dec\_opnchk}_B^{\omega^{n+1}} := \mathsf{KZG.Ver}(\mathsf{KZGPar}, [B(\tau)]_1, \omega^{n+1}, 1, [O_B^{\omega^{n+1}}(\tau)]_1, 1)$

  (Verify degree check π_{deg} using pairing)
- $\mathsf{dec\_degchk} := (e([Q_x(\tau)]_1, [\tau]_2) = e([Q_t(\tau)]_1, [1]_2)$

  (Check the weight)
- $\mathsf{dec\_wtchk} := (w \geq t_{\mathsf{th}})$

  (Compute final decision bit)
-

$$\mathsf{dec\_bit} := \mathsf{dec\_sigchk} \wedge \mathsf{dec\_sumchk} \wedge \mathsf{dec\_scq} \wedge \mathsf{dec\_opnchk}_B^r \wedge \mathsf{dec\_opnchk}_W^r \wedge \mathsf{dec\_opnchk}_{\mathsf{ps}}^{\omega} \wedge \mathsf{dec\_opnchk}_{\mathsf{ps}}^{r} \wedge \mathsf{dec\_opnchk}_{\mathsf{ps}}^{r\omega} \wedge \mathsf{dec\_opnchk}_{Q_{\mathsf{ps}}}^{q} \wedge \mathsf{dec\_ztq} \wedge \mathsf{dec\_opnchk}_{Q_B}^{r} \wedge \mathsf{dec\_opnchk}_B^{\omega^{n+1}} \mathsf{dec\_degchk} \wedge \mathsf{dec\_wtchk}$$

3.4.3 Optimizing Signature Size and Verification Time

Since both signature size and verification time are crucial parameters, we use a number of standard techniques to reduce them, as described below.

Alternative Representation and Merging the quotients We observe that there are two equations being checked above for which the RHS is a product of $Z(x)$ and a quotient polynomial.

- $PS(x\omega) - PS(x) - (W(x) - w \cdot L_{n+1}(x)) \cdot B(x) = Q_{\mathsf{ps}}(x)Z(x)$
- $B(x) \cdot (1 - B(x) = Q_B(x)Z(x)$

The verification is done by checking these at a random point r (generated by Fiat-Shamir, for soundness) and then opening the KZG commitments of the quotient polynomials at r. So, the aggregated signature ASIG must contain 2 field elements $Q_{\mathsf{ps}}(r), Q_B(r)$ plus two KZG openings (group elements) $[O^r_{Q_{\mathsf{ps}}}(\tau)]_1, [O^r_{Q_B}(\tau)]_1$. Using a standard trick (described in Plonk [GWC19]), one can easily merge these two equations to one using another random value χ_Q (generated using Fiat-Shamir), and then only verify a single equation:

$$(PS(x\omega) - PS(x) - (W(x) - w \cdot L_{n+1}(x)) \cdot B(x)) + \chi_Q(B(x) \cdot (1 - B(x)) = Q(x)Z(x)$$

where $Q(x) = Q_{\mathsf{ps}}(x) + \chi_Q Q_B(x)$. Consequently, the signature would contain a single field element $Q(r)$ and a single opening $[O_Q(\tau)]_1$[14] More generally, if there are multiple (say k) such equations:

- $P_1(x) = Q_1(x)Z(x)$
- $P_2(x) = Q_2(x)Z(x)$
- $\cdots$
- $P_k(x) = Q_k(x)Z(x)$

One can merge them to:

$$P_1(x) + \chi_Q P_2(x) + \chi_Q{}^2 P_3(x) + \ldots = (Q_1(x) + \chi_Q Q_2(x) + \chi_Q{}^2 Q_3(x) + \ldots) \cdot Z(x)$$

So, one of the way we can reduce signature size is by replacing the existing checks with checks of the above form, and then merging along. Towards that, we note that instead of verifying $PS(\omega) = 0$ and $B(\omega^{n+1}) = 1$ we can equivalently use Plonkish zero-tests for the polynomials $L_1(x) \cdot PS(x)$ and $L_{n+1} \cdot (1 - B(x))$—the first expression is the same as $PS(\omega)$, while the second expression is equivalent to $1 - B(\omega^{n+1})$. Therefore, for zero-tests we need to verify the following equations:

- $L_1(x) \cdot PS(x) = Q_{\mathsf{ps}^\omega}(x)Z(x)$

[14] Also this immediately reduces verification time as only one KZG openings need to be verified instead of two. However, we discuss that next in the context of merging multiple KZG openings.

- $L_{n+1} \cdot (1 - B(x)) = Q_{B^{\omega^{n+1}}}(x)Z(x)$

So, combining these with the above two equations, we can have:

$$P_{\mathsf{ps}}(x)+\chi_Q P_B(x)+{\chi_Q}^2 \cdot P_{\mathsf{ps}^\omega}(x)+{\chi_Q}^3 \cdot P_{B^{\omega^{n+1}}}(x) = (Q_{\mathsf{ps}}(x)+\chi_Q Q_B(x)+{\chi_Q}^2 \cdot Q_{\mathsf{ps}^\omega}(x)+{\chi_Q}^3 \cdot Q_{B^{\omega^{n+1}}}(x))Z(x)$$

where:

- $P_{\mathsf{ps}}(x) := PS(x\omega) - PS(x) - (W(x) - w \cdot L_{n+1}(x)) \cdot B(x)$
- $P_B(x) := B(x) \cdot (1 - B(x))$
- $P_{\mathsf{ps}^\omega}(x) := L_1(x) \cdot PS(x)$
- $P_{B^{\omega^{n+1}}}(x) := L_{n+1}(x) \cdot (1 - B(x))$

Let us now define $Q^{\mathsf{mrg}} := Q_{\mathsf{ps}}(x) + \chi_Q Q_B(x) + {\chi_Q}^2 \cdot Q_{\mathsf{ps}^\omega}(x) + {\chi_Q}^3 \cdot Q_{B^{\omega^{n+1}}}(x)$. It suffices to now include a field element $Q^{\mathsf{mrg}}(r)$ and a KZG opening of $Q^{\mathsf{mrg}}(x)$ at r. Also, since we are not using the openings of $PS(\omega) = 0$ and $B(\omega^{n+1}) = 1$, those do not need to be included.

Merging KZG openings One of the main techniques we plan to use is the optimization using the homomorphism of KZG, as discussed in Sec 3.2. The idea boils down to merge the openings at the same point. When the above optimization is used, the openings of $B(\omega^{n+1})$ or $PS(\omega)$ are not needed anymore. Instead, we need openings at $Q^{\mathsf{mrg}}(r)$ along with the existing ones at r: $PS(r), B(r), W(r)$ and they are all coming from an identity involving zero-tests. So all of them can be merged using another random value χ_{op} (and its powers).

Resuing Pairing Finally, we observe that the following pairing verifications have a common computation $e([Q_x(\tau)]_1, [\tau]_2)$:

- $\mathsf{dec\_sumchk} := (e([B(\tau)]_1, [SK(\tau)]_2) = e(\mathsf{AVK}^{\frac{1}{n+1}}, [1]_2) \cdot e([Q_x(\tau)]_1, [\tau]_2) \cdot e([Q_Z(\tau)]_1, [Z(\tau)]_2))$
- $\mathsf{dec\_degchk} := \left(e([Q_x(\tau)]_1, [\tau]_2) \stackrel{?}{=} e([Q_x(\tau) \cdot \tau]_1, [1]_2)\right)$

Therefore, it needs to be computed only once, and then can be stored and reused the second time.

Optimized hinTS: Signature Aggregation and Verification Using all of the above, we now present the optimized Signature Aggregation and Hints Verification. Necessarily, to capture the optimizations, we shall be writing the steps without any abstraction. We assume that the hash function $\mathcal{H}_{\mathbb{F}}$ outputs 3 field elements now as opposed to 1 previously.[15] The new comments for the new steps are marked in violet, whereas the steps which are not needed anymore are striked out in the same color.

hinTS: Signature Aggregation (Optimized)

- $\mathsf{hinTS.SignAggr}^{\mathsf{opt}}(\mathsf{SRS}, \mathsf{AK}, \mathsf{msg}, \{i, \sigma_i\}_{i \in \mathcal{S}}) \to \mathsf{ASIG}$
  - $\left(\mathcal{V}, \left\{w_i, \mathsf{vk}_i, [\mathsf{sk}_i \cdot H_{i,1}(\tau)]_1, [\mathsf{sk}_i \cdot H_{i,2}(\tau)]_1, [\mathsf{sk}_i \cdot H_{i,3}(\tau)]_1, \left[\sum_{j \neq i} \mathsf{sk}_j \cdot G_{i,j}(\tau)\right]_1\right\}_{i \in \mathcal{V}}\right) := \mathsf{AK}$
    (Participating set, and corresponding vector $\boldsymbol{b}$, polynomial $B(x)$, its KZG commitments in $\mathbb{G}_2$ are computed)
  - $\mathcal{S} := \{i \ : \ i \in \mathcal{V} \wedge \mathsf{hinTS.PartVerify}(\mathsf{msg}, \sigma_i, \mathsf{vk}_i)\}$
  - For $i \in [n+1]$:
    $$b_i := \begin{cases} 1 & \text{if } i \in \mathcal{S} \cup \{n+1\} \\ 0 & \text{otherwise} \end{cases}$$
  - $\boldsymbol{b} := (b_1 b_2 \ldots b_{n+1})$

[15] This is easily implemented by using, for example, a counter.

(Note: b_{n+1} is set to 1 for Plonkish sumcheck (cf. Sec 3.4.1))

- $B(x) := \sum_{i\in[n+1]} b_i \cdot L_i(x)$
- $[B(\tau)]_1 := \mathsf{ExpEval}(B(x), \boldsymbol{s}_1)$

(The aggregated weight w, and also $W(x)$ is computed)

- $w = \sum_{i\in\mathcal{S}} w_i$
- $W(x) := \sum_{i\in[n+1]} w_i \cdot L_i(x)$

(Note: Since $w_{n+1} = 0$, we will use $W(x) - w \cdot L_{n+1}(x)$ below)

(Computing the aggregated signature and aggregated verification key)

- $\mathsf{AVK} := (\prod_{i\in\mathcal{S}} \mathsf{vk}_i)$
- $\sigma := (\prod_{i\in\mathcal{S}} \sigma_i)$

(Computing the proof π_1 for Generalized Sumcheck (Eqn. 3) using the hints; the proof consists of KZG commitments of $Q_x(x)$ and $Q_Z(x)$)

- $[Q_Z(\tau)]_1 := \prod_{i\in[n+1]} \left([\mathsf{sk}_i \cdot H_{i,1}(\tau)]_1 \cdot \left[\sum_{j\neq i} \mathsf{sk}_j \cdot G_{i,j}(\tau)\right]_1\right)^{b_i}$
- $[Q_x(\tau)]_1 := \prod_{i\in[n+1]} ([\mathsf{sk}_i \cdot H_{i,3}(\tau)]_1)^{b_i}$

(Computing the proof π_3 for Plonkish sumcheck; zero-test proof π_{zt} for proving $\boldsymbol{b}$ is binary; π_{deg} degree of $Q_x(x)$ is bounded by $n-1$ together)

(Next (Plonkish Sumcheck): Computing $PS(x)$ and $Q_{\mathsf{ps}}(x)$)

- $PS(x) := \sum_{i\in\{2,\ldots,n+1\}} \left(\sum_{j=1}^{i-1} b_j \cdot w_j\right) \cdot L_i(x)$

(Note: This implicitly sets $PS(\omega^{n+2}) = PS(\omega) = 0$)

- $Q_{\mathsf{ps}}(x) := \frac{PS(x\omega) - PS(x) - (W(x) - w\cdot L_{n+1}(x))\cdot B(x)}{Z(x)}$

(Note: $W(x)$ is offset by $-w \cdot L_{n+1}(x)$)

(Next (Plonkish Sumcheck): Generating KZG commitments of $PS(x), Q_{\mathsf{ps}}(x)$)

- $[PS(\tau)]_1 := \mathsf{ExpEval}(PS(x), \boldsymbol{s}_1)$
- ~~$[Q_{\mathsf{ps}}(\tau)]_1 := \mathsf{ExpEval}(Q_{\mathsf{ps}}(x), \boldsymbol{s}_1)$~~

(Next (Zero-test): Computing $Q_B(x)$ and its KZG commitment)

(Next (Degree-check): Compute proof π_{deg} for degree-check, which is just a KZG commitment)

- $[Q_t(\tau)]_1 := \prod_{i\in\mathcal{S}} [\mathsf{sk}_i \cdot H_{i,2}(\tau)]_1$
- $Q_B(x) := \frac{B(x)\cdot(1-B(x))}{Z(x)}$
- ~~$[Q_B(\tau)]_1 := \mathsf{ExpEval}(Q_B(x), \boldsymbol{s}_1)$~~

(Compute Q_{ps^ω} and $Q_{B^{\omega^{n+1}}}$)

- $Q_{\mathsf{ps}^\omega}(x) := \frac{L_1(x)\cdot PS(x)}{Z(x)}$
- $Q_{B^{\omega^{n+1}}}(x) := \frac{L_{n+1}(x)\cdot(1-B(x))}{Z(x)}$

(Computing the common Fiat-Shamir challenge)
(The hash function now outputs three field elements.)

- $(r, \chi_Q, \chi_{op}) := \mathcal{H}_{\mathbb{F}}(\mathsf{SRS}, w, \mathsf{AVK}, [Q_Z(\tau)]_1, [Q_x(\tau)]_1, [Q_t(\tau)]_1, [B(\tau)]_1, [PS(\tau)]_1, [Q(\tau)]_1)$

(Merge the quotient polynomials)

- $Q^{\mathsf{mrg}}(x) := Q_{\mathsf{ps}}(x) + \chi_Q Q_B(x) + {\chi_Q}^2 \cdot Q_{\mathsf{ps}^\omega}(x) + {\chi_Q}^3 \cdot Q_{B^{\omega^{n+1}}}(x)$

(Open the polynomials $Q(x), PS(x), B(x), W(x)$ together using KZG optimization)

- $Q(x) := Q^{\mathsf{mrg}}(x) + \chi_{op} \cdot PS(x) + {\chi_{op}}^2 \cdot B(x) + {\chi_{op}}^3 \cdot W(x)$
- $[O^r_Q(\tau)]_1 := \mathsf{KZG.Open}(\mathsf{KZGPar}, Q(x), r, 1)$
- ~~$[O^{\omega}_{\mathsf{ps}}(\tau)]_1 := \mathsf{KZG.Open}(\mathsf{KZGPar}, PS(x), \omega, 1)$~~
- ~~$[O^{r}_{\mathsf{ps}}(\tau)]_1 := \mathsf{KZG.Open}(\mathsf{KZGPar}, PS(x), r, 1)$~~
- $[O^{r\omega}_{\mathsf{ps}}(\tau)]_1 := \mathsf{KZG.Open}(\mathsf{KZGPar}, PS(x), r\omega, 1)$
- ~~$[O^{r}_{B}(\tau)]_1 := \mathsf{KZG.Open}(\mathsf{KZGPar}, B(x), r, 1)$~~
- ~~$[O^{r}_{W}(\tau)]_1 := \mathsf{KZG.Open}(\mathsf{KZGPar}, W(x), r, 1)$~~
- ~~$[O^{r}_{Q_{\mathsf{ps}}}(\tau)]_1 := \mathsf{KZG.Open}(\mathsf{KZGPar}, Q_{\mathsf{ps}}(x), r, 1)$~~
- ~~$[O^{r}_{Q_B}(\tau)]_1 := \mathsf{KZG.Open}(\mathsf{KZGPar}, Q_B(x), r, 1)$~~
- ~~$[O^{\omega^{n+1}}_{B}(\tau)]_1 := \mathsf{KZG.Open}(\mathsf{KZGPar}, B(x), \omega^{n+1}, 1)$~~
- $\mathbf{\Pi} := \left(\begin{smallmatrix} w, PS(r), PS(r\omega), W(r), B(r), Q^{\mathsf{mrg}}(r), \\ [B(\tau)]_1, [Q_x(\tau)]_1, [Q_Z(\tau)]_1, [Q_t(\tau)]_1, [PS(\tau)]_1, [Q(\tau)]_1, \\ [O^r_Q(\tau)]_1, [O^{r\omega}_{\mathsf{ps}}(\tau)]_1, \end{smallmatrix} \right)$

  (Computing the final aggregated signature)
- $\mathsf{ASIG} := (\sigma, \mathsf{AVK}, \mathbf{\Pi})$

hinTS: Verification

- $\mathsf{hinTS.Verify}(\mathsf{SRS}, \mathsf{msg}, \mathsf{ASIG}, t_{\mathsf{th}}, \mathsf{VK}) \rightarrow \mathsf{dec\_bit}$
  - $(\sigma, \mathsf{AVK}, \mathbf{\Pi}) := \mathsf{ASIG}$
  - $\left(\begin{smallmatrix} w, PS(r), PS(r\omega), W(r), B(r), Q^{\mathsf{mrg}}(r), \\ [B(\tau)]_1, [Q_x(\tau)]_1, [Q_Z(\tau)]_1, [Q_t(\tau)]_1, [PS(\tau)]_1, [Q(\tau)]_1, \\ [O^r_Q(\tau)]_1, [O^{r\omega}_{\mathsf{ps}}(\tau)]_1, \end{smallmatrix} \right) := \mathbf{\Pi}$
  - $(\cdots, \mathsf{KZGPar}, \cdots) := \mathsf{SRS}$
  - $([SK(\tau)]_2, [W(\tau)]_1) := \mathsf{VK}$

    (KZG commitment of the vanishing polynomial)
  - $[Z(\tau)]_2 := \mathsf{ExpEval}(Z(x), \boldsymbol{s}_2)$

    (Note: This can be done ahead of time to save verifier's cost, and the same $[Z(\tau)]_2$ can be used for all verification.)

    (Re-generating the Fiat-Shamir challenge) (The hash function now outputs three field elements.)
  - $(r, \chi_Q, \chi_{op}) := \mathcal{H}_{\mathbb{F}}(\mathsf{SRS}, w, \mathsf{AVK}, [Q_Z(\tau)]_1, [Q_x(\tau)]_1, [Q_t(\tau)]_1, [B(\tau)]_1, [PS(\tau)]_1, [Q(\tau)]_1)$

    (Verify BLS aggregated signature)
  - $\mathsf{dec\_sigchk} := (e(\mathsf{AVK}, \mathcal{H}(\mathsf{msg})) = e([1]_1, \sigma))$

    (Verify π_1, the Generalized sumcheck in the exponent)

    (Store the computed pairing $e([Q_x(\tau)]_1, [\tau]_2)$)
  - $[\tau \cdot Q_x(\tau)]_T := e([Q_x(\tau)]_1, [\tau]_2)$
  - $\mathsf{dec\_sumchk} := (e([B(\tau)]_1, [SK(\tau)]_2) = e(\mathsf{AVK}^{\frac{1}{n+1}}, [1]_2) \cdot [\tau \cdot Q_x(\tau)]_T \cdot e([Q_Z(\tau)]_1, [Z(\tau)]_2))$

    (Compute the merged LHS at r)
  - $P_{\mathsf{ps}}(r) := PS(r\omega) - PS(r) - (W(r) - w \cdot L_{n+1}(r)) \cdot B(r)$
  - $P_B(r) := B(r) \cdot (1 - B(r))$
  - $P_{\mathsf{ps}^{\omega}}(r) := L_1(r) \cdot PS(r)$
  - $P_{B^{\omega^{n+1}}}(r) := L_{n+1}(r) \cdot (1 - B(r))$

- $P^{\mathsf{mrg}}(r) := P_{\mathsf{ps}}(r) + \chi_Q P_B(r) + {\chi_Q}^2 \cdot P_{\mathsf{ps}^\omega}(r) + {\chi_Q}^3 \cdot P_{B^{\omega^{n+1}}}(r)$

  (Check the merged quotient equation at r)
- $\mathsf{dec\_Q}^{\mathsf{mrg}} := (P^{\mathsf{mrg}}(r) \stackrel{?}{=} Q^{\mathsf{mrg}}(r) \cdot Z(r))$

  (Compute $Q(r)$)
- $Q(r) := Q^{\mathsf{mrg}}(r) + \chi_{op} \cdot PS(r) + {\chi_{op}}^2 \cdot B(r) + {\chi_{op}}^3 \cdot W(r)$

  (Verify the openings at r with computed $Q(r)$)
- $\mathsf{dec\_opn}^r := \mathsf{KZG.Ver}(\mathsf{KZGPar}, [Q(\tau)]_1, r, Q(r), [O^r_Q(\tau)]_1, 1)$

  (Verify the only remaining KZG openings at $r\omega$)
- ~~$\mathsf{dec\_scq} := \left(PS(r\omega) - PS(r) - (W(r) - w \cdot L_{n+1}(r)) \cdot B(r) \stackrel{?}{=} Q_{\mathsf{ps}}(r) Z(r)\right)$~~
- ~~$\mathsf{dec\_opnchk}^r_B := \mathsf{KZG.Ver}(\mathsf{KZGPar}, [B(\tau)]_1, r, B(r), [O^r_B(\tau)]_1, 1)$~~
- ~~$\mathsf{dec\_opnchk}^r_W := \mathsf{KZG.Ver}(\mathsf{KZGPar}, [W(\tau)]_1, r, W(r), [O^r_W(\tau)]_1, 1)$~~
- ~~$\mathsf{dec\_opnchk}^\omega_{\mathsf{ps}} := \mathsf{KZG.Ver}(\mathsf{KZGPar}, [PS(\tau)]_1, \omega, 0, [O^\omega_{\mathsf{ps}}(\tau)]_1, 1)$~~
- ~~$\mathsf{dec\_opnchk}^r_{\mathsf{ps}} := \mathsf{KZG.Ver}(\mathsf{KZGPar}, [PS(\tau)]_1, r, PS(r), [O^r_{\mathsf{ps}}(\tau)]_1, 1)$~~
- $\mathsf{dec\_opnchk}^{r\omega}_{\mathsf{ps}} := \mathsf{KZG.Ver}(\mathsf{KZGPar}, [PS(\tau)]_1, r\omega, PS(r\omega), [O^{r\omega}_{\mathsf{ps}}(\tau)]_1, 1)$
- ~~$\mathsf{dec\_opnchk}^r_{Q_{\mathsf{ps}}} := \mathsf{KZG.Ver}(\mathsf{KZGPar}, [Q_{\mathsf{ps}}(\tau)]_1, r, Q_{\mathsf{ps}}(r), [O^r_{Q_{\mathsf{ps}}}(\tau)]_1, 1)$~~
- ~~$\mathsf{dec\_ztq} := (B(r) \cdot (1 - B(r)) = Q_B(r) Z(r))$~~
- ~~$\mathsf{dec\_opnchk}^r_{Q_B} := \mathsf{KZG.Ver}(\mathsf{KZGPar}, [Q_B(\tau)]_1, r, Q_B(r), [O^r_{Q_B}(\tau)]_1, 1)$~~
- ~~$\mathsf{dec\_opnchk}^{\omega^{n+1}}_B := \mathsf{KZG.Ver}(\mathsf{KZGPar}, [B(\tau)]_1, \omega^{n+1}, 1, [O^{\omega^{n+1}}_B(\tau)]_1, 1)$~~

  (Verify degree check π_{deg} using pairing)

  (Utilizing the precomputed value $[\tau \cdot Q_x(\tau)]_T$)
- $\mathsf{dec\_degchk} := ([\tau \cdot Q_x(\tau)]_T \stackrel{?}{=} e([Q_t(\tau)]_1, [1]_2)$

  (Check the weight)
- $\mathsf{dec\_wtchk} := (w \geq t_{\mathsf{th}})$

  (Compute final decision bit)
- $\mathsf{dec\_bit} := \mathsf{dec\_sigchk} \wedge \mathsf{dec\_sumchk} \wedge \mathsf{dec\_Q}^{\mathsf{mrg}} \wedge \mathsf{dec\_opn}^r \wedge \mathsf{dec\_degchk} \wedge \mathsf{dec\_wtchk}$

3.4.4 hinTS Ceremony

This is executed among a set of participants, $C_1, \ldots, C_N$ and in the **any-trust** threat model, where the security is hinged upon the assumption that at least one of these participants is honest.

3.5 Security and Efficiency of hinTS

Security. The unforgeability of hinTS reduces to the following:

1. **BLS EU-CMA.** Forging the aggregated BLS signature σ on a fresh message msg without corrupting a threshold-weight subset of signers reduces to the co-CDH problem via the standard BLS reduction.
2. **d-SDH (KZG binding).** The succinct proofs π_1 (generalized sumcheck), π_3 (Plonkish sumcheck), π_{zt} (zero-test), and π_{deg} (degree check) are each sound by Schwartz–Zippel plus d-SDH: the prover cannot produce commitments that pass the pairing checks without the underlying polynomials satisfying the required identities over $\mathbb{H}$.
3. **Hint consistency.** The hint verification step in hinTS.PreProcess uses pairings to check $e([\mathsf{sk}_i \cdot f(\tau)]_1, [1]_2) = e(\mathsf{vk}_i, [f(\tau)]_2)$ for each hint polynomial $f \in \{L_i, H_{i,1}, H_{i,2}, H_{i,3}, G_{i,j}\}$. Under the co-CDH

assumption, a party cannot submit hints corresponding to a secret key different from its registered vk_i without being detected and discarded.

We do not require zero-knowledge: the proofs Π are succinct arguments of knowledge, and upgrading to zero-knowledge adds only a constant blinding cost [GJM+24].

Efficiency. The efficiency profile of hinTS after the optimizations in Section 3.4.3 is as follows.

Operation	**Cost**
Hint generation (per party P_i)	$O(n)$ MSMs of size 1 in $\mathbb{G}_1/\mathbb{G}_2$
Aggregated key generation (hinTS.PreProcess)	4 pairings per party (hint verification)
Signature aggregation (hinTS.SignAggr)	$O(n)$ group exponentiations (one per $b_i = 1$) + $O(1)$ MSMs for $PS(x)$, $Q^{\mathsf{mrg}}(x)$, $Q(x)$
Verification (hinTS.Verify)	≈ 7 pairings + $O(1)$ field ops (const. in n)
Signature size ($\lvert\mathsf{ASIG}\rvert$)	$O(1)$ group + field elements ($\approx$ several hundred bytes)

The key invariant is that *neither the verifier's computation nor the signature size grows with n (number of validators) or $|\mathcal{S}|$ (quorum size).* Hint generation is a one-time cost per key: once a party's hints are broadcast and hinTS.PreProcess has run, the resulting $(\mathsf{AK}, \mathsf{VK})$ can be reused across an unlimited number of signing sessions without repeating that cost.

4 Components for Committee Rotation and Key Updates (Wraps)

We start with Schnorr multi-signature, which will be used in the key update.

4.1 Schnorr Multi-signature

First let us describe Schnorr (single) signature.

Schnorr Signature: Ingredients, Parameters, Notations, and Setting

- Cyclic group $\mathbb{G}$ of prime order p with generator g; shorthand $[x] := g^x$.
- Hash function $\mathcal{H} : \{0,1\}^* \to \mathbb{F}_p$.

Schnorr Signature: Construction

- $\mathsf{Schnorr.KGen} \to (\mathsf{ssk}, \mathsf{svk})$:
  - $\mathsf{ssk} \leftarrow_\$ \mathbb{F}_p$
  - $\mathsf{svk} := [\mathsf{ssk}]$
- $\mathsf{Schnorr.Sign}(\mathsf{ssk}, \mathsf{svk}, \mathsf{msg}) \to \sigma$:

  (Sample nonce and compute commitment)
  - $k \leftarrow_\$ \mathbb{F}_p$
  - $R := [k]$

  (Compute Fiat-Shamir challenge)
  - $c := \mathcal{H}(R \| \mathsf{svk} \| \mathsf{msg})$

  (Compute response)
  - $s := k + c \cdot \mathsf{ssk} \pmod p$

- $\sigma := (R, s)$

- $\mathsf{Schnorr.Verify}(\mathsf{svk}, \mathsf{msg}, \sigma) \to \mathsf{dec\_bit}$:
  - $(R, s) := \sigma$
  - $c := \mathcal{H}(R \| \mathsf{svk} \| \mathsf{msg})$
  - $\mathsf{dec\_bit} := \left([s] \stackrel{?}{=} R \cdot \mathsf{svk}^c\right)$

4.1.1 The Schnorr multi-signature

The standard way to create a multi-signature is simply to aggregate the signatures and verification keys. However, the signing procedure becomes interactive as the parties need to coordinate to obtain the common c. We describe it next.

Let $S \subseteq [n]$ denote the set of active signers for a message $\mathsf{msg} \in \{0,1\}^*$. We denote the secret key vector $\mathbf{ssk} = (\mathsf{ssk}_1 \cdots \mathsf{ssk}_n)$ and the public key vector $\mathbf{svk} = [\mathbf{ssk}] = ([\mathsf{ssk}_1] \cdots [\mathsf{ssk}_n])$. The signing procedure is now split into two parts.

Schnorr Multi-Signature: Ingredients, Parameters, Notations, and Setting

- Cyclic group $\mathbb{G}$ of prime order p with generator g; shorthand $[x] := g^x$.
- Hash function $\mathcal{H} : \{0,1\}^* \to \mathbb{F}_p$.
- Positive integers: total number of parties n; party i's weight w_i; total weight $\hat{w} = \sum_{i=1}^{n} w_i$; threshold $t \leqslant \hat{w}$.

  (Note: No proof of possession is required at key registration.)
- Binary signer vector $\boldsymbol{b} = (b_1 \cdots b_n)$ where $b_i = 1$ iff $i \in S$.

Schnorr Multi-Signature: Construction

$\mathsf{Mult.Schnorr.KGen}(i) \to (\mathsf{ssk}_i, \mathsf{svk}_i)$:

- $(\mathsf{ssk}_i, \mathsf{svk}_i) \leftarrow \mathsf{Schnorr.KGen}$

$\mathsf{Mult.Schnorr.Sign.Commit} \to R_i$

(Sampling nonce and partial commitment)

- $k_i \leftarrow_\$ \mathbb{F}_p$
- $R_i := [k_i]$

$\mathsf{Mult.Schnorr.Sign.Part}(\mathsf{ssk}_i, \mathbf{svk}\{R_j\}_{j\in S}, \mathsf{msg}) \to \sigma_i$

(Construct common commitment)

- $R := \prod_{j\in S} R_j$

(Compute common Fiat-Shamir challenge)

- $c := \mathcal{H}(R \,\|\, \{\mathsf{svk}_j\}_{j\in S} \,\|\, \mathsf{msg})$

(Compute partial response)

- $s_i := k_i + c \cdot \mathsf{ssk}_i \pmod p$

(Compute partial signature)

- $\sigma_i := (R, s_i)$

$\mathsf{Mult.Schnorr.Aggr}(\{\mathsf{svk}_j, \sigma_j\}_{j \in S}) \to \sigma$

- For $j \in S$:
  - $(R, s_j) := \sigma_j$

    (Verifying partial signatures)
  - $\mathsf{dec\_psig}_j := \left([s_j] \stackrel{?}{=} R_j \cdot \mathsf{svk}_j^c\right)$
- $V := \{j \ : \ j \in S \ \wedge \ \mathsf{dec\_psig}_j = 1\}$

  (Aggregating partial signatures)
- $s := \sum_{j \in V} s_j \bmod p$
- $\sigma := (R, s, S, V)$

$\mathsf{Mult.Schnorr.Verify}(\mathbf{svk}, \boldsymbol{w}, \mathsf{msg}, \sigma) \to \mathsf{dec\_bit}$

- $(R, s, S, V) := \sigma$

  (Aggregating relevant verification keys)
- $\mathsf{AVK}_{\mathsf{sc}} := \prod_{j \in V} \mathsf{svk}_j$
- $c := \mathcal{H}(R \ \|\{\mathsf{svk}_j\}_{j \in S}\| \ \mathsf{msg})$

  (Verifying final signature and weight)
- $\mathsf{dec\_sigchk} := \left([s] \stackrel{?}{=} R \cdot \mathsf{AVK}_{\mathsf{sc}}^c\right)$
- $\mathsf{dec\_wt} := \left(\sum_{j \in V} w_j \geq t\right)$
- $\mathsf{dec\_} := \mathsf{dec\_sigchk} \wedge \mathsf{dec\_wt}$

Correctness If all signers are honest:

$$\begin{aligned}
[s] &= \left[\sum_{j \in V} s_j\right] = \left[\sum_{j \in V} (k_j + c \cdot \mathsf{ssk}_j)\right] \\
&= \left[\sum_{j \in V} k_j\right] \cdot \left[\sum_{j \in V} \mathsf{ssk}_j\right]^c \\
&= R \cdot \mathsf{AVK}_{\mathsf{sc}}^c. \quad \checkmark
\end{aligned}$$

Protocol Flow We now describe the protocol flow of the Schnorr multi-signature scheme:

1. **Key-registration:** Each party-i generates their keys $(\mathsf{ssk}_i, \mathsf{svk}_i) \leftarrow \mathsf{Mult.Schnorr.KGen}$. They keep ssk_i private and register svk_i onto the blockchain. We assume that each party's weight w_i is also available on-chain.
2. **Signing:** A quorum of parties, idenufied by a subset $S \subseteq [n]$ decides to participate in a signing session of a message msg. Each party $i \in S$ executes:
   (a) Run $R_i \leftarrow \mathsf{Mult.Schnorr.Sign.Commit}$, and then broadcast R_i to all parties.
   (b) Receive $\{R_j\}_{j \in S \setminus \{i\}}$ and then run $\sigma_i \leftarrow \mathsf{Mult.Schnorr.Sign.Part}(\mathsf{ssk}_i, \{R_j\}_{j \in S}, \mathsf{msg})$. Broadcast σ_i.

3. **Aggregation:** The aggregator collects all $\{\sigma_j\}_{j\in S}$ and then uses the available $\{\mathsf{svk}_j\}_{j\in S}$ to run $\sigma \leftarrow \mathsf{Mult.Schnorr.Aggr}(\{\mathsf{svk}_j, \sigma_j\}_{j\in S})$. It posts the signature σ onto the blockchain. This procedure is public, and can be run by anyone, even someone outside the signing parties.

4. **Verification:** The verification simply runs $\mathsf{dec\_bit} \leftarrow \mathsf{Mult.Schnorr.Verify}(\mathbf{svk}, \boldsymbol{w}, \mathsf{msg}, \sigma)$. This is also public and can be executed by anyone.

Rogue-key Attack Note that, in the above multi-signature scheme, the aggregated public key is simply

$$\mathsf{AVK}_{\mathsf{sc}} := \prod_{i\in V} \mathsf{svk}_i = \left[\sum_{i\in V} \mathsf{ssk}_i\right]$$

Therefore, an adversary controlling party-$i^* \in V$ can register the rogue public key

$$\mathsf{svk}_{i^*} := [\mathsf{ssk}_{i^*}] \cdot \left(\prod_{i\in V\setminus\{i^*\}} \mathsf{svk}_i\right)^{-1},$$

so that the aggregated key becomes

$$\mathsf{AVK}^*_{\mathsf{sc}} = \prod_{i\in V\setminus\{i^*\}} \mathsf{svk}_i \cdot \mathsf{svk}_{i^*} = [\mathsf{ssk}_{i^*}].$$

Now, the adversary controls $\mathsf{AVK}^*_{\mathsf{sc}}$ alone and can forge any multi-signature without the cooperation of *any* other party thereby gaining unilateral signing power over the aggregated key.

4.1.2 Preventing Rogue-key attack using Proof of Possession (PoP)

To prevent the rogue key attack each party must prove knowledge of the discrete logarithm of their public key at registration time. This is accomplished via a zero-knowledge proof, again due to Schnorr [Sch91], also known as a *proof of possession*. From the perspective of Schnorr signature, essentially this requires a party, who possesses a certain knowledge of discrete log (note that, ssk is the discrete log of $\mathsf{svk} = [\mathsf{ssk}]$), produces a "Schnorr signature" on a canonical message (e.g. the empty string or its own identity) to demonstrate the knowledge without revealing the discrete log value itself. We describe the construction below:

Proof of Possession: Ingredients, Parameters, Notations, and Setting

- Cyclic group $\mathbb{G}$ of prime order p with generator g; shorthand $[x] := g^x$.
- Hash function $\mathcal{H} : \{0,1\}^* \to \mathbb{F}_p$.
- Public input / instance: public key $\mathsf{svk} \in \mathbb{G}$.
- Private witness: secret key $\mathsf{ssk} \in \mathbb{F}_p$ such that $\mathsf{svk} = [\mathsf{ssk}]$.

Proof of Possession: Construction

- $\mathsf{PoP.Prove}(\mathsf{ssk}, \mathsf{svk}) \to \pi_{\mathsf{PoP}}$:

  (Sample randomness and compute commitment)

  - $k \leftarrow_{\$} \mathbb{F}_p$
  - $R := [k]$

  (Compute non-interactive challenge via Fiat--Shamir)

  - $c := \mathcal{H}(g \,\|\, \mathsf{svk} \,\|\, R)$

(Compute response)

- $s := k - c \cdot \mathsf{ssk} \pmod p$
- $\pi_{\mathsf{PoP}} := (R, s)$

- $\mathsf{PoP.Verify}(\mathsf{svk}, \pi_{\mathsf{PoP}}) \to \mathsf{dec\_bit}$:
  - $(R, s) := \pi_{\mathsf{PoP}}$
  - $c := \mathcal{H}(g \,\|\, \mathsf{svk} \,\|\, R)$
  - $\mathsf{dec\_bit} := \left([s] \cdot \mathsf{svk}^c \stackrel{?}{=} R\right)$

Protocol Flow with PoP Incorporating this, we obtain the following modified protocol flow, which is not susceptible to rogue-key attack (the changes are colored in violet):

1. **Key-registration:** Each party-i generates their keys $(\mathsf{ssk}_i, \mathsf{svk}_i) \leftarrow \mathsf{Mult.Schnorr.KGen}$. They also generate the PoP $\pi^{(i)}_{\mathsf{PoP}} \leftarrow \mathsf{PoP.Prove}(\mathsf{ssk}_i, \mathsf{svk}_i)$. They keep ssk_i private and register svk_i (accompanied by $\pi^{(i)}_{\mathsf{PoP}}$) onto the blockchain. We assume that each party's weight w_i is also available on-chain.
2. **Key-verification:** Each party-i verifies everyone else's registered verification key svk_j $(i \neq j)$ using $\mathsf{dec\_pop}^i \leftarrow \mathsf{PoP.Verify}(\mathsf{svk}, \pi_{\mathsf{PoP}})$. If $\mathsf{dec\_pop}^j = 0$ for any party-j, then party-j is blacklisted, and all future messages from that party are ignored. In the below, we assume that parties in $\{1, \ldots, n\}$ are not blacklisted—this can be adjusted easily based on the outcome of this step.
3. **Signing:** A quorum of parties, idenufied by a subset $S \subseteq [n]$ decides to participate in a signing session of a message msg. Each party $i \in S$ executes:
   (a) Run $R_i \leftarrow \mathsf{Mult.Schnorr.Sign.Commit}$, and then broadcast R_i to all parties.
   (b) Receive $\{R_j\}_{j \in S \setminus \{i\}}$ and then run $\sigma_i \leftarrow \mathsf{Mult.Schnorr.Sign.Part}(\mathsf{ssk}_i, \{R_j\}_{j \in S}, \mathsf{msg})$. Broadcast σ_i.
4. **Aggregation:** The aggregator collects all $\{\sigma_j\}_{j \in S}$ and then uses the available $\{\mathsf{svk}_j\}_{j \in S}$ to run $\sigma \leftarrow \mathsf{Mult.Schnorr.Aggr}(\{\mathsf{svk}_j, \sigma_j\}_{j \in S})$. It posts the signature σ onto the blockchain. This procedure is public, and can be run by anyone, even someone outside the signing parties.
5. **Verification:** The verification simply runs $\mathsf{dec\_bit} \leftarrow \mathsf{Mult.Schnorr.Verify}(\boldsymbol{\mathsf{svk}}, \boldsymbol{w}, \mathsf{msg}, \sigma)$. This is also public and can be executed by anyone.

Next, we describe R1CS constraint systems, used by Groth16, and also in NOVA (in an extended form).

4.2 Rank-1 Constraint Systems (R1CS)

R1CS is the standard arithmetic circuit representation used in SNARK constructions. It encodes *any* computation as a system of *bilinear constraints* over a finite field. An R1CS structure over field $\mathbb{F}$ consists of sparse matrices $A, B, C \in \mathbb{F}^{m \times n}$, where m is the number of constraints, and n is the number of variables, which is the sum of public variables (instance) and private variable (witness) and constant 1. An R1CS *instance* is a public input $\boldsymbol{x} \in \mathbb{F}^{\ell}$. A witness is $\boldsymbol{w} \in \mathbb{F}^{n-\ell-1}$. Define the full assignment vector:

$$\boldsymbol{z} := (1, \boldsymbol{x}, \boldsymbol{w}) \in \mathbb{F}^n.$$

Using Hadamard product notation (cf. Sec 2) we can write that, the R1CS instance-witness pair $(\boldsymbol{x}, \boldsymbol{w})$ is *satisfying* if and only if:

$$(A\boldsymbol{z}) \circ (B\boldsymbol{z}) = C\boldsymbol{z}.$$

That is, for each constraint $i \in [m]$:

$$\left(\sum_j A_{i,j} z_j\right) \cdot \left(\sum_j B_{i,j} z_j\right) = \sum_j C_{i,j} z_j.$$

It is left to describe how the matrix-triple (A, B, C) is obtained from an arithmetic circuit, generally denoted as a function F, for which the above relation holds. We denote by $(A, B, C) := \mathsf{R1CS}(F)$ and illustrate this using the following **two toy examples of increasing complexity, showing how computations are encoded as** R1CS:

Example 1: $y = x^2 + 3$ **(single constraint)**

Consider the computation $y = x^2 + 3$, where x is a private input and y is a public input/output.[16] Recall that each R1CS constraint has the form (linear) $\cdot$ (linear) = (linear). Since x^2 is the only non-linear term and it is degree-2, a single constraint suffices. The additive constant $+3$ cannot be added directly to the product $x \cdot x$ on the left-hand side—it must live inside one of the three linear combinations $A\boldsymbol{z}$, $B\boldsymbol{z}$, or $C\boldsymbol{z}$. The most natural encoding is $x \cdot x = y - 3$, placing it on the output side. No intermediate wire is needed. The system has $m = 1$ constraint and $n = 3$ variables: $\boldsymbol{z} = (\underbrace{1}_{\text{const}}, \underbrace{y}_{\text{public}}, \underbrace{x}_{\text{witness}})$, so $\ell = 1$ and $|\boldsymbol{w}| = 1$. The matrices $A, B, C \in \mathbb{F}^{1\times 3}$ are:

$$A = \begin{pmatrix} 0 & 0 & 1 \end{pmatrix}, \quad B = \begin{pmatrix} 0 & 0 & 1 \end{pmatrix}, \quad C = \begin{pmatrix} -3 & 1 & 0 \end{pmatrix}$$

This implies: $A\boldsymbol{z} = x$, $B\boldsymbol{z} = x$, $C\boldsymbol{z} = -3 + y$. The constraint checks $x \cdot x = y - 3$, i.e. $y = x^2 + 3$. The constant 3 lives inside C's linear combination.[17] Now, it is easy to see that, for any $x = 3$, $y = 12$, and $\boldsymbol{z} = (1, 12, 3)$. And then:

$$A\boldsymbol{z} = 3, \quad B\boldsymbol{z} = 3, \quad C\boldsymbol{z} = 9, \quad (A\boldsymbol{z})(B\boldsymbol{z}) = 9 = C\boldsymbol{z}. \quad \checkmark$$

Example 2: $y = x^3 + 2x + 1$ **(flattening required)**

Now consider $y = x^3 + 2x + 1$, again with private x and public y. The term x^3 is degree-3, but each R1CS constraint is bilinear (degree-2 at most). No algebraic rearrangement can express x^3 as a product of two linear forms in x, so we are forced to *flatten*: introduce a fresh intermediate wire v to store the result of the first multiplication, then feed it into the second. We define a new private variable $v := x \cdot x$ and obtain two constraints:

1. $x \cdot x = v$
2. $v \cdot x = y - 2x - 1$

This decomposition is unavoidable. More generally, computing x^d requires $d-1$ constraints and $d-2$ intermediate wires. The key feature of R1CS is that additions and constants cost nothing—only multiplications consume constraints. This is exactly what arithmetic circuit compilers (e.g. Circom [nInT+23], Arkworks [ark22]) do automatically. Now $m = 2$ constraints and $n = 4$ variables: $\boldsymbol{z} = (\underbrace{1}_{\text{const}}, \underbrace{y}_{\text{public}}, \underbrace{x, v}_{\text{witness}})$, so $\ell = 1$, $|\boldsymbol{w}| = 2$. The matrices $A, B, C \in \mathbb{F}^{2\times 4}$ are computed as:

$$A = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad B = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}, \quad C = \begin{pmatrix} 0 & 0 & 0 & 1 \\ -1 & 1 & -2 & 0 \end{pmatrix}$$

This implies:

- Row 1: $(A\boldsymbol{z})_1 = x$, $(B\boldsymbol{z})_1 = x$, $(C\boldsymbol{z})_1 = v$, giving $x \cdot x = v$.
- Row 2: $(A\boldsymbol{z})_2 = v$, $(B\boldsymbol{z})_2 = x$, $(C\boldsymbol{z})_2 = -1 + y - 2x$, giving $v \cdot x = y - 2x - 1$, i.e. $y = vx + 2x + 1 = x^3 + 2x + 1$.

Now with $x = 3$: $v = 9$, $y = 27 + 6 + 1 = 34$, $\boldsymbol{z} = (1, 34, 3, 9)$:

$$A\boldsymbol{z} = \begin{pmatrix} 3 \\ 9 \end{pmatrix}, \quad B\boldsymbol{z} = \begin{pmatrix} 3 \\ 3 \end{pmatrix}, \quad C\boldsymbol{z} = \begin{pmatrix} 9 \\ 27 \end{pmatrix}, \quad (A\boldsymbol{z}) \circ (B\boldsymbol{z}) = \begin{pmatrix} 9 \\ 27 \end{pmatrix} = C\boldsymbol{z}. \quad \checkmark$$

[16] Though in this representation y is essentially an output, we can rewrite $y - x^2 - 3 = 0$ generically to ensure that both x, y are semantically inputs, and the output is always 0. So, we use output and public input interchangeably.

[17] Note that, additions and constants never cost extra constraints.

Using $\mathsf{trace}_{\mathsf{R1}}$ For any efficient function F with inputs $\boldsymbol{I}$, $\mathsf{trace}_{\mathsf{R1}}$ runs F on $\boldsymbol{I}$ and records the execution as a satisfying R1CS assignment:

$$(\boldsymbol{x}, \boldsymbol{w}) := \mathsf{trace}_{\mathsf{R1}}(F, \boldsymbol{I}),$$

where $\boldsymbol{x}$ is the public part and $\boldsymbol{w}$ is the witness (the private inputs together with all intermediate values produced during execution). When only the public portion $\boldsymbol{I}_{\mathsf{pub}}$ is available—as at the verifier, which re-derives the public-input encoding from the statement alone—we overload $\mathsf{trace}_{\mathsf{R1}}$ to return only the public part:

$$\boldsymbol{x} := \mathsf{trace}_{\mathsf{R1}}(F, \boldsymbol{I}_{\mathsf{pub}}).$$

Applied to **Example 1** ($y = x^2 + 3$) on private input $x = 3$: $\mathsf{trace}_{\mathsf{R1}}$ runs F, computes $y = 12$, and packages

$$(\boldsymbol{x}, \boldsymbol{w}) := \mathsf{trace}_{\mathsf{R1}}(F, x = 3) \;=\; \big((12),\ (3)\big),$$

reassembling to $\boldsymbol{z} = (1, 12, 3)$. The overloaded verifier call $\mathsf{trace}_{\mathsf{R1}}(F, y = 12) = (12)$ returns only $\boldsymbol{x}$, without access to the private input.

Applied to **Example 2** ($y = x^3 + 2x + 1$) on $x = 3$: $\mathsf{trace}_{\mathsf{R1}}$ additionally records the intermediate $v := x \cdot x = 9$ as part of the witness, producing

$$(\boldsymbol{x}, \boldsymbol{w}) := \mathsf{trace}_{\mathsf{R1}}(F, x = 3) \;=\; \big((34),\ (3,\ 9)\big),$$

reassembling to $\boldsymbol{z} = (1, 34, 3, 9)$. The overloaded call $\mathsf{trace}_{\mathsf{R1}}(F, y = 34) = (34)$ again returns only the public part.

4.3 Groth16 Succinct Proof System

Groth16 [Gro16] is a pairing-based succinct proof system (SNARK) for R1CS.[18] It produces constant-size proofs consisting of 3 group elements, verified by a *single* pairing equation. Looking ahead, in our key-update, Groth16 serves as the final *decider* SNARK that compresses the accumulated NOVA IVC proof into a succinct proof suitable for on-chain verification. One important limitation of Groth16 is that the parameters depend on the R1CS matrices / the computation / algebraic circuit.

Additional Preliminaries The Groth16 construction re-uses the KZG polynomial commitment infrastructure and notations from the hinTS construction (cf. Section 3). In particular, the setup generates a KZG structured reference string, and the prover uses that commit to polynomials derived from the R1CS instance (A, B, C). However, KZG opening/verification is never used. So, we just use the KZG commitment notations, as ExpEval. We assume each matrix in the instance is of dimension $m \times n$.

As in the hinTS construction, we use a vanishing polynomial, but over a different evaluation domain. Let $\mathbb{H}_{\mathsf{G}} \subset \mathbb{F}$ be a multiplicative subgroup of size $|\mathbb{H}_{\mathsf{G}}| = m$ generated by an m-th root of unity $\omega_{\mathsf{G}} \in \mathbb{F}$ with $\omega_{\mathsf{G}}^m = 1$. The vanishing polynomial over $\mathbb{H}_{\mathsf{G}}$ is:

$$Z_{\mathsf{G}}(x) := \prod_{j=1}^{m} (x - \omega_{\mathsf{G}}^j).$$

For each variable $i \in \{1, \ldots, n\}$, we derive a *column polynomial* $A_i(x)$ from the i-th column A_i of matrix A: it is the unique polynomial of degree $\leqslant m-1$ satisfying $A_i(\omega_{\mathsf{G}}^j) = A_{j,i}$ for all $j \in [m]$, obtained via Lagrange interpolation over $\mathbb{H}_{\mathsf{G}}$. Column polynomials $B_i(x)$ and $C_i(x)$ are derived analogously from B_i and C_i.

Groth16: Ingredients, Parameters, Notations, and Setting

- Asymmetric pairing group parameters $\mathsf{PairPar} := (\mathbb{F}, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2, g_T)$ with pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$. Recall notation: $[x]_1 := g_1^x$, $[x]_2 := g_2^x$, $[x]_T := g_T^x$.
- R1CS structure: matrices $A, B, C \in \mathbb{F}^{m \times n}$ with ℓ public inputs.

[18] SNARK spells out as Succinct Non-interactive Argument of Knowledge.

- KZG polynomial commitment notation (cf. Section 3.2) with KZG.Gen and ExpEval (which is the same as ExpEval)

Groth16: Construction

- Groth16.Setup$(A, B, C) \to (pk_{\mathsf{G}}, vk_{\mathsf{G}})$:
  - $\alpha, \beta, \gamma, \delta \leftarrow_{\$} \mathbb{F}$
  - $U_i(x) := \beta \cdot A_i(x) + \alpha \cdot B_i(x) + C_i(x)$

    (Generate KZG SRS with max degree $d = 2m-2$)
  - $(\mathsf{PairPar}, \boldsymbol{s}_1, \boldsymbol{s}_2) \leftarrow \mathsf{KZG.Gen}$

    (Compute public proving-key and verification-key using KZG.Com))
  - For $i \in [\ell+1]$: $[U_i(\tau)/\gamma]_1 := \mathsf{ExpEval}(U_i(x)/\gamma, \boldsymbol{s}_1)$
  - For $i \in \{\ell+2, \ldots, n\}$: $[U_i(\tau)/\delta]_1 := \mathsf{ExpEval}(U_i(x)/\delta, \boldsymbol{s}_1)$
  - For $i \in \{0, \ldots, m-2\}$: $[\tau^i \cdot Z_{\mathsf{G}}(\tau)/\delta]_1 := \mathsf{ExpEval}(x^i \cdot Z_{\mathsf{G}}(x)/\delta, \boldsymbol{s}_1)$

    (Assemble proving and verification keys)
  - $pk_{\mathsf{G}} := \left(\begin{smallmatrix} (\mathsf{PairPar}, \boldsymbol{s}_1, \boldsymbol{s}_2),\ [\alpha]_1, [\beta]_1, [\beta]_2, [\delta]_1, [\delta]_2, \\ \{[U_i(\tau)/\gamma]_1\}_{i=1}^{\ell+1},\ \{[U_i(\tau)/\delta]_1\}_{i=\ell+2}^{n},\ \{[\tau^i \cdot Z_{\mathsf{G}}(\tau)/\delta]_1\}_{i=0}^{m-2} \end{smallmatrix} \right)$
  - $vk_{\mathsf{G}} := \left([\alpha]_1, [\beta]_2, [\gamma]_2, [\delta]_2,\ \{[U_i(\tau)/\gamma]_1\}_{i=1}^{\ell+1} \right)$

    (Note: $\alpha, \beta, \gamma, \delta$ must be deleted after setup. Knowledge of these values would allow forging proofs.)
- Groth16.Prove$(pk_{\mathsf{G}}, \boldsymbol{x}, \boldsymbol{w}) \to \pi$:

  (Define the aggregate polynomials from the R1CS assignment)
  - $z_1 := 1$, $(z_2, \ldots, z_{\ell+1}) := \boldsymbol{x}$, $(z_{\ell+2}, \ldots, z_n) := \boldsymbol{w}$
  - $A(x) := \sum_{i=1}^{n} z_i \cdot A_i(x)$, $\quad B(x) := \sum_{i=1}^{n} z_i \cdot B_i(x)$, $\quad C(x) := \sum_{i=1}^{n} z_i \cdot C_i(x)$

    (Compute the quotient polynomial---this is well-defined iff $(\boldsymbol{x}, \boldsymbol{w})$ satisfies the R1CS)
  - $Q(x) := \frac{A(x) \cdot B(x) - C(x)}{Z_{\mathsf{G}}(x)}$

    (Note: $Q(x)$ has degree $\leqslant m-2$; it exists because $A(\omega_{\mathsf{G}}^j) \cdot B(\omega_{\mathsf{G}}^j) = C(\omega_{\mathsf{G}}^j)$ for all $j \in [m]$)

    (Sample blinding factors and compute KZG commitments of aggregate polynomials)
  - $r, s \leftarrow_{\$} \mathbb{F}$
  - $[A(\tau)]_1 := \mathsf{ExpEval}(A(x), \boldsymbol{s}_1)$
  - $[B(\tau)]_2 := \mathsf{ExpEval}(B(x), \boldsymbol{s}_2)$
  - $[B(\tau)]_1 := \mathsf{ExpEval}(B(x), \boldsymbol{s}_1)$
  - $[Q(\tau)]_1 := \mathsf{ExpEval}(Q(x), \boldsymbol{s}_1)$

    (Assemble the three proof elements)
  - $[\mathsf{A}]_1 := [\alpha]_1 \cdot [A(\tau)]_1 \cdot [\delta]_1^r$
  - $[\mathsf{B}]_2 := [\beta]_2 \cdot [B(\tau)]_2 \cdot [\delta]_2^s$
  - $[\mathsf{C}]_1 := \prod_{i=\ell+2}^{n} [U_i(\tau)/\delta]_1^{w_i} \cdot [Q(\tau)]_1 \cdot [Z_{\mathsf{G}}(\tau)/\delta]_1 \cdot [\mathsf{A}]_1^s \cdot [B(\tau)]_1^r \cdot [\delta]_1^{-rs}$

    ($[\mathsf{C}]_1$ combines the witness contribution (via pre-computed $[U_i(\tau)/\delta]_1$ from pk), the quotient (via KZG.Com), and the blinding terms.)

- $\mathsf{Groth16.Verify}(\mathsf{vk}, \boldsymbol{x}, \pi) \to \mathsf{dec\_bit}$:
  - $([\mathsf{A}]_1, [\mathsf{B}]_2, [\mathsf{C}]_1) := \pi$

    (Compute the public-input contribution ($z_1 = 1$ is the constant; $z_2, \ldots, z_{\ell+1}$ are the public inputs))
  - $[\mathsf{pub}]_1 := \prod_{i=1}^{\ell+1} [U_i(\tau)/\gamma]_1^{z_i}$

    (Verify via a single pairing equation (total 4 pairings))
  - $\mathsf{dec\_bit} := \left(e([\mathsf{A}]_1, [\mathsf{B}]_2) \stackrel{?}{=} e([\alpha]_1, [\beta]_2) \cdot e([\mathsf{pub}]_1, [\gamma]_2) \cdot e([\mathsf{C}]_1, [\delta]_2)\right)$

Optimizing the number of pairings by pre-computation. The verification equation $e([\mathsf{A}]_1, [\mathsf{B}]_2) \stackrel{?}{=} e([\alpha]_1, [\beta]_2) \cdot e([\mathsf{pub}]_1, [\gamma]_2) \cdot e([\mathsf{C}]_1, [\delta]_2)$ naively requires 4 pairing computations. However, we can amortize by pre-computing $e([\alpha]_1, [\beta]_2) \in \mathbb{G}_T$ once during setup and store it as part of vk. At verification time, the verifier computes only 3 pairings—$e([\mathsf{A}]_1, [\mathsf{B}]_2)$, $e([\mathsf{pub}]_1, [\gamma]_2)$, and $e([\mathsf{C}]_1, [\delta]_2)$—plus one $\mathbb{G}_T$ multiplication with the precomputed value. Equivalently, rearranging to a single check against $[1]_T$:

$$e([\mathsf{A}]_1, [\mathsf{B}]_2) \cdot e(-[\mathsf{pub}]_1, [\gamma]_2) \cdot e(-[\mathsf{C}]_1, [\delta]_2) \stackrel{?}{=} e([\alpha]_1, [\beta]_2)$$

where the right-hand side is a stored constant and the left-hand side requires exactly 3 pairings (using the negation in $\mathbb{G}_1$ to avoid computing inverses in $\mathbb{G}_T$).

Properties. Let us briefly mention the most important properties of Groth16.

1. **Succinctness.** The proof $\pi = ([A]_1, [B]_2, [C]_1)$ consists of 2 elements in $\mathbb{G}_1$ and 1 element in $\mathbb{G}_2$—roughly 192 bytes on BN254, regardless of m or n.
2. **Constant-time verification.** Verification requires computing $O(\ell)$ scalar multiplications for the public input and a fixed number of pairings, independent of m, n, or the witness size.
3. **Knowledge soundness.** Under the q-type knowledge-of-exponent assumptions in the bilinear group, Groth16 is knowledge-sound: an efficient extractor can recover a valid witness from any accepting proof.
4. **Trusted setup.** The proving and verification keys depend on sensitive values $(\alpha, \beta, \gamma, \delta, \tau)$. A multi-party ceremony is used to ensure no single participant knows them.

Remark 1. In HieroTSS deployment Groth16 is instantiated over BN254, which offers efficient pairing operations and EVM compatibility. Looking ahead, the Groth16 proof compresses the NOVA accumulated Relaxed-R1CS instance (whose verification takes $O(m \cdot n)$ time) into a constant-size proof that any external verifier can check with a single pairing equation

Next, we describe Pedersen's homomorphic commitment schemes, which will be used later in the recursive proof system / folding scheme called NOVA [KST22] for incremental verifiable computation.

4.4 Pedersen Commitments

Pedersen Commitment: Ingredients, Parameters, Notations, and Setting

- Cyclic group $\mathbb{G}$ of prime order p with generator g; shorthand $[x] := g^x$.
- Additional public generators $h_1, h_2, \ldots, h_m \leftarrow_{\$} \mathbb{G}$ (form a common reference string).

Pedersen Commitment: Construction

- $\mathsf{Com}(\boldsymbol{v}; r) \to C$:

  (Commit to a vector $\boldsymbol{v} = (v_1, \ldots, v_m) \in \mathbb{F}^m$ with randomness/opening $r \in \mathbb{F}$)
  - $C := g^r \cdot \prod_{i=1}^{m} h_i^{v_i}$

- $\mathsf{Com.Verify}(\boldsymbol{v}, r, C) \to \mathsf{dec\_bit}$:

  (Check the opening r)

  - $\mathsf{dec\_bit} := \left(C \stackrel{?}{=} g^r \cdot \prod_{i=1}^{m} h_i^{v_i}\right)$

Properties.

1. **Hiding.** Given $C = \mathsf{Com}(\boldsymbol{v}; r)$, the commitment C reveals no information about $\boldsymbol{v}$ (perfect hiding, since r is uniform).
2. **Binding.** Under the discrete log assumption in $\mathbb{G}$, it is computationally infeasible to find $(\boldsymbol{v}, r) \neq (\boldsymbol{v}', r')$ such that $\mathsf{Com}(\boldsymbol{v}; r) = \mathsf{Com}(\boldsymbol{v}'; r')$.
3. **Additive homomorphism.** For any $\boldsymbol{v}_1, \boldsymbol{v}_2 \in \mathbb{F}^m$ and $r_1, r_2, \alpha \in \mathbb{F}$:

$$\mathsf{Com}(\boldsymbol{v}_1; r_1) \cdot \mathsf{Com}(\boldsymbol{v}_2; r_2) = \mathsf{Com}(\boldsymbol{v}_1 + \boldsymbol{v}_2;\ r_1 + r_2)$$

$$\mathsf{Com}(\boldsymbol{v}_1; r_1)^{\alpha} = \mathsf{Com}(\alpha \cdot \boldsymbol{v}_1;\ \alpha \cdot r_1)$$

Looking ahead **the homomorphism is what enables folding**: given commitments $\overline{W}_1 = \mathsf{Com}(\boldsymbol{w}_1; r_{W_1})$ and $\overline{W}_2 = \mathsf{Com}(\boldsymbol{w}_2; r_{W_2})$, the folded commitment $\overline{W}_1 \cdot \overline{W}_2^{r} = \mathsf{Com}(\boldsymbol{w}_1 + r \cdot \boldsymbol{w}_2;\ r_{W_1} + r \cdot r_{W_2})$ is automatically a valid commitment to the "folded" witness $\boldsymbol{w}_1 + r \cdot \boldsymbol{w}_2$ without opening either commitment.[19]

4.5 NOVA Folding Scheme

We now describe NOVA [KST22], a folding-based approach to Incrementally Verifiable Computation (IVC). An IVC scheme allows a prover to iteratively execute a function F over T steps, producing at each step a proof π_t attesting that the entire computation from step 1 through step t was performed correctly. Crucially, the proof size and verifier time remain *constant* regardless of T.

The central idea in NOVA is *folding*: instead of producing a full SNARK proof at every step, the prover "folds" two instances into one, using only cheap group operations. This accumulated instance can later be compressed into a succinct proof using a final SNARK (e.g.Groth16). In our setting, NOVA is used for the *key update* mechanism: each address book rotation corresponds to one IVC step, and the accumulated NOVA proof attests to the validity of the entire chain of rotations from genesis.

4.5.1 NOVA: Technical Overview

We start by describing why standard R1CS does not directly support folding, and how the *relaxation* introduced by NOVA resolves this.

Why standard R1CS does not compose. Suppose we have two satisfying R1CS instances with assignment vectors $\boldsymbol{z}_1$ and $\boldsymbol{z}_2$, so that $(A\boldsymbol{z}_1) \circ (B\boldsymbol{z}_1) = C\boldsymbol{z}_1$ and $(A\boldsymbol{z}_2) \circ (B\boldsymbol{z}_2) = C\boldsymbol{z}_2$. A natural approach to "compress" both into a single instance is to take a random linear combination $\boldsymbol{z} := \boldsymbol{z}_1 + r \cdot \boldsymbol{z}_2$ for a random challenge $r \in \mathbb{F}$, and check whether $(A\boldsymbol{z}) \circ (B\boldsymbol{z}) = C\boldsymbol{z}$. Expanding the left-hand side:

$$(A\boldsymbol{z}) \circ (B\boldsymbol{z}) = (A\boldsymbol{z}_1 + rA\boldsymbol{z}_2) \circ (B\boldsymbol{z}_1 + rB\boldsymbol{z}_2) \tag{7}$$

$$= \underbrace{(A\boldsymbol{z}_1) \circ (B\boldsymbol{z}_1)}_{=C\boldsymbol{z}_1} + r \cdot \underbrace{\left[(A\boldsymbol{z}_1) \circ (B\boldsymbol{z}_2) + (A\boldsymbol{z}_2) \circ (B\boldsymbol{z}_1)\right]}_{\text{cross-term}} + r^2 \cdot \underbrace{(A\boldsymbol{z}_2) \circ (B\boldsymbol{z}_2)}_{=C\boldsymbol{z}_2} \tag{8}$$

But the right-hand side of standard R1CS gives $C\boldsymbol{z} = C\boldsymbol{z}_1 + r \cdot C\boldsymbol{z}_2$, which is *degree 1* in r. The left-hand side is *degree 2* in r due to the Hadamard product. These cannot be equal for all r unless the cross-term happens to equal $C\boldsymbol{z}_2 + C\boldsymbol{z}_1$ (which does not hold except with a negligible probability). So the folded assignment does not satisfy standard R1CS.

[19] We note that, folding essentially compresses size by generating a single instance from two instances recursively. For more we refer to the next Section 4.5

The NOVA solution: relaxation. NOVA resolves this by introducing two new components to the R1CS relation: a *scalar* $u \in \mathbb{F}$ and an *error vector* $\boldsymbol{E} \in \mathbb{F}^m$ (one entry per constraint). The relaxed relation becomes:

$$(A\boldsymbol{z}) \circ (B\boldsymbol{z}) = u \cdot (C\boldsymbol{z}) + \boldsymbol{E}.$$

Any standard R1CS instance trivially satisfies this with $u = 1$ and $\boldsymbol{E} = \mathbf{0}$. But after folding, the scalar u absorbs the degree-2 coefficient and the error vector $\boldsymbol{E}$ absorbs the cross-term. This allows the folded instance to satisfy the relaxed equation even though it does not correspond to any valid circuit execution (which must have $u = 1$ and $\boldsymbol{E} = \mathbf{0}$). To prevent the prover from cheating by manipulating $\boldsymbol{E}$, the error vector (and the witness) are hidden using Pedersen commitments—in other words $\boldsymbol{E}$ also becomes a part of the witness. This is formalized as committed Relaxed-R1CS.

Committed Relaxed-R1CS A Relaxed-R1CS structure consists of the same matrices $A, B, C \in \mathbb{F}^{m \times n}$. A Relaxed-R1CS *instance-witness pair* consists of:

- Instance: $\mathbb{U} := (\boldsymbol{x}, u, \overline{E}, \overline{W}) \in \mathbb{F}^{\ell} \times \mathbb{F} \times \mathbb{G} \times \mathbb{G}$
- Witness: $\mathbb{W} := (\boldsymbol{w}, \boldsymbol{E}, r_W, r_E) \in \mathbb{F}^{n-\ell-1} \times \mathbb{F}^m \times \mathbb{F} \times \mathbb{F}$

where $\overline{W} := \mathsf{Com}(\boldsymbol{w}; r_W)$ and $\overline{E} := \mathsf{Com}(\boldsymbol{E}; r_E)$ are Pedersen commitments. The pair is satisfying if and only if:

$$(A\boldsymbol{z}) \circ (B\boldsymbol{z}) = u \cdot (C\boldsymbol{z}) + \boldsymbol{E}$$

where $\boldsymbol{z} := (u, \boldsymbol{x}, \boldsymbol{w})$. Any satisfying standard R1CS instance $(\boldsymbol{x}, \boldsymbol{w})$ is also a satisfying Relaxed-R1CS instance with $u := 1$, $\boldsymbol{E} := \mathbf{0}$, $r_W \leftarrow_{\$} \mathbb{F}$, $r_E := 0$, $\overline{W} := \mathsf{Com}(\boldsymbol{w}; r_W)$, and $\overline{E} := \mathsf{Com}(\mathbf{0}; 0) = [0]$. Indeed:

$$(A\boldsymbol{z}) \circ (B\boldsymbol{z}) = C\boldsymbol{z} = 1 \cdot C\boldsymbol{z} + \mathbf{0}. \quad \checkmark$$

Now, consider two satisfying Relaxed-R1CS pairs with $\boldsymbol{z}_1 = (u_1, \boldsymbol{x}_1, \boldsymbol{w}_1)$ and $\boldsymbol{z}_2 = (u_2, \boldsymbol{x}_2, \boldsymbol{w}_2)$. The *cross-term* is defined as:

$$\boldsymbol{T} := (A\boldsymbol{z}_1) \circ (B\boldsymbol{z}_2) + (A\boldsymbol{z}_2) \circ (B\boldsymbol{z}_1) - u_1 \cdot C\boldsymbol{z}_2 - u_2 \cdot C\boldsymbol{z}_1. \tag{9}$$

Let us define the folded error vector to be $\boldsymbol{E} := \boldsymbol{E}_1 + r \cdot \boldsymbol{T} + r^2 \cdot \boldsymbol{E}_2$, and the folded pair satisfies Relaxed-R1CS with $u := u_1 + r \cdot u_2$. Therefore:

$$\begin{aligned}
(A\boldsymbol{z}) \circ (B\boldsymbol{z}) &= \underbrace{(A\boldsymbol{z}_1) \circ (B\boldsymbol{z}_1)}_{=u_1 \cdot (C\boldsymbol{z}_1) + \boldsymbol{E}_1} + r \cdot \big[(A\boldsymbol{z}_1) \circ (B\boldsymbol{z}_2) + (A\boldsymbol{z}_2) \circ (B\boldsymbol{z}_1)\big] + r^2 \cdot \underbrace{(A\boldsymbol{z}_2) \circ (B\boldsymbol{z}_2)}_{=u_2 \cdot (C\boldsymbol{z}_2) + \boldsymbol{E}_2} \\
&= u_1 \cdot (\underbrace{C(\boldsymbol{z}_1 + r\boldsymbol{z}_2)}_{=C\boldsymbol{z}}) + ru_2 \cdot (\underbrace{C(\boldsymbol{z}_1 + r\boldsymbol{z}_2)}_{=C\boldsymbol{z}}) + \\
&\qquad r \cdot \underbrace{\big[(A\boldsymbol{z}_1) \circ (B\boldsymbol{z}_2) + (A\boldsymbol{z}_2) \circ (B\boldsymbol{z}_1) - u_1 \cdot C\boldsymbol{z_1} - u_2 \cdot C\boldsymbol{z_2}\big]}_{=\boldsymbol{T}} + \boldsymbol{E}_1 + r^2 \cdot \boldsymbol{E}_2 \\
&= \underbrace{(u_1 + r \cdot u_2)}_{=u} \cdot (C\boldsymbol{z}) + \underbrace{r \cdot \boldsymbol{T} + \boldsymbol{E}_1 + r^2 \cdot \boldsymbol{E}_2}_{=\boldsymbol{E}} = u \cdot (C\boldsymbol{z}) + \boldsymbol{E}. \ \checkmark
\end{aligned}$$

As illustrative toy examples we show how folding produces non-trivial Relaxed-R1CS instances (with $u \neq 1$ and $\boldsymbol{E} \neq \mathbf{0}$) using the same circuits from Section 4.2.

Example 1: $y = x^2 + 3$**.** Using the same matrices $A, B, C \in \mathbb{F}^{1 \times 3}$ from Example 1 of Section 4.2, as:

$$A = \begin{pmatrix} 0 & 0 & 1 \end{pmatrix}, \quad B = \begin{pmatrix} 0 & 0 & 1 \end{pmatrix}, \quad C = \begin{pmatrix} -3 & 1 & 0 \end{pmatrix}$$

take two satisfying standard instances $\boldsymbol{z}_1 = (1, 12, 3)$ (x=3) and $\boldsymbol{z}_2 = (1, 7, 2)$ (x=2). Both trivially embed into Relaxed-R1CS with $u = 1$, $\boldsymbol{E} = \mathbf{0}$. The cross-term:

$$\boldsymbol{T} = A\boldsymbol{z}_1 \circ B\boldsymbol{z}_2 + A\boldsymbol{z}_2 \circ B\boldsymbol{z}_1 - C\boldsymbol{z}_2 - C\boldsymbol{z}_1 = 3 \cdot 2 + 2 \cdot 3 - 4 - 9 = -1.$$

Folding with a random $r = 2$ (say):

$$\boldsymbol{z} = \boldsymbol{z}_1 + 2\boldsymbol{z}_2 = (3,\ 26,\ 7), \qquad u := u_1 + ru_2 = 3, \qquad \boldsymbol{E} = \boldsymbol{E}_1 + r \cdot \boldsymbol{T} + r^2 \cdot \boldsymbol{E}_2 = 0 + 2 \cdot (-1) + 4 \cdot 0 = -2.$$

Verification: $(A\boldsymbol{z})(B\boldsymbol{z}) = 49$ and $u \cdot C\boldsymbol{z} + E = 3 \cdot 17 - 2 = 49$. ✓

Note that, the folded vector $\boldsymbol{z} = (3, 26, 7)$ has $z_0 = 3 \neq 1$ and $26 \neq 7^2 + 3$—it does **not** correspond to any valid execution of $y = x^2 + 3$, yet it satisfies the Relaxed-R1CS equation. A valid execution of any arithmetic computation must have a standard R1CS representation.

We can **fold again** with a third fresh instance $\boldsymbol{z}_3 = (1, 19, 4)$ (x=4, u_3=1, E_3=0). The cross-term is $\boldsymbol{T}' = 7 \cdot 4 + 4 \cdot 7 - 3 \cdot 16 - 1 \cdot 17 = -9$. With random $r' = 1$ (say):

$$\boldsymbol{z}' = (4,\ 45,\ 11);\ u' = 4;\ \boldsymbol{E}' = -2 - 9 = -11.$$

Verification: $(A\boldsymbol{z}')(B\boldsymbol{z}') = 121$ and $u' \cdot C\boldsymbol{z}' + \boldsymbol{E}' = 4 \cdot 33 - 11 = 121$. ✓

After two folds, $u = 4$ and $E = -11$—these grow with each fold, absorbing the accumulated cross-terms. In NOVA IVC, this accumulation continues for T steps, and the final Relaxed-R1CS instance is compressed by a SNARK, e.g. Groth16.

Example 2: $y = x^3 + 2x + 1$**.** Using the matrices from Example 2 of Section 4.2, $A, B, C \in \mathbb{F}^{2\times 4}$ computed as:

$$A = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad B = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}, \quad C = \begin{pmatrix} 0 & 0 & 0 & 1 \\ -1 & 1 & -2 & 0 \end{pmatrix}$$

fold $\boldsymbol{z}_1 = (1, 34, 3, 9)$ (x=3) and $\boldsymbol{z}_2 = (1, 13, 2, 4)$ (x=2):

$$\begin{aligned} \boldsymbol{T} &:= (A\boldsymbol{z}_1) \circ (B\boldsymbol{z}_2) + (A\boldsymbol{z}_2) \circ (B\boldsymbol{z}_1) - C\boldsymbol{z}_2 - C\boldsymbol{z}_1 \\ &= \begin{pmatrix} 6 \\ 18 \end{pmatrix} + \begin{pmatrix} 6 \\ 12 \end{pmatrix} - \begin{pmatrix} 4 \\ 8 \end{pmatrix} - \begin{pmatrix} 9 \\ 27 \end{pmatrix} = \begin{pmatrix} -1 \\ -5 \end{pmatrix}. \end{aligned}$$

With $r = 1$ (say): $\boldsymbol{z} = (2,\ 47,\ 5,\ 13)$, $u = 2$, $\boldsymbol{E} = (-1,\ -5)^\top$. Verification:

$$(A\boldsymbol{z}) \circ (B\boldsymbol{z}) = \begin{pmatrix} 25 \\ 65 \end{pmatrix}, \qquad u \cdot C\boldsymbol{z} + \boldsymbol{E} = 2 \begin{pmatrix} 13 \\ 35 \end{pmatrix} + \begin{pmatrix} -1 \\ -5 \end{pmatrix} = \begin{pmatrix} 25 \\ 65 \end{pmatrix}. \quad ✓$$

The updatd trace For any efficient function F which takes inputs $\boldsymbol{I}$ **we can define** $\mathsf{trace}_{\mathsf{RxR1}}$ as:

$$(\mathbb{U}, \mathbb{W}) := \mathsf{trace}_{\mathsf{RxR1}}(F, \boldsymbol{I})$$

where $(\mathbb{U}, \mathbb{W})$ is a committed Relaxed-R1CS pair representing a standard R1CS instance: $\mathsf{trace}_{\mathsf{RxR1}}$ first runs $\mathsf{trace}_{\mathsf{R1}}(F, \boldsymbol{I})$ to obtain $(\boldsymbol{x}, \boldsymbol{w})$, then embeds the result into Relaxed-R1CS by setting $u := 1$, $\boldsymbol{E} := \boldsymbol{0}$, sampling $r_W \leftarrow_\$ \mathbb{F}$, setting $r_E := 0$, and computing the commitments $\overline{W} := \mathsf{Com}(\boldsymbol{w}; r_W)$ and $\overline{E} := \mathsf{Com}(\boldsymbol{0}; 0)$.

Applied to **Example 1** ($y = x^2 + 3$) on input $x = 3$: $\mathsf{trace}_{\mathsf{RxR1}}$ produces $\boldsymbol{x} = (12)$, $\boldsymbol{w} = (3)$, and $\mathsf{trace}_{\mathsf{RxR1}}$ packages

$$\mathbb{U} = \big((12),\ u{=}1,\ \overline{E}{=}\mathsf{Com}(\boldsymbol{0}; 0),\ \overline{W}{=}\mathsf{Com}((3); r_W)\big), \qquad \mathbb{W} = \big((3),\ \boldsymbol{0},\ r_W,\ 0\big).$$

Applied to **Example 2** ($y = x^3 + 2x + 1$) on $x = 3$: $\mathsf{trace}_{\mathsf{RxR1}}$ produces $\boldsymbol{x} = (34)$, $\boldsymbol{w} = (3, 9)$ (the intermediate $v = 9$ is part of the witness), and $\mathsf{trace}_{\mathsf{RxR1}}$ packages

$$\mathbb{U} = \big((34),\ u{=}1,\ \overline{E}{=}\mathsf{Com}(\boldsymbol{0}; 0),\ \overline{W}{=}\mathsf{Com}((3, 9); r_W)\big), \qquad \mathbb{W} = \big((3, 9),\ \boldsymbol{0},\ r_W,\ 0\big).$$

In both cases, $\mathbb{U}$ is a fresh instance ($u = 1$, $\overline{E} = \mathsf{Com}(\boldsymbol{0}; 0)$), which is the form checked by dec_fresh inside the augmented circuit F'.

4.5.2 NOVA: Non-Interactive Folding Scheme (NIFS)

We now formalize the folding procedure.[20]

[20] The original NOVA paper [KST22] presents an interactive protocol made non-interactive via Fiat-Shamir. We directly present the non-interactive version, which we will be using in HieroTSS.

NIFS: Ingredients, Parameters, Notations, and Setting

- R1CS structure: matrices $A, B, C \in \mathbb{F}^{m \times n}$.
- Pedersen homomorphic commitment scheme Com over group $\mathbb{G}$.
- Hash function $\mathcal{H}_{\mathsf{fs}} : \{0,1\}^* \to \mathbb{F}$ for Fiat-Shamir.
- Two Relaxed-R1CS instance-witness pairs $(\mathbb{U}_1, \mathbb{W}_1)$ and $(\mathbb{U}_2, \mathbb{W}_2)$, where $\mathbb{U}_i = (\boldsymbol{x}_i, u_i, \overline{E}_i, \overline{W}_i)$ and $\mathbb{W}_i = (\boldsymbol{w}_i, \boldsymbol{E}_i, r_{W_i}, r_{E_i})$.

NIFS: Construction

- NIFS.Prove$(\mathbb{U}_1, \mathbb{W}_1, \mathbb{U}_2, \mathbb{W}_2) \to (\mathbb{U}, \mathbb{W}, \overline{T})$:

  (Compute full assignment vectors)
  - For $i \in [2]$: $(\boldsymbol{x}_i, u_i, \overline{E}_i, \overline{W}_i) := \mathbb{U}_i \quad (\boldsymbol{w}_i, \boldsymbol{E}_i, r_{W_i}, r_{E_i}) := \mathbb{W}_i$;
  - $\boldsymbol{z}_1 := (u_1, \boldsymbol{x}_1, \boldsymbol{w}_1)$; $\quad \boldsymbol{z}_2 := (u_2, \boldsymbol{x}_2, \boldsymbol{w}_2)$

  (Compute the cross-term vector $\boldsymbol{T}$ (cf. Eq 9), and its commitment)
  - $\boldsymbol{T} := (A\boldsymbol{z}_1) \circ (B\boldsymbol{z}_2) + (A\boldsymbol{z}_2) \circ (B\boldsymbol{z}_1) - u_1 \cdot C\boldsymbol{z}_2 - u_2 \cdot C\boldsymbol{z}_1$
  - $r_T \leftarrow_{\$} \mathbb{F}$; $\quad \overline{T} := \mathsf{Com}(\boldsymbol{T}; r_T)$

  (Compute the Fiat-Shamir challenge from public values)
  - $r := \mathcal{H}_{\mathsf{fs}}(\mathbb{U}_1, \mathbb{U}_2, \overline{T})$

  (Fold the instances)
  - $\boldsymbol{x} := \boldsymbol{x}_1 + r \cdot \boldsymbol{x}_2$; $\quad u := u_1 + r \cdot u_2$
  - $\overline{E} := \overline{E}_1 \cdot \overline{T}^{\,r} \cdot \overline{E}_2^{\,r^2}$; $\quad \overline{W} := \overline{W}_1 \cdot \overline{W}_2^{\,r}$

  (Note: Uses homomorphism: $\overline{E} = \mathsf{Com}(\boldsymbol{E}_1 + r\boldsymbol{T} + r^2\boldsymbol{E}_2;\ r_{E_1} + r\, r_T + r^2 r_{E_2})$)
  - $\mathbb{U} := (\boldsymbol{x}, u, \overline{E}, \overline{W})$

  (Fold the witnesses)
  - $\boldsymbol{w} := \boldsymbol{w}_1 + r \cdot \boldsymbol{w}_2$; $\quad \boldsymbol{E} := \boldsymbol{E}_1 + r \cdot \boldsymbol{T} + r^2 \cdot \boldsymbol{E}_2$
  - $r_W := r_{W_1} + r \cdot r_{W_2}$; $\quad r_E := r_{E_1} + r \cdot r_T + r^2 \cdot r_{E_2}$
  - $\mathbb{W} := (\boldsymbol{w}, \boldsymbol{E}, r_W, r_E)$

- NIFS.Verify$(\mathbb{U}_1, \mathbb{U}_2, \overline{T}) \to \mathbb{U}$:

  (Recompute the Fiat-Shamir challenge)
  - $r := \mathcal{H}_{\mathsf{fs}}(\mathbb{U}_1, \mathbb{U}_2, \overline{T})$

  (Fold the instances---same as the prover, but without witness)
  - $\boldsymbol{x} := \boldsymbol{x}_1 + r \cdot \boldsymbol{x}_2$; $\quad u := u_1 + r \cdot u_2$
  - $\overline{E} := \overline{E}_1 \cdot \overline{T}^{\,r} \cdot \overline{E}_2^{\,r^2}$; $\quad \overline{W} := \overline{W}_1 \cdot \overline{W}_2^{\,r}$
  - $\mathbb{U} := (\boldsymbol{x}, u, \overline{E}, \overline{W})$

Remark 2. The verifier performs only group scalar multiplications and additions—no R1CS evaluation, no pairings. This is the key efficiency advantage of folding.

Correctness. If both $(\mathbb{U}_1, \mathbb{W}_1)$ and $(\mathbb{U}_2, \mathbb{W}_2)$ satisfy Relaxed-R1CS, then the folded pair $(\mathbb{U}, \mathbb{W})$ also satisfies

Relaxed-R1CS:

$$\begin{aligned}(A\boldsymbol{z}) \circ (B\boldsymbol{z}) &= (u_1 C\boldsymbol{z}_1 + \boldsymbol{E}_1) + r \cdot \boldsymbol{T} + r^2 \cdot (u_2 C\boldsymbol{z}_2 + \boldsymbol{E}_2) + r \cdot (u_1 C\boldsymbol{z}_2 + u_2 C\boldsymbol{z}_1) \\ &= (u_1 + ru_2) \cdot C(\boldsymbol{z}_1 + r\boldsymbol{z}_2) + (\boldsymbol{E}_1 + r\boldsymbol{T} + r^2\boldsymbol{E}_2) = u \cdot C\boldsymbol{z} + \boldsymbol{E}. \quad \checkmark\end{aligned}$$

Soundness. The folding scheme satisfies knowledge soundness. By the Schwartz-Zippel lemma, a non-trivial polynomial of degree 2 in r can have at most 2 roots in $\mathbb{F}$, so the probability of a cheating prover succeeding is at most $2/|\mathbb{F}| = \mathsf{negl}(\kappa)(\kappa)$.

4.6 NOVA: Incrementally Verifiable Computation (IVC)

An IVC scheme for a function $F : \mathbb{F}^d \to \mathbb{F}^d$ allows a prover to iteratively compute $\boldsymbol{\psi}_0 \xrightarrow{F} \boldsymbol{\psi}_1 \xrightarrow{F} \cdots \xrightarrow{F} \boldsymbol{\psi}_T$ and at each step t produce a proof π_t attesting that $\boldsymbol{\psi}_t = F^{(t)}(\boldsymbol{\psi}_0)$, where the proof size and verification time are $O(1)$, independent of T. An IVC scheme has the following syntax:

- $\mathsf{IVC.Setup}(F) \to \mathsf{pp}$. Setup outputs public parameters.
- $\mathsf{IVC.Prove}(\mathsf{pp}, t, \boldsymbol{\psi}_0, \boldsymbol{\psi}_t, \pi_{t-1}) \to \pi_t$. Given the previous proof (or $\perp$ if $t = 1$), produce a new proof.
- $\mathsf{IVC.Verify}(\mathsf{pp}, t, \boldsymbol{\psi}_0, \boldsymbol{\psi}_t, \pi_t) \to \mathsf{dec\_bit}$. Verify the proof.

How NOVA constructs IVC from folding The NIFS folding scheme (Section 4.5.2) gives us a way to compress two Relaxed-R1CS instances into one. The idea behind NOVA IVC is to use this as the "recursive step": rather than producing an expensive SNARK proof at each step and then verifying it inside the next circuit (as in traditional recursive SNARKs), the prover simply *folds* the current step's instance into a running accumulator. Since the NIFS verifier consists only of scalar multiplications and group additions—no heavy operations like pairings—it is cheap to encode as an R1CS circuit.[21] Concretely, the prover maintains two variables across steps:

1. A *running accumulated* Relaxed-R1CS instance $\mathbb{U}_{\mathsf{acc}}$ (with potentially non-trivial u and $\boldsymbol{E}$), which "summarizes" all previous steps (though its size is independent of them).
2. A *fresh* standard R1CS instance $\mathbb{U}_t$ (with $u = 1$, $\boldsymbol{E} = \boldsymbol{0}$) for the current step.

At each step t, the prover:

(a) Folds $\mathbb{U}_{\mathsf{acc}}^{(t-1)}$ with the previous step's fresh instance $\mathbb{U}_{t-1}$ using NIFS.Prove, producing a new accumulated instance $\mathbb{U}_{\mathsf{acc}}^{(t)}$.

(b) Computes $\boldsymbol{\psi}_t = F(\boldsymbol{\psi}_{t-1})$ and produces a new fresh instance $\mathbb{U}_t$ for the *augmented* circuit F' as described below. We denote the augmentation by $F' := \mathsf{Augment}(F)$.

Broadly, the augmented circuit F' enforces two things simultaneously: (i) that $\boldsymbol{\psi}_{t+1} = F(\boldsymbol{\psi}_t)$ (the actual computation), and (ii) that the folding was performed correctly, i.e. $\mathbb{U}_{\mathsf{acc}}^{(t)} = \mathsf{NIFS.Verify}(\mathbb{U}_{\mathsf{acc}}^{(t-1)}, \mathbb{U}_t, \overline{T})$. Since $\mathbb{U}_t$ attests to the correctness of the fold that produced $\mathbb{U}_{\mathsf{acc}}^{(t+1)}$, and $\mathbb{U}_{\mathsf{acc}}^{(t+1)}$ in turn "contains" all previous instances, the chain of trust propagates back to "genesis". After any t steps, verifying the IVC proof requires checking: (1) the accumulated instance $\mathbb{U}_{\mathsf{acc}}^{(t)}$ satisfies the Relaxed-R1CS relation, and (2) the last fresh instance $\mathbb{U}_t$ satisfies standard R1CS.[22] We now describe the augmented circuit F', then the formal NOVA IVC construction, and conclude with a worked example that traces every component.

Additional Notation. Given a Relaxed-R1CS instance $\mathbb{U} = (\boldsymbol{x}, u, \overline{E}, \overline{W})$, we use dot notation to access its components: $\mathbb{U}.\boldsymbol{x}$ for the public input, $\mathbb{U}.u$ for the scalar, $\mathbb{U}.\overline{E}$ for the error commitment, and $\mathbb{U}.\overline{W}$ for the witness commitment. A trivial/default instance is denoted by $\mathbb{U}_\perp$, for which $\mathbb{U}_\perp.\overline{E} = \mathsf{Com}(\boldsymbol{0}; 0)$; the corresponding witness is denoted by $\mathbb{W}_\perp := (\boldsymbol{0}, \boldsymbol{0}, 0, 0)$. If $\mathbb{U}$ is a fresh R1CS instance we have that $\mathbb{U}.u = 1$, $\mathbb{U}.\overline{E} = \mathsf{Com}(\boldsymbol{0}; 0) = \mathbb{U}_\perp.\overline{E}$. Consequently for a fresh instance Below we initialize $\mathbb{U}_{\mathsf{acc}}^{(0)} := \mathbb{U}_\perp$.

[21] A non-trivial part in the NIFS verification is the Fiat-Shamir hash, which is unavoidable for SNARKs too. Using a SNARK friendly hash, such as Poseidon one can make the circuit reasonably efficient.

[22] This verification is $O(m \cdot n)$—not yet succinct. The *decider* (cf. Sec 4.6.3) compresses it to a constant-size Groth16 proof.

4.6.1 The augmented circuit F'

Description of $F' := \mathsf{Augment}(F)$

Ingredients: Two hash functions $\mathcal{H}_{\mathsf{io}}, \mathcal{H}_{\mathsf{fs}}$.

Input:

- pp: the IVC public parameters.
- $t \in \{0, 1, 2, \ldots\}$: the step counter.
- $\boldsymbol{\psi}_0 \in \mathbb{F}^d$: the initial IVC state (genesis).
- $\boldsymbol{\psi}_t \in \mathbb{F}^d$: the IVC state at step-$t$.
- $\mathbb{U}_{\mathsf{acc}}^{(t-1)}$: the running accumulated Relaxed-R1CS instance (folded upto $(t-1)$-th step)
- $\mathbb{U}_t$: the fresh R1CS instance from the previous step.
- $\overline{T} \in \mathbb{G}$: the cross-term commitment.
- aux_t: auxiliary input for F, if any.
- r_t, r_{t+1}: randomnesses for t-th and $(t+1)$-th step.

Description of $F'(\mathsf{pp}, \mathbb{U}_{\mathsf{acc}}^{(t-1)}, \mathbb{U}_t, (t, \boldsymbol{\psi}_0, \boldsymbol{\psi}_t), \mathsf{aux}_t, \overline{T}, (r_t, r_{t+1})) \to \mathsf{out}_{F'}$

- Base case (If $t = 0$):
  1. Compute the first step:
  $$\boldsymbol{\psi}_1 := F(\boldsymbol{\psi}_0, \mathsf{aux}_0)$$
  2. Set output:
  $$\mathsf{out}_{F'} := \mathcal{H}_{\mathsf{io}}(\mathsf{pp}, 1, \boldsymbol{\psi}_0, \boldsymbol{\psi}_1, \mathbb{U}_\perp, r_1))$$
- Recursive case (Else $t \geq 1$) :
  1. Check that the public part of the current fresh instance was correctly formed:
  $$\mathsf{dec\_pub} := \left(\mathbb{U}_t.\boldsymbol{x} \stackrel{?}{=} \mathcal{H}_{\mathsf{io}}(\mathsf{pp}, t, \boldsymbol{\psi}_0, \boldsymbol{\psi}_t, \mathbb{U}_{\mathsf{acc}}^{(t-1)}, r_t)\right)$$
  2. Check that the current fresh instance is a standard (non-relaxed) instance:
  $$\mathsf{dec\_fresh} := \left((\mathbb{U}_t.\overline{E},\ \mathbb{U}_t.u) \stackrel{?}{=} (\mathbb{U}_\perp.\overline{E},\ 1)\right)$$
  3. Compute the new accumulated instance by invoking the NIFS verifier:
  $$\mathbb{U}_{\mathsf{acc}}^{(t)} \leftarrow \mathsf{NIFS.Verify}^{\mathcal{H}_{\mathsf{fs}}}(\mathbb{U}_{\mathsf{acc}}^{(t-1)}, \mathbb{U}_t, \overline{T})$$
  4. Execute one step of the actual computation:
  $$\boldsymbol{\psi}_{t+1} := F(\boldsymbol{\psi}_t, \mathsf{aux}_t)$$
  5. Set output
  $$\mathsf{out}_{F'} := \begin{cases} \mathcal{H}_{\mathsf{io}}\big(\mathsf{pp},\ t{+}1,\ \boldsymbol{\psi}_0,\ \boldsymbol{\psi}_{t+1},\ \mathbb{U}_{\mathsf{acc}}^{(t)}, r_{t+1}\big) & \text{if } \big(\mathsf{dec\_fresh} \wedge \mathsf{dec\_pub}\big) = 1 \\ \perp & \text{otherwise} \end{cases}$$

Output: $\mathsf{out}_{F'}$

Compiling F' into R1CS via trace. We need F' to be represented as a committed Relaxed-R1CS structure.

For that, we use $\mathsf{trace}_{\mathsf{RxR1}}$, as defined above.

$$(\mathbb{U}_{t+1}, \mathbb{W}_{t+1}) \leftarrow \mathsf{trace}_{\mathsf{RxR1}}(F', (\mathsf{pp}, \mathbb{U}_{\mathsf{acc}}^{(t-1)}, \mathbb{U}_t, (t, \boldsymbol{\psi}_0, \boldsymbol{\psi}_t), \mathsf{aux}_t, \overline{T}, (r_t, r_{t+1})))$$

This executes F' on the given inputs and records the execution to produce a satisfying committed Relaxed-R1CS instance-witness pair $(\mathbb{U}_{t+1}, \mathbb{W}_{t+1})$ for the next step:

- The public part of instance $\mathbb{U}_{t+1}$ is the output $\mathsf{out}_{F'}$ of F'.
- The witness / private part $\mathbb{W}_{t+1}$ contains all inputs to F' plus the intermediate values.
- Since $\mathbb{U}_{t+1}$ is produced by running F', it is always a standard R1CS instance: $\mathbb{U}_{t+1}.u = 1$ and $\mathbb{U}_{t+1}.\overline{E} = \mathbb{U}_{t+1}.\overline{E} = \mathsf{Com}(\mathbf{0}; 0)$.

Circuit complexity. The augmented circuit F' has R1CS constraints from three sources: (a) the step function F itself, (b) the NIFS verifier (scalar multiplications over the group, encoded as R1CS constraints over the scalar field), and (c) two hash evaluations ($\mathcal{H}_{\mathsf{io}}$ and $\mathcal{H}_{\mathsf{fs}}$).

Component	**Approximate R1CS constraint count**
Step function F	application-dependent
NIFS verifier (2 scalar muls in $\mathbb{G}$)	$\approx$ 10,000–20,000
Two Poseidon hashes ($\mathcal{H}_{\mathsf{io}}$, $\mathcal{H}_{\mathsf{fs}}$)	$\approx$ 5,000–10,000
Conditional logic (base case flag)	$\approx$ 100
Total overhead (beyond F)	$\approx$ 15,000–30,000

Note that, the overhead is **constant per step**—the same (A', B', C') matrices, generated from F', are reused at every IVC step, and the cost does not grow with T.

Soundness. Intuitively, the public input of the fresh instance $\mathbb{U}_t$ at step t is $\mathbb{U}_t.\boldsymbol{x} = \mathcal{H}_{\mathsf{io}}(\mathsf{pp}, t, \boldsymbol{\psi}_0, \boldsymbol{\psi}_t, \mathbb{U}_{\mathsf{acc}}^{(t-1)}, r_t)$, which commits to the genesis state, the current output, and the previous accumulated instance. Step 5 ensures this hash was honestly computed inside the circuit. Step 1 ensures that the *previous* hash $\boldsymbol{x}_{t-1}$ was also honestly formed. Moreover, Step 3 ensures that the folding was done correctly, and Step 2 ensures that the new instance is legitimate. Together, these form an inductive chain: if the verifier checks $\mathbb{U}_t$ and $\mathbb{U}_{\mathsf{acc}}^{(t)}$ at any point of time, the chain of trust propagates back to the genesis state $\boldsymbol{\psi}_0$.

4.6.2 The IVC Construction

NOVA IVC: Ingredients, Parameters, Notations, and Setting

- A step function $F : \mathbb{F}^d \to \mathbb{F}^d$ (possibly with auxiliary input aux), encoded as R1CS with matrices A, B, C and the initial state $\boldsymbol{\psi}_0 \in \mathbb{F}^d$.
- A non-interactive folding scheme $\mathsf{NIFS} = (\mathsf{NIFS.Prove}, \mathsf{NIFS.Verify})$ for committed Relaxed-R1CS (cf. Section 4.5.2).
- Pedersen commitment scheme $\mathsf{Com}(\cdot\,;\cdot)$ over cyclic group $\mathbb{G}$ with generators $g, h_1, \ldots, h_m$ (cf. Section 4.4).
- Hash functions $\mathcal{H}_{\mathsf{io}}, \mathcal{H}_{\mathsf{fs}}$.

NOVA IVC: Construction

- $\mathsf{IVC.Setup}(F) \to \mathsf{pp}$:
  - $F' := \mathsf{Augment}(F)$
  - $(A', B', C') := \mathsf{R1CS}(F')$
  - $\mathsf{pp} := (g, h_1, \ldots, h_m,\ A', B', C')$

- $\mathsf{IVC.Prove}(\mathsf{pp}, t, \boldsymbol{\psi}_0, \boldsymbol{\psi}_t, \pi_{t-1}) \to \pi_t$:
  - If $t = 0$:

    (Run F' via trace to produce the first fresh instance-witness pair.)
    - $r_1 \leftarrow_{\$} \mathbb{F}$
    - $(\mathbb{U}_1, \mathbb{W}_1) \leftarrow \mathsf{trace}(F',\ (\mathsf{pp}, \mathbb{U}_\perp, \mathbb{U}_\perp, (0, \boldsymbol{\psi}_0, \boldsymbol{\psi}_0), \mathsf{aux}_0, [0], (0, r_1)))$

      (Note: F' takes the base-case branch: computes $\boldsymbol{\psi}_1 = F(\boldsymbol{\psi}_0, \mathsf{aux}_0)$ and sets $\mathsf{out}_{F'} = \mathcal{H}_{\mathsf{io}}(\mathsf{pp}, 1, \boldsymbol{\psi}_0, \boldsymbol{\psi}_1, \mathbb{U}_\perp, r_1)$. The trace produces a committed Relaxed-R1CS instance $\mathbb{U}_1$ with $\mathbb{U}_1.\boldsymbol{x} = \mathsf{out}_{F'}$, $\mathbb{U}_1.u = 1$, $\mathbb{U}_1.\overline{E} = \mathbb{U}_\perp.\overline{E}$, and the corresponding witness $\mathbb{W}_1$.)

      (Initialize the accumulated instance with a trivial instance)
    - $\mathbb{U}_{\mathsf{acc}}^{(0)} := \mathbb{U}_\perp$, and $\mathbb{W}_{\mathsf{acc}}{}^{(0)} := (\mathbf{0}, \mathbf{0}, 0, 0)$

      (Output:)
    - $\pi_0 := (\mathbb{U}_{\mathsf{acc}}^{(0)}, \mathbb{W}_{\mathsf{acc}}{}^{(0)}, \mathbb{U}_1, \mathbb{W}_1, r_1)$
  - Else if $t \geq 1$:
    - $(\mathbb{U}_{\mathsf{acc}}^{(t-1)}, \mathbb{W}_{\mathsf{acc}}{}^{(t-1)}, \mathbb{U}_t, \mathbb{W}_t, r_t) := \pi_{t-1}$

      (Fold the accumulated instance with the previous fresh instance)
    - $(\mathbb{U}_{\mathsf{acc}}^{(t)}, \mathbb{W}_{\mathsf{acc}}{}^{(t)}, \overline{T}_t) := \mathsf{NIFS.Prove}(\mathbb{U}_{\mathsf{acc}}^{(t-1)}, \mathbb{W}_{\mathsf{acc}}{}^{(t-1)}, \mathbb{U}_t, \mathbb{W}_t)$

      (Note: After this fold, $\mathbb{U}_{\mathsf{acc}}^{(t)}$ is a Relaxed-R1CS instance that ''summarizes'' steps 1 through t.)

      (Run F' via trace with inputs corresponding to the current instance:)
    - $r_{t+1} \leftarrow_{\$} \mathbb{F}$
    - $(\mathbb{U}_{t+1}, \mathbb{W}_{t+1}) \leftarrow \mathsf{trace}_{\mathsf{RxR1}}(F', (\mathsf{pp}, \mathbb{U}_{\mathsf{acc}}^{(t-1)}, \mathbb{U}_t, (t, \boldsymbol{\psi}_0, \boldsymbol{\psi}_t), \mathsf{aux}_t, \overline{T}, (r_t, r_{t+1})))$

      (Note: F' takes the recursive branch: and sets $\mathsf{out}_{F'}$ (Step 5). The trace records this execution to produce $\mathbb{U}_{t+1}$ and $\mathbb{W}_{t+1}$. If any of the check fails the output will not be a valid R1CS instance, and will fail the next check.)
  - $\pi_t := (\mathbb{U}_{\mathsf{acc}}^{(t)}, \mathbb{W}_{\mathsf{acc}}{}^{(t)}, \mathbb{U}_{t+1}, \mathbb{W}_{t+1}, r_{t+1})$
- $\mathsf{IVC.Verify}(\mathsf{pp}, t, \boldsymbol{\psi}_0, \boldsymbol{\psi}_{t+1}, \pi_t) \to \mathsf{dec\_bit}$:
  - $(\mathbb{U}_{\mathsf{acc}}^{(t)}, \mathbb{W}_{\mathsf{acc}}{}^{(t)}, \mathbb{U}_{t+1}, \mathbb{W}_{t+1}, r_{t+1}) := \pi_t$
  - $(g, h_1, \ldots, h_m,\ A', B', C') := \mathsf{pp}$

    (Check that the fresh instance's public part is consistent with the claimed IVC state.)
  - $\mathsf{dec\_io} := \left(\mathbb{U}_{t+1}.\boldsymbol{x} \stackrel{?}{=} \mathcal{H}_{\mathsf{io}}(\mathsf{pp}, t+1, \boldsymbol{\psi}_0, \boldsymbol{\psi}_{t+1}, \mathbb{U}_{\mathsf{acc}}^{(t)}, r_{t+1})\right)$

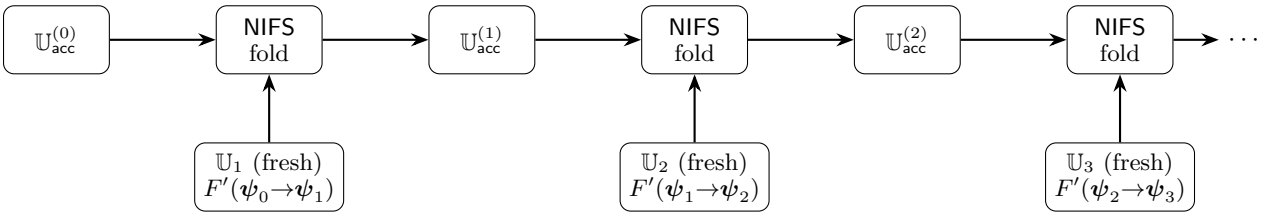
    (Check the Relaxed-R1CS equation for the accumulated instance)
  - $(\boldsymbol{x}_{\mathsf{acc}}, u_{\mathsf{acc}}, \overline{E}_{\mathsf{acc}}, \overline{W}_{\mathsf{acc}}) := \mathbb{U}_{\mathsf{acc}}^{(t)}$
  - $(\boldsymbol{w}_{\mathsf{acc}}, \boldsymbol{E}_{\mathsf{acc}}, r_{W,\mathsf{acc}}, r_{E,\mathsf{acc}}) := \mathbb{W}_{\mathsf{acc}}{}^{(t)}$
  - $\boldsymbol{z}_{\mathsf{acc}} := (u_{\mathsf{acc}}, \boldsymbol{x}_{\mathsf{acc}}, \boldsymbol{w}_{\mathsf{acc}})$
  - $\mathsf{dec\_RlxR1CS} := \left((A'\boldsymbol{z}_{\mathsf{acc}}) \circ (B'\boldsymbol{z}_{\mathsf{acc}}) \stackrel{?}{=} u_{\mathsf{acc}} \cdot C'\boldsymbol{z}_{\mathsf{acc}} + \boldsymbol{E}_{\mathsf{acc}}\right) \wedge \left(\mathsf{Com}(\boldsymbol{E}_{\mathsf{acc}};\ r_{E,\mathsf{acc}}) \stackrel{?}{=} \overline{E}_{\mathsf{acc}}\right) \wedge \left(\mathsf{Com}(\boldsymbol{w}_{\mathsf{acc}};\ r_{W,\mathsf{acc}}) \stackrel{?}{=} \overline{W}_{\mathsf{acc}}\right)$

    (Check the standard R1CS equation for the fresh instance)
  - $(\boldsymbol{x}, u, \overline{E}, \overline{W}) := \mathbb{U}_{t+1}$
  - $(\boldsymbol{w}, \boldsymbol{E}, r_W, r_E) := \mathbb{W}_{t+1}$

- $\boldsymbol{z} := (u, \boldsymbol{x}, \boldsymbol{w})$
- $\mathsf{dec\_R1CS} := \Big((A'\boldsymbol{z}) \circ (B'\boldsymbol{z}) \stackrel{?}{=} C'\boldsymbol{z}\Big) \wedge \Big(u \stackrel{?}{=} 1\Big) \wedge \Big(\boldsymbol{E} \stackrel{?}{=} \boldsymbol{0}\Big) \wedge \Big(r_w \stackrel{?}{=} r_E \stackrel{?}{=} 0\Big) \wedge \Big(\overline{E} \stackrel{?}{=} \overline{W} \stackrel{?}{=} \mathsf{Com}(\boldsymbol{0}; 0)\Big)$

  (Compute final decision bit)
- $\mathsf{dec\_bit} := \mathsf{dec\_io} \wedge \mathsf{dec\_RlxR1CS} \wedge \mathsf{dec\_R1CS}$

IVC **state diagram.** The following diagram illustrates the data flow:



4.6.3 The Decider: Compressing the IVC Proof using Groth16

The NOVA IVC proof $\pi_t = (\mathbb{U}_{\mathsf{acc}}^{(t)}, \mathbb{W}_{\mathsf{acc}}{}^{(t)}, \mathbb{U}_{t+1}, \mathbb{W}_{t+1}, r_{t+1})$ contains two committed Relaxed-R1CS instance-witness pairs: the accumulator $(\mathbb{U}_{\mathsf{acc}}^{(t)}, \mathbb{W}_{\mathsf{acc}}{}^{(t)})$, which summarizes the folds of $\mathbb{U}_1, \ldots, \mathbb{U}_t$, and the fresh instance $(\mathbb{U}_{t+1}, \mathbb{W}_{t+1})$, which attests to the correctness of step t's F' execution (including the fact that the most recent fold was computed correctly). Verifying π_t directly via IVC.Verify costs $O(m \cdot n)$, dominated by checking the Relaxed-R1CS relation and both commitment openings.

Following the fold-then-SNARK decomposition of NOVA, the Decider splits IVC.Verify into relatively cheap work performed by the external verifier and a single Relaxed-R1CS satisfiability check wrapped in a Groth16 proof. The prover folds $(\mathbb{U}_{\mathsf{acc}}^{(t)}, \mathbb{W}_{\mathsf{acc}}{}^{(t)})$ and $(\mathbb{U}_{t+1}, \mathbb{W}_{t+1})$ into a single pair $(\mathbb{U}'_{\mathsf{acc}}, \mathbb{W}_{\mathsf{acc}}{}')$ using NIFS.Prove, then produces a Groth16 proof attesting only that $(\mathbb{U}'_{\mathsf{acc}}, \mathbb{W}_{\mathsf{acc}}{}')$ satisfies Relaxed-R1CS. The external verifier performs the hash binding check (dec_io), the freshness check on $\mathbb{U}_{t+1}$ (dec_R1CS), recomputes $\mathbb{U}'_{\mathsf{acc}}$ via NIFS.Verify, and runs Groth16.Verify on the folded instance.

Compared to encoding the entire IVC.Verify inside the circuit, this decomposition approximately halves the number of commitment-opening constraints inside C_{dec} (one folded instance rather than both $\mathbb{U}_{\mathsf{acc}}^{(t)}$ and $\mathbb{U}_{t+1}$), and removes the hash computation (which would have been hugely expensive) and freshness check from the circuit entirely.

Decider: Ingredients, Parameters, Notations, and Setting

- NOVA IVC scheme with public parameters $\mathsf{pp} = (g, h_1, \ldots, h_m, A', B', C')$.
- Non-interactive folding scheme NIFS = (NIFS.Prove, NIFS.Verify) for committed Relaxed-R1CS (cf. Section 4.5.2).
- Groth16 proof system (Groth16.Setup, Groth16.Prove, Groth16.Verify) (cf. Section 4.3).
- Decider circuit C_{dec} with public input $\mathbb{U}'_{\mathsf{acc}} = (\boldsymbol{x}', u', \overline{E}', \overline{W}')$ and witness $(\boldsymbol{w}', \boldsymbol{E}', r_{W'}, r_{E'})$. The circuit enforces:
  - $\mathsf{Com}(\boldsymbol{E}'; r_{E'}) \stackrel{?}{=} \overline{E}'$
  - $\mathsf{Com}(\boldsymbol{w}'; r_{W'}) \stackrel{?}{=} \overline{W}'$
  - $(A'\boldsymbol{z}') \circ (B'\boldsymbol{z}') \stackrel{?}{=} u' \cdot (C'\boldsymbol{z}') + \boldsymbol{E}'$, where $\boldsymbol{z}' = (u', \boldsymbol{x}', \boldsymbol{w}')$

Decider: Construction

- $\mathsf{Decider.Setup}(\mathsf{pp}) \to (pk_{\mathsf{G}}, vk_{\mathsf{G}})$:
  - $(A, B, C) := \mathsf{R1CS}(C_{\mathsf{dec}})$
  - $(pk_{\mathsf{G}}, vk_{\mathsf{G}}) := \mathsf{Groth16.Setup}(A, B, C)$
- $\mathsf{Decider.Prove}(pk_{\mathsf{G}}, (t, \boldsymbol{\psi}_0, \boldsymbol{\psi}_{t+1}, \pi_t)) \to \pi_{\mathsf{dec}}$:
  - Parse π_t as $(\mathbb{U}_{\mathsf{acc}}^{(t)}, \mathbb{W}_{\mathsf{acc}}{}^{(t)}, \mathbb{U}_{t+1}, \mathbb{W}_{t+1}, r_{t+1})$.
  - $(\mathbb{U}'_{\mathsf{acc}}, \mathbb{W}_{\mathsf{acc}}{}', \overline{T}) := \mathsf{NIFS.Prove}\big(\mathsf{pp}, (\mathbb{U}_{\mathsf{acc}}^{(t)}, \mathbb{W}_{\mathsf{acc}}{}^{(t)}), (\mathbb{U}_{t+1}, \mathbb{W}_{t+1})\big)$
    (Fold the two pairs into a single committed Relaxed-R1CS instance-witness pair; $\overline{T}$ is the cross-term commitment.)
  - $(\boldsymbol{x}, \boldsymbol{w}) := \mathsf{trace}_{\mathsf{R1}}(C_{\mathsf{dec}}, \mathbb{U}'_{\mathsf{acc}}, \mathbb{W}_{\mathsf{acc}}{}')$
  - $\pi_{\mathbb{U}'_{\mathsf{acc}}} := \mathsf{Groth16.Prove}(pk_{\mathsf{G}}, \boldsymbol{x}, \boldsymbol{w})$
  - Output $\pi_{\mathsf{dec}} := (\mathbb{U}_{\mathsf{acc}}^{(t)}, \mathbb{U}_{t+1}, r_{t+1}, \overline{T}, \pi_{\mathbb{U}'_{\mathsf{acc}}})$.
- $\mathsf{Decider.Verify}(vk_{\mathsf{G}}, (t, \boldsymbol{\psi}_0, \boldsymbol{\psi}_{t+1}, \pi_{\mathsf{dec}})) \to \mathsf{dec\_bit}$:
  - Parse π_{dec} as $(\mathbb{U}_{\mathsf{acc}}^{(t)}, \mathbb{U}_{t+1}, r_{t+1}, \overline{T}, \pi_{\mathbb{U}'_{\mathsf{acc}}})$.
    (Hash binding check:)
  - $\mathsf{dec\_io} := \big(\mathbb{U}_{t+1}.\boldsymbol{x} \stackrel{?}{=} \mathcal{H}_{\mathsf{io}}(\mathsf{pp}, t+1, \boldsymbol{\psi}_0, \boldsymbol{\psi}_{t+1}, \mathbb{U}_{\mathsf{acc}}^{(t)}, r_{t+1})\big)$
    (Freshness check on the fresh instance:)
  - $\mathsf{dec\_R1CS} := (\mathbb{U}_{t+1}.\overline{E} \stackrel{?}{=} \mathbb{U}_{\perp}.\overline{E}) \wedge (\mathbb{U}_{t+1}.u \stackrel{?}{=} 1)$
    (Recompute the folded instance using the cross-term from π_{dec}:)
  - $\mathbb{U}'_{\mathsf{acc}} := \mathsf{NIFS.Verify}(\mathsf{pp}, \mathbb{U}_{\mathsf{acc}}^{(t)}, \mathbb{U}_{t+1}, \overline{T})$
  - $\boldsymbol{x} := \mathsf{trace}_{\mathsf{R1}}(C_{\mathsf{dec}}, \mathbb{U}'_{\mathsf{acc}})$ (public-input encoding of $\mathbb{U}'_{\mathsf{acc}}$)
  - $\mathsf{dec\_RlxR1CS} := \mathsf{Groth16.Verify}(vk_{\mathsf{G}}, \boldsymbol{x}, \pi_{\mathbb{U}'_{\mathsf{acc}}})$
  - $\mathsf{dec\_bit} := \mathsf{dec\_io} \wedge \mathsf{dec\_R1CS} \wedge \mathsf{dec\_RlxR1CS}$

(The external verifier's work is dominated by the Groth16 pairing check. The hash binding check, the freshness check, and NIFS.Verify each cost $O(1)$ field/group operations. The heavy Relaxed-R1CS satisfiability and commitment-opening check sits inside C_{dec} and is verified succinctly via the single Groth16 proof $\pi_{\mathbb{U}'_{\mathsf{acc}}}$.)

Comparison with the monolithic approach. A naive decider would encode all of IVC.Verify — the hash check, both Relaxed-R1CS openings for $(\mathbb{U}_{\mathsf{acc}}^{(t)}, \mathbb{W}_{\mathsf{acc}}{}^{(t)})$, both R1CS openings for $(\mathbb{U}_{t+1}, \mathbb{W}_{t+1})$, the freshness check, and both satisfiability relations — inside C_{dec}. The Nova-style fold-then-SNARK decomposition above reduces the in-circuit cost in two ways:

- **One opening instead of two.** The dominant cost of C_{dec} is the Pedersen opening check, which encodes an $O(m)$-sized multi-scalar multiplication as R1CS constraints. Folding first means only the openings of $\overline{W}'$ and $\overline{E}'$ are enforced in-circuit, rather than openings for both $\mathbb{U}_{\mathsf{acc}}^{(t)}$ and $\mathbb{U}_{t+1}$.
- **Hash and freshness are free.** $\mathcal{H}_{\mathsf{io}}$ and the comparisons $(\mathbb{U}_{t+1}.\overline{E} \stackrel{?}{=} \mathbb{U}_{\perp}.\overline{E})$, $(\mathbb{U}_{t+1}.u \stackrel{?}{=} 1)$ move out of the circuit and are re-run directly by the external verifier at $O(1)$ cost.

The tradeoff is that π_{dec} is no longer a single Groth16 triple: it additionally carries $(\mathbb{U}_{\mathsf{acc}}^{(t)}, \mathbb{U}_{t+1}, r_{t+1}, \overline{T})$, i.e., two committed Relaxed-R1CS instances, a hash randomizer, and one cross-term commitment. All of these remain $O(1)$-sized, so π_{dec} is still succinct.

Remark (on opening-cost elimination). NOVA itself avoids the in-circuit opening cost *entirely* by instantiating the final SNARK with a Spartan-based zkSNARK for committed Relaxed-R1CS (cf. Section 6 of the NOVA paper), whose verifier handles commitment openings natively via a polynomial commitment scheme rather than via circuit constraints. We deliberately choose Groth16 instead, trading the linear-sized opening check inside C_{dec} for a constant-size on-chain verifier and pairing-based verification — a reasonable tradeoff for the HieroTSS deployment, where the external verifier is a smart contract.

4.7 CycleFold: Eliminating Non-Native Arithmetic

The augmented circuit $F' := \mathsf{Augment}(F)$ from Section 4.6.1 encodes, among other things, the NIFS.Verify computation at each IVC step. Before introducing CycleFold [KS23], we open up F' to identify which sub-operations are native over the circuit's field and which require non-native (base-field) emulation; the latter dominate the per-step cost and are precisely what CycleFold removes.

Anatomy of F' without CycleFold. We work over elliptic curve groups. Each curve EC has a base field and a different scalar field. So, we use the curve EC_1 as the Pedersen commitment group $\mathbb{G} \subseteq \mathsf{EC}_1(\mathbb{F}_2)$, so that EC_1 has base field $\mathbb{F}_2$ and scalar field $\mathbb{F}_1$, which is $\mathbb{G}$'s scalar field. The R1CS circuit F' is defined over $\mathbb{F}_1$, hence the witness, which are inputs to the Pedersen commitments, are also in the same field. At step $t \geq 1$, F' receives $(\mathsf{pp}, \mathbb{U}_{\mathsf{acc}}^{(t-1)}, \mathbb{U}_t, (t, \boldsymbol{\psi}_0, \boldsymbol{\psi}_t), \mathsf{aux}_t, \overline{T}, (r_t, r_{t+1}))$ as input and returns $\mathsf{out}_{F'}$ attesting to the correctness of the recursive computation up to $\boldsymbol{\psi}_{t+1}$. Elaborating the description of F' (cf. the $\mathsf{Augment}(F)$ box in Section 4.6.1) gives the atomic steps listed below; steps (b)–(d) together implement NIFS.Verify, (a) and (f) are the input/output hash-consistency checks, (e) applies the step function, and (g) imposes a simple if-else decision step.

(a) **IO hash check.** Verifies $\mathbb{U}_t.\boldsymbol{x} \stackrel{?}{=} \mathcal{H}_{\mathsf{io}}(\mathsf{pp}, t, \boldsymbol{\psi}_0, \boldsymbol{\psi}_t, \mathbb{U}_{\mathsf{acc}}^{(t-1)}, r_t)$. *Native over* $\mathbb{F}_1$: the hash function (implemented with Poseidon) takes $\mathbb{F}_1$-elements as input. A small, fixed non-native cost arises from hashing the $\mathbb{F}_2$-coordinates of the group element $\mathbb{U}_{\mathsf{acc}}^{(t-1)}$.[23]

(b) **Fiat–Shamir challenge.** Computes $r := \mathcal{H}_{\mathsf{fs}}(\mathbb{U}_{\mathsf{acc}}^{(t-1)}, \mathbb{U}_t, \overline{T}) \in \mathbb{F}_1$. *Native over* $\mathbb{F}_1$, modulo the same hashing overhead as (a).

(c) **Field-part fold.** Computes $\boldsymbol{x}' := \boldsymbol{x}_1 + r \cdot \boldsymbol{x}_2$ and $u' := u_1 + r \cdot u_2$. *Fully native over* $\mathbb{F}_1$: linear combinations of $\mathbb{F}_1$-scalars.

(d) **Commitment-part fold.** Computes

$$\overline{E} := \overline{E}_1 \cdot \overline{T}^{r} \cdot \overline{E}_2^{r^2}, \tag{10}$$

$$\overline{W} := \overline{W}_1 \cdot \overline{W}_2^{r}. \tag{11}$$

Non-native over $\mathbb{F}_1$. Since Pedersen commitments are over the cyclic group $\mathbb{G}$, these are a number of group operations. Because every group element has coordinates in $\mathbb{F}_2$, double-and-add (aka square-and-multiply) inside an $\mathbb{F}_1$-R1CS (which is native to F_1') requires emulating $\mathbb{F}_2$-arithmetic: $\approx \kappa$ non-native $\mathbb{F}_2$-multiplications per scalar multiplication, each costing $\approx$ 5–10 $\mathbb{F}_1$-constraints, for an aggregate of $\approx$ 10,000–20,000 $\mathbb{F}_1$-constraints per NIFS.Verify call.

(e) **Step-function evaluation.** Computes $\boldsymbol{\psi}_{t+1} := F(\boldsymbol{\psi}_t, \mathsf{aux}_t)$. *Native over* $\mathbb{F}_1$, since F is an $\mathbb{F}_1$-circuit.

(f) **Output IO hash.** Computes $\mathsf{out}_{F'} := \mathcal{H}_{\mathsf{io}}(\mathsf{pp}, t{+}1, \boldsymbol{\psi}_0, \boldsymbol{\psi}_{t+1}, \mathbb{U}_{\mathsf{acc}}^{(t)}, r_{t+1})$. Same native/non-native split as (a).

(g) **Base-vs. recursive branch.** Simple if-else between the base-case output (at $t = 0$) and the recursive output. *Native over* $\mathbb{F}_1$.

[23] Hashing a group element $P = (P_x, P_y) \in \mathsf{EC}_1(\mathbb{F}_2)$ inside an $\mathbb{F}_1$-circuit means hashing its coordinates, which live in the *wrong* field: the Poseidon sponge state is over $\mathbb{F}_1$, but $P_x, P_y \in \mathbb{F}_2$. The standard fix is to bit-decompose each coordinate into $\lceil \log_2 p_2 \rceil \approx 254$ booleans (one $\mathbb{F}_1$-constraint per bit), then repack into two $\mathbb{F}_1$ parts. Soundness requires an additional range check enforcing that the reconstructed integer is strictly less than p_2; otherwise the prover could "equivocate" between two different representations of the same $\mathbb{F}_2$-element. The total cost is $O(\log p_2)$ constraints per coordinate, a few thousand per step in aggregate. This is much cheaper than step (d) because here we only *represent* $\mathbb{F}_2$-elements inside $\mathbb{F}_1$-constraints; we never *multiply* in $\mathbb{F}_2$. This cost persists under CycleFold.

Clearly, step (d) is the *only* sub-operation with a significant non-native (for $\mathbb{F}_1$) operation: (a), (b), (f) contribute a constant overhead and (c), (e), (g) are fully native. Extracting (d) into a circuit over a field where group operations over $\mathbb{G}$ are native therefore eliminates the dominant cost. This is the CycleFold idea.

Curve cycle. A cycle of elliptic curves consists of a pair $(\mathsf{EC}_1, \mathsf{EC}_2)$ satisfying:

- EC_1 over base field $\mathbb{F}_2$ with scalar field $\mathbb{F}_1$;
- EC_2 over base field $\mathbb{F}_1$ with scalar field $\mathbb{F}_2$.

The scalar field of each curve equals the base field of the other.[24] A canonical instance for our deployment is the BN254/Grumpkin cycle: BN254 $= \mathsf{EC}_1$ is the pairing-friendly curve used by hinTS and Groth16, and Grumpkin $= \mathsf{EC}_2$ completes the cycle (Remark 3).

The CycleFold split. CycleFold splits F' into two circuits, F'_1 and F'_2, working natively over $\mathbb{F}_1$ and $\mathbb{F}_2$ respectively. Before we describe them, a brief notational note is in order. Throughout this subsection, NIFS.Prove and NIFS.Verify always refer to the *same* folding scheme of Section 4.5.2; when applied to variables for the primary execution thread (e.g. $\mathbb{U}_{\mathsf{acc}}^{(t,1)}, \mathbb{U}_{t,1}$) it folds over $\mathbb{G} \subseteq \mathsf{EC}_1(\mathbb{F}_2)$, and when applied to variables for the secondary execution thread (e.g. $\mathbb{U}_{\mathsf{acc}}^{(t,2)}, \mathbb{U}_{t,2}$) it folds over $\mathsf{EC}_2(\mathbb{F}_1)$. The algorithm is identical in both cases; the execution thread is clear from the operand subscripts. Now we describe the two circuits.

1. **Primary circuit F'_1 over $\mathbb{F}_1$**: this circuit retains the sub-operations (a)–(c) and (e)–(g) of F' natively, exactly as before. In place of (d), it *defers* the three group operations (over $\mathbb{G}$) to a "secondary instance" $\mathbb{U}_{t,2}$ of the circuit F'_2 (described next), whose public input $\boldsymbol{x}_{t,2}$ (detailed below) carries the values needed to compute (10)–(11). Mirroring the way the original F' from Section 4.6.1 updates its accumulated proof, F'_1 *updates* the secondary accumulator by executing $\mathbb{U}_{\mathsf{acc}}^{(t,2)} := \mathsf{NIFS.Verify}(\mathbb{U}_{\mathsf{acc}}^{(t-1,2)}, \mathbb{U}_{t-1,2}, \overline{T}_{t-1,2})$, using the previous-step secondary fresh instance and its cross-term as input. The curve operations involved are over EC_2, and are (mostly) native over $\mathbb{F}_1$.
2. **Secondary circuit F'_2 over $\mathbb{F}_2$**: a small, fixed-size R1CS circuit that, on input $\boldsymbol{x}_{t,2}$, computes $(\overline{E}, \overline{W})$ via (10)–(11) and returns them as part of its public output. Since $\mathbb{G}$ has base field $\mathbb{F}_2$, point addition and doubling (used in the double-and-add procedure for scalar multiplication) need only $\mathbb{F}_2$-arithmetic, which is precisely the native field of F'_2—the group operations are therefore *native*. A single F'_2 invocation costs a few hundred $\mathbb{F}_2$-constraints, and crucially, $|F'_2|$ is independent of both $|F|$ and t.

Note the indexing above: F'_1 at step t absorbs the secondary fresh instance $\mathbb{U}_{t-1,2}$ produced at step $t-1$, not the one produced at the same step. This is exactly analogous to how the original F' at step t absorbs the *previous* primary fresh instance $\mathbb{U}_t$ produced at step $t-1$: in both cases the fresh instance and its cross-term are carried as input into the next invocation of the augmented circuit. Consequently, the CycleFold proof must carry the most recent secondary fresh instance $\mathbb{U}_{t,2}$ along with its cross-term $\overline{T}_{t,2}$—just as a standard NOVA proof carries the most recent primary $\mathbb{U}_t, \mathbb{W}_t$.

The secondary public input $\boldsymbol{x}_{t,2}$. Concretely,

$$\boldsymbol{x}_{t,2} := \big(\underbrace{r}_{\in \mathbb{F}_1},\ \underbrace{\overline{E}_1,\ \overline{E}_2,\ \overline{T},\ \overline{W}_1,\ \overline{W}_2}_{\in \mathsf{EC}_1(\mathbb{F}_2)}\big),$$

a tuple whose point components are native $\mathbb{F}_2$-elements (two coordinates each), and whose scalar component r is bit-decomposed so that F'_2 can perform scalar multiplications by r via a standard bit-by-bit double-and-add routine. Every entry is produced inside F'_1 during steps (b)–(c): r is the primary Fiat–Shamir challenge, and $\overline{E}_1, \overline{E}_2, \overline{T}, \overline{W}_1, \overline{W}_2$ are extracted from $\mathbb{U}_{\mathsf{acc}}^{(t-1,1)}, \mathbb{U}_{t,1}$ together with the primary cross-term. Running F'_2 on $\boldsymbol{x}_{t,2}$ then returns the folded commitments $(\overline{E}, \overline{W}) \in \mathsf{EC}_1(\mathbb{F}_2)^2$ as public outputs. Importantly, these are bound into $\mathbb{U}_{t,2}.\boldsymbol{x}$ (via $\mathsf{trace}_{\mathsf{RxR1}}$), so that, once $\mathbb{U}_{t,2}$ is folded into the secondary accumulator (at step $t+1$, inside F'_1), the secondary accumulator attests to the correctness of $(\overline{E}, \overline{W})$.

Figure 1 shows the flow between the two (primary and secondary)circuits over two consecutive IVC steps. Formally, the two circuits are described as follows.

[24] Writing $p_i := |\mathbb{F}_i|$ and letting $\mathsf{EC}_i(\mathbb{F}_j)$ denote the finite abelian group of $\mathbb{F}_j$-rational points on EC_i (solutions $(x, y) \in \mathbb{F}_j \times \mathbb{F}_j$ to its Weierstrass equation, plus the point at infinity), a cycle requires $|\mathsf{EC}_1(\mathbb{F}_2)| = p_1$ and $|\mathsf{EC}_2(\mathbb{F}_1)| = p_2$: the group order of each curve—which, for a prime-order curve, is its scalar-field size—coincides with the prime defining the other curve's base field. This is a non-trivial Diophantine condition on (p_1, p_2) and is what makes curve cycles rare: only a handful are known at cryptographically useful sizes (e.g., Pasta, BN254/Grumpkin, secp256k1/secq256k1).

Description of $F'_1 := \mathsf{CycleFold}_1(F)$ (primary circuit, over $\mathbb{F}_1$)

Ingredients: Two hash functions $\mathcal{H}_{\mathsf{io}}, \mathcal{H}_{\mathsf{fs}}$ (over $\mathbb{F}_1$).

Input:

- pp: the IVC public parameters.
- $t \in \{0, 1, 2, \ldots\}$: the step counter.
- $\boldsymbol{\psi}_0 \in \mathbb{F}_1^d$: the initial IVC state (genesis).
- $\boldsymbol{\psi}_t \in \mathbb{F}_1^d$: the IVC state at step-t.
- $\mathbb{U}_{\mathsf{acc}}^{(t-1,1)}$: the running primary accumulator (folded up to step $t-1$).
- $\mathbb{U}_{t,1}$: the primary fresh instance from the previous step.
- $\overline{T}_{t,1} \in \mathbb{G}$: the primary cross-term commitment.
- aux_t: auxiliary input for F, if any.
- r_t, r_{t+1}: randomnesses for the t-th and $(t+1)$-th steps.
- *Secondary-thread input:* $\mathbb{U}_{\mathsf{acc}}^{(t-1,2)}, \mathbb{U}_{t-1,2}, \overline{T}_{t-1,2}$—the previous-step secondary accumulator, fresh instance, and cross-term.

Description of $F'_1(\cdot) \to \mathsf{out}_{F'}$:

- Base case (If $t = 0$):
  1. Compute the first step: $\boldsymbol{\psi}_1 := F(\boldsymbol{\psi}_0, \mathsf{aux}_0)$.
  2. Initialize the secondary accumulator: $\mathbb{U}_{\mathsf{acc}}^{(0,2)} := \mathbb{U}_{\perp,2}$.
  3. Set the output: $\mathsf{out}_{F'} := \mathcal{H}_{\mathsf{io}}(\mathsf{pp}, 1, \boldsymbol{\psi}_0, \boldsymbol{\psi}_1, \mathbb{U}_{\perp,1}, \mathbb{U}_{\mathsf{acc}}^{(0,2)}, r_1)$.
- Recursive case (Else $t \geq 1$):
  1. Check that the public part of the current primary fresh instance was correctly formed:
  $$\mathsf{dec\_pub}_1 := \big(\mathbb{U}_{t,1}.\boldsymbol{x} \stackrel{?}{=} \mathcal{H}_{\mathsf{io}}(\mathsf{pp}, t, \boldsymbol{\psi}_0, \boldsymbol{\psi}_t, \mathbb{U}_{\mathsf{acc}}^{(t-1,1)}, \mathbb{U}_{\mathsf{acc}}^{(t-1,2)}, r_t)\big).$$
  2. Check that $\mathbb{U}_{t,1}$ is a standard (non-relaxed) instance:
  $$\mathsf{dec\_fresh}_1 := \big((\mathbb{U}_{t,1}.\overline{E}, \mathbb{U}_{t,1}.u) \stackrel{?}{=} (\mathbb{U}_{\perp,1}.\overline{E}, 1)\big).$$
  3. *Primary-thread (a)–(c) fold.* Compute $r := \mathcal{H}_{\mathsf{fs}}(\mathbb{U}_{\mathsf{acc}}^{(t-1,1)}, \mathbb{U}_{t,1}, \overline{T}_{t,1})$, and then $\boldsymbol{x}' := \boldsymbol{x}_1 + r \cdot \boldsymbol{x}_2$ and $u' := u_1 + r \cdot u_2$, where $(\boldsymbol{x}_1, u_1)$ and $(\boldsymbol{x}_2, u_2)$ are parsed from $\mathbb{U}_{\mathsf{acc}}^{(t-1,1)}$ and $\mathbb{U}_{t,1}$.
  4. *Assemble the secondary input and run trace.* Set $\boldsymbol{x}_{t,2} := (r, \overline{E}_1, \overline{E}_2, \overline{T}_{t,1}, \overline{W}_1, \overline{W}_2)$, where $(\overline{E}_i, \overline{W}_i)$ are extracted from $\{\mathbb{U}_{\mathsf{acc}}^{(t-1,1)}, \mathbb{U}_{t,1}\}$. Then read $(\overline{E}, \overline{W})$ from $\mathbb{U}_{t-1,2}.\boldsymbol{x}$ and assemble the new primary accumulator $\mathbb{U}_{\mathsf{acc}}^{(t,1)} := (\boldsymbol{x}', u', \overline{E}, \overline{W})$.
  5. *Update the secondary accumulator natively.* Compute
  $$\mathbb{U}_{\mathsf{acc}}^{(t,2)} := \mathsf{NIFS.Verify}\big(\mathbb{U}_{\mathsf{acc}}^{(t-1,2)}, \mathbb{U}_{t-1,2}, \overline{T}_{t-1,2}\big).$$
  (Recall that the EC_2-scalar multiplications inside $\mathsf{NIFS.Verify}$ are native over $\mathbb{F}_1$.)
  6. Execute one step: $\boldsymbol{\psi}_{t+1} := F(\boldsymbol{\psi}_t, \mathsf{aux}_t)$.
  7. Set the output:
  $$\mathsf{out}_{F'} := \begin{cases} \mathcal{H}_{\mathsf{io}}\big(\mathsf{pp}, t+1, \boldsymbol{\psi}_0, \boldsymbol{\psi}_{t+1}, \mathbb{U}_{\mathsf{acc}}^{(t,1)}, \mathbb{U}_{\mathsf{acc}}^{(t,2)}, r_{t+1}\big) & \text{if } (\mathsf{dec\_pub}_1 \wedge \mathsf{dec\_fresh}_1) = 1 \\ \perp & \text{otherwise.} \end{cases}$$

Output: $\mathsf{out}_{F'}$.

Description of $F'_2 := \mathsf{CycleFold}_2$ (secondary circuit, over $\mathbb{F}_2$)

Input:

- $r \in \mathbb{F}_1$ (supplied bit-decomposed).
- $\overline{E}_1, \overline{E}_2, \overline{T}, \overline{W}_1, \overline{W}_2 \in \mathsf{EC}_1(\mathbb{F}_2)$.

Description of $F'_2(r, \overline{E}_1, \overline{E}_2, \overline{T}, \overline{W}_1, \overline{W}_2) \to (\overline{E}, \overline{W})$:

1. Compute $\overline{E} := \overline{E}_1 \cdot \overline{T}^r \cdot \overline{E}_2^{r^2}$ via bit-by-bit double-and-add; all EC_1-group operations are native over $\mathbb{F}_2$.
2. Similarly compute $\overline{W} := \overline{W}_1 \cdot \overline{W}_2^r$.

Output: $(\overline{E}, \overline{W}) \in \mathsf{EC}_1(\mathbb{F}_2)^2$, returned as part of the public output, so that they are bound into $\mathbb{U}_{t,2}.\boldsymbol{x}$ after $\mathsf{trace}_{\mathsf{RxR1}}$.

Two accumulated instances. The prover now maintains *two* running Relaxed-R1CS accumulated instances: $\mathbb{U}_{\mathsf{acc}}^{(t,1)}$ over $\mathbb{F}_1$ (accumulating fresh instances of F'_1), and $\mathbb{U}_{\mathsf{acc}}^{(t,2)}$ over $\mathbb{F}_2$ (accumulating fresh instances $\mathbb{U}_{t,2}$ of F'_2). At each IVC step $t \geq 1$, the prover performs the following:

1. Runs NIFS.Prove to fold the previous primary fresh instance into $\mathbb{U}_{\mathsf{acc}}^{(t-1,1)}$, producing $\mathbb{U}_{\mathsf{acc}}^{(t,1)}$ along with the primary cross-term $\overline{T}_{t,1}$. The EC_1-commitment updates of this fold are *not* performed in-circuit.
2. Runs NIFS.Prove to fold the *previous-step* secondary fresh instance $\mathbb{U}_{t-1,2}$ into $\mathbb{U}_{\mathsf{acc}}^{(t-1,2)}$, producing $\mathbb{U}_{\mathsf{acc}}^{(t,2)}$. Note that this step takes place outside the circuit; the corresponding in-circuit verification of the *same* fold is performed by F'_1 at step t (next step), using the cross-term $\overline{T}_{t-1,2}$ recorded in π_{t-1}.
3. Runs F'_1 via $\mathsf{trace}_{\mathsf{RxR1}}$ to produce the new primary fresh instance. Inside F'_1, steps (a)–(c) and (e)–(g) run as before; step (d) is replaced by assembling $\boldsymbol{x}_{t,2}$ and reading back $(\overline{E}, \overline{W})$ from $\mathbb{U}_{t-1,2}.\boldsymbol{x}$; and a fresh sub-routine computes $\mathbb{U}_{\mathsf{acc}}^{(t,2)} := \mathsf{NIFS.Verify}(\mathbb{U}_{\mathsf{acc}}^{(t-1,2)}, \mathbb{U}_{t-1,2}, \overline{T}_{t-1,2})$ natively over $\mathbb{F}_1$.[25]
4. Assembles $\boldsymbol{x}_{t,2}$ (using quantities extracted from $\mathbb{U}_{\mathsf{acc}}^{(t-1,1)}, \mathbb{U}_{t,1}$ and the primary cross-term $\overline{T}_{t,1}$), and then runs F'_2 via $\mathsf{trace}_{\mathsf{RxR1}}$ to compute $(\overline{E}, \overline{W})$ natively in $\mathbb{F}_2$. This produces the new secondary fresh instance $\mathbb{U}_{t,2}$ together with its witness $\mathbb{W}_{t,2}$ and the next-step cross-term $\overline{T}_{t,2}$ (the cross-term precomputed here is the one that NIFS.Prove will use at step $t+1$ to fold $\mathbb{U}_{t,2}$).

Circuit-size reduction. The non-native scalar multiplications that dominated F' are replaced by a native secondary folding instance inside F'_1 along with a fixed-size F'_2:

Component	**Without** CycleFold	**With** CycleFold
Non-native $\mathbb{G}$-ops in F' (step (d))	$\approx$ 10,000–20,000	0
Native NIFS.Verify inside F'_1	—	$\approx$ 500–1,000
Secondary circuit F'_2 over $\mathbb{F}_2$	—	$\approx$ 3,000–5,000 (fixed)
Field operations (a)–(c), (e)–(g)	$\approx$ 5,000–10,000	$\approx$ 5,000–10,000

Putting it together, the per-step overhead of the augmented circuit drops significantly.

[25] This is the asymmetry that CycleFold exploits: EC_2-scalar multiplications cost $\approx$ 500–1,000 $\mathbb{F}_1$-constraints inside F'_1, whereas the EC_1-scalar multiplications that F' previously carried cost $\approx$ 10,000–20,000 $\mathbb{F}_1$-constraints.

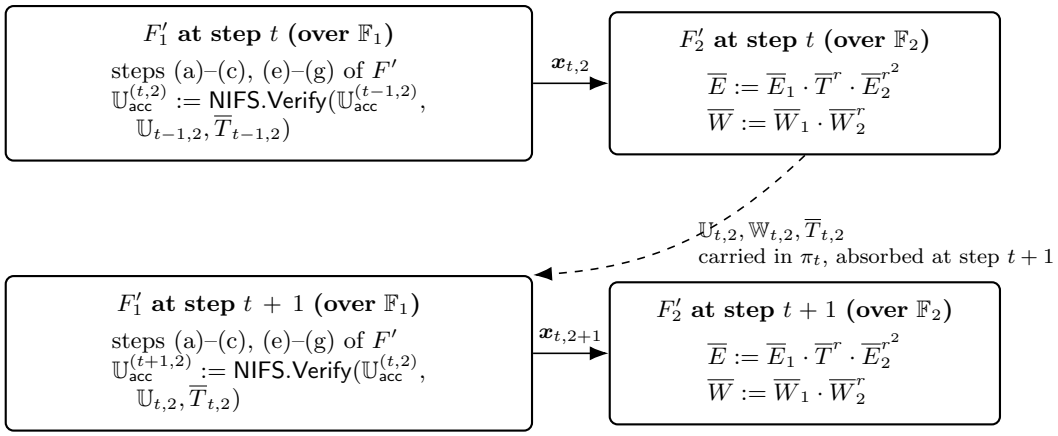


Figure 1: Data flow of the CycleFold split across two consecutive IVC steps. At step t, the primary circuit F'_1 sends $\boldsymbol{x}_{t,2}$ to the secondary circuit F'_2, which computes $(\overline{E}, \overline{W})$ natively in $\mathbb{F}_2$ and returns the secondary fresh instance $\mathbb{U}_{t,2}$. Similar to standard NOVA's handling of the primary instance, this fresh instance is not "absorbed" at step t; instead, it is carried inside π_t together with the cross-term $\overline{T}_{t,2}$, and is "absorbed" at step $t+1$, when F'_1 executes NIFS.Verify natively (recall that the EC_2-scalar multiplications inside NIFS.Verify are native over $\mathbb{F}_1$).

Modified IVC: full construction

We now spell out the CycleFold-modified IVC.Prove and IVC.Verify in full, diff-highlighted against the construction of Section 4.6. Additions and modifications are shown in violet; removed lines are shown ~~struck through~~ for reference.

NOVA IVC with CycleFold: Construction

- $\mathsf{IVC.Setup}(F) \to \mathsf{pp}$:
  - ~~$F' := \mathsf{Augment}(F)$~~ $F'_1 := \mathsf{CycleFold}_1(F)$ over $\mathbb{F}_1$; and $F'_2 := \mathsf{CycleFold}_2(F)$ over $\mathbb{F}_2$.
  - $(A'_1, B'_1, C'_1) := \mathsf{R1CS}(F'_1)$
  - $(A'_2, B'_2, C'_2) := \mathsf{R1CS}(F'_2)$
  - $\mathsf{pp} := \big(g_1, h_{1,1}, \ldots, h_{m_1,1};\ g_2, h_{1,2}, \ldots, h_{m_2,2};\ A'_1, B'_1, C'_1;\ A'_2, B'_2, C'_2\big)$
- $\mathsf{IVC.Prove}(\mathsf{pp}, t, \boldsymbol{\psi}_0, \boldsymbol{\psi}_t, \pi_{t-1}) \to \pi_t$:
  - If $t = 0$:

    (Run F'_1 via trace to produce the first primary fresh instance--witness pair. The secondary thread is empty at the base case.)
    - $r_1 \leftarrow_{\$} \mathbb{F}_1$
    - $(\mathbb{U}_{1,1}, \mathbb{W}_{1,1}) \leftarrow \mathsf{trace}_{\mathsf{RxR1}}\big(F'_1,\ (\mathsf{pp}, \mathbb{U}_{\perp,1}, \mathbb{U}_{\perp,1}, (0, \boldsymbol{\psi}_0, \boldsymbol{\psi}_0), \mathsf{aux}_0, [0], (0, r_1)); \mathbb{U}_{\perp,2}, \mathbb{U}_{\perp,2}, [0]_2\big)$

      (F'_1 takes the base case branch.)
    - $\mathbb{U}_{0,2} := \mathbb{U}_{\perp,2}, \quad \mathbb{W}_{0,2} := (\mathbf{0}, \mathbf{0}, 0, 0), \quad \overline{T}_{0,2} := [0]_2$

      (Initialize both accumulators with trivial instances.)
    - $\mathbb{U}_{\mathsf{acc}}^{(0,1)} := \mathbb{U}_{\perp,1}, \quad \mathbb{W}_{\mathsf{acc}}{}^{(0,1)} := (\mathbf{0}, \mathbf{0}, 0, 0)$
    - $\mathbb{U}_{\mathsf{acc}}^{(0,2)} := \mathbb{U}_{\perp,2}, \quad \mathbb{W}_{\mathsf{acc}}{}^{(0,2)} := (\mathbf{0}, \mathbf{0}, 0, 0)$

      (Output:)
    - $\pi_0 := \big(\mathbb{U}_{\mathsf{acc}}^{(0,1)}, \mathbb{W}_{\mathsf{acc}}{}^{(0,1)}, \mathbb{U}_{1,1}, \mathbb{W}_{1,1};\ \mathbb{U}_{\mathsf{acc}}^{(0,2)}, \mathbb{W}_{\mathsf{acc}}{}^{(0,2)}, \mathbb{U}_{0,2}, \mathbb{W}_{0,2}, \overline{T}_{0,2};\ r_1\big)$
  - Else if $t \geq 1$:

* $\left(\mathbb{U}_{\mathsf{acc}}^{(t-1,1)}, \mathbb{W}_{\mathsf{acc}}{}^{(t-1,1)}, \mathbb{U}_{t,1}, \mathbb{W}_{t,1}, \mathbb{U}_{\mathsf{acc}}^{(t-1,2)}, \mathbb{W}_{\mathsf{acc}}{}^{(t-1,2)}, \mathbb{U}_{t-1,2}, \mathbb{W}_{t-1,2}, \overline{T}_{t-1,2}, r_t\right)$ $:= \pi_{t-1}$

  (Fold on the primary thread.)
* $(\mathbb{U}_{\mathsf{acc}}^{(t,1)}, \mathbb{W}_{\mathsf{acc}}{}^{(t,1)}, \overline{T}_{t,1}) := \mathsf{NIFS.Prove}\left(\mathbb{U}_{\mathsf{acc}}^{(t-1,1)}, \mathbb{W}_{\mathsf{acc}}{}^{(t-1,1)}, \mathbb{U}_{t,1}, \mathbb{W}_{t,1}\right)$

  (Fold on the secondary thread)
* $(\mathbb{U}_{\mathsf{acc}}^{(t,2)}, \mathbb{W}_{\mathsf{acc}}{}^{(t,2)}) := \mathsf{NIFS.Prove}^{\star}\left(\mathbb{U}_{\mathsf{acc}}^{(t-1,2)}, \mathbb{W}_{\mathsf{acc}}{}^{(t-1,2)}, \mathbb{U}_{t-1,2}, \mathbb{W}_{t-1,2}, \overline{T}_{t-1,2}\right)$

  (Note: Here $\mathsf{NIFS.Prove}^{\star}$ denotes NIFS.Prove with the cross-term supplied rather than recomputed)

  (Assemble the secondary public input and run the trace)
* $r := \mathcal{H}_{\mathsf{fs}}(\mathbb{U}_{\mathsf{acc}}^{(t-1,1)}, \mathbb{U}_{t,1}, \overline{T}_{t,1}) \in \mathbb{F}_1$
* $\overline{E}_1 := \mathbb{U}_{\mathsf{acc}}^{(t-1,1)}.\overline{E},\ \overline{W}_1 := \mathbb{U}_{\mathsf{acc}}^{(t-1,1)}.\overline{W},\ \overline{E}_2 := \mathbb{U}_{t,1}.\overline{E},\ \overline{W}_2 := \mathbb{U}_{t,1}.\overline{W}.$
* $\boldsymbol{x}_{t,2} := \left(r,\ \overline{E}_1,\ \overline{E}_2,\ \overline{T}_{t,1},\ \overline{W}_1,\ \overline{W}_2\right).$
* $(\mathbb{U}_{t,2}, \mathbb{W}_{t,2}, \overline{T}_{t,2}) \leftarrow \mathsf{trace}_{\mathsf{RxR1}}\left(F'_2, \boldsymbol{x}_{t,2};\ \mathbb{U}_{\mathsf{acc}}^{(t,2)}\right).$

  (Note: the cross-term $\overline{T}_{t,2}$, used for folding $\mathbb{U}_{t,2}$ into $\mathbb{U}_{\mathsf{acc}}^{(t,2)}$ at the next step, is precomputed alongside the trace.)
* $r_{t+1} \leftarrow_{\$} \mathbb{F}_1$
* $(\mathbb{U}_{t+1,1}, \mathbb{W}_{t+1,1}) \leftarrow \mathsf{trace}_{\mathsf{RxR1}}\left(F'_1,\ (\mathsf{pp}, \mathbb{U}_{\mathsf{acc}}^{(t-1,1)}, \mathbb{U}_{t,1}, (t, \boldsymbol{\psi}_0, \boldsymbol{\psi}_t), \mathsf{aux}_t, \overline{T}_{t,1}, (r_t, r_{t+1}));\right.$ $\left.\mathbb{U}_{\mathsf{acc}}^{(t-1,2)}, \mathbb{U}_{t-1,2}, \overline{T}_{t-1,2}\right)$

– $\pi_t := \left(\mathbb{U}_{\mathsf{acc}}^{(t,1)}, \mathbb{W}_{\mathsf{acc}}{}^{(t,1)}, \mathbb{U}_{t+1,1}, \mathbb{W}_{t+1,1};\ \mathbb{U}_{\mathsf{acc}}^{(t,2)}, \mathbb{W}_{\mathsf{acc}}{}^{(t,2)}, \mathbb{U}_{t,2}, \mathbb{W}_{t,2}, \overline{T}_{t,2};\ r_{t+1}\right)$

- $\mathsf{IVC.Verify}(\mathsf{pp}, t, \boldsymbol{\psi}_0, \boldsymbol{\psi}_{t+1}, \pi_t) \to \mathsf{dec\_bit}$:

  – $\left(\mathbb{U}_{\mathsf{acc}}^{(t,1)}, \mathbb{W}_{\mathsf{acc}}{}^{(t,1)}, \mathbb{U}_{t+1,1}, \mathbb{W}_{t+1,1}, \mathbb{U}_{\mathsf{acc}}^{(t,2)}, \mathbb{W}_{\mathsf{acc}}{}^{(t,2)}, \mathbb{U}_{t,2}, \mathbb{W}_{t,2}, \overline{T}_{t,2}, r_{t+1}\right) := \pi_t$

  – $\left(g_1, h_{1,1}, \ldots, h_{m_1,1};\ g_2, h_{1,2}, \ldots, h_{m_2,2};\ A'_1, B'_1, C'_1;\ A'_2, B'_2, C'_2\right) := \mathsf{pp}$

  (IO-hash consistency on the primary fresh instance)

  – $\mathsf{dec\_io} := \left(\mathbb{U}_{t+1,1}.\boldsymbol{x} \stackrel{?}{=} \mathcal{H}_{\mathsf{io}}(\mathsf{pp}, t+1, \boldsymbol{\psi}_0, \boldsymbol{\psi}_{t+1}, \mathbb{U}_{\mathsf{acc}}^{(t,1)}, \mathbb{U}_{\mathsf{acc}}^{(t,2)}, r_{t+1})\right)$

  (Primary Relaxed-R1CS check on the primary accumulated instance)

  – $(\boldsymbol{x}_{\mathsf{acc},1}, u_{\mathsf{acc},1}, \overline{E}_{\mathsf{acc},1}, \overline{W}_{\mathsf{acc},1}) := \mathbb{U}_{\mathsf{acc}}^{(t,1)}$

  – $(\boldsymbol{w}_{\mathsf{acc},1}, \boldsymbol{E}_{\mathsf{acc},1}, r_{W,\mathsf{acc},1}, r_{E,\mathsf{acc},1}) := \mathbb{W}_{\mathsf{acc}}{}^{(t,1)}$

  – $\boldsymbol{z}_{\mathsf{acc},1} := (u_{\mathsf{acc},1}, \boldsymbol{x}_{\mathsf{acc},1}, \boldsymbol{w}_{\mathsf{acc},1})$

  – $\mathsf{dec\_RlxR1CS}_1 := \left((A'_1 \boldsymbol{z}_{\mathsf{acc},1}) \circ (B'_1 \boldsymbol{z}_{\mathsf{acc},1}) \stackrel{?}{=} u_{\mathsf{acc},1} \cdot C'_1 \boldsymbol{z}_{\mathsf{acc},1} + \boldsymbol{E}_{\mathsf{acc},1}\right)$ $\wedge\left(\mathsf{Com}(\boldsymbol{E}_{\mathsf{acc},1}; r_{E,\mathsf{acc},1}) \stackrel{?}{=} \overline{E}_{\mathsf{acc},1}\right) \wedge \left(\mathsf{Com}(\boldsymbol{w}_{\mathsf{acc},1}; r_{W,\mathsf{acc},1}) \stackrel{?}{=} \overline{W}_{\mathsf{acc},1}\right)$

  (Standard R1CS check on the primary fresh instance)

  – $\mathsf{dec\_R1CS}_1 := \left((A'_1 \boldsymbol{z}_1) \circ (B'_1 \boldsymbol{z}_1) \stackrel{?}{=} C'_1 \boldsymbol{z}_1\right) \wedge (u_1 \stackrel{?}{=} 1) \wedge (\boldsymbol{E}_1 \stackrel{?}{=} \boldsymbol{0})$ $\wedge(r_{w,1} \stackrel{?}{=} r_{E,1} \stackrel{?}{=} 0) \wedge (\overline{E}_1 \stackrel{?}{=} \overline{W}_1 \stackrel{?}{=} \mathsf{Com}(\boldsymbol{0}; 0))$

  (New: Secondary Relaxed-R1CS check on the secondary accumulator.)

  – $(\boldsymbol{x}_{\mathsf{acc},2}, u_{\mathsf{acc},2}, \overline{E}_{\mathsf{acc},2}, \overline{W}_{\mathsf{acc},2}) := \mathbb{U}_{\mathsf{acc}}^{(t,2)}$

  – $(\boldsymbol{w}_{\mathsf{acc},2}, \boldsymbol{E}_{\mathsf{acc},2}, r_{W,\mathsf{acc},2}, r_{E,\mathsf{acc},2}) := \mathbb{W}_{\mathsf{acc}}{}^{(t,2)}$

  – $\boldsymbol{z}_{\mathsf{acc},2} := (u_{\mathsf{acc},2}, \boldsymbol{x}_{\mathsf{acc},2}, \boldsymbol{w}_{\mathsf{acc},2})$

  – $\mathsf{dec\_RlxR1CS}_2 := \left((A'_2 \boldsymbol{z}_{\mathsf{acc},2}) \circ (B'_2 \boldsymbol{z}_{\mathsf{acc},2}) \stackrel{?}{=} u_{\mathsf{acc},2} \cdot C'_2 \boldsymbol{z}_{\mathsf{acc},2} + \boldsymbol{E}_{\mathsf{acc},2}\right)$ $\wedge\left(\mathsf{Com}(\boldsymbol{E}_{\mathsf{acc},2}; r_{E,\mathsf{acc},2}) \stackrel{?}{=} \overline{E}_{\mathsf{acc},2}\right) \wedge \left(\mathsf{Com}(\boldsymbol{w}_{\mathsf{acc},2}; r_{W,\mathsf{acc},2}) \stackrel{?}{=} \overline{W}_{\mathsf{acc},2}\right)$

(Standard R1CS check on the secondary fresh instance)

- $\mathsf{dec\_R1CS}_2 := \big((A'_2 \boldsymbol{z}_{t,2}) \circ (B'_2 \boldsymbol{z}_{t,2}) \stackrel{?}{=} C'_2 \boldsymbol{z}_{t,2}\big) \wedge (\mathbb{U}_{t,2}.u \stackrel{?}{=} 1) \wedge (\mathbb{U}_{t,2}.\boldsymbol{E} \stackrel{?}{=} \mathbf{0})$
  $\wedge (r_{w,2} \stackrel{?}{=} r_{E,2} \stackrel{?}{=} 0) \wedge (\mathbb{U}_{t,2}.\overline{E} \stackrel{?}{=} \mathbb{U}_{t,2}.\overline{W} \stackrel{?}{=} \mathsf{Com}(\mathbf{0}; 0))$
  (Note: not yet folded into $\mathbb{U}_{\mathsf{acc}}^{(t,2)}$ and waiting to be absorbed by F'_1 at step $t+1$).)
  (Final decision bit.)
- $\mathsf{dec\_bit} := \mathsf{dec\_io} \wedge \mathsf{dec\_RlxR1CS}_1 \wedge \mathsf{dec\_R1CS}_1 \wedge \mathsf{dec\_RlxR1CS}_2 \wedge \mathsf{dec\_R1CS}_2$

Modified compressed Decider. The compressed decider circuit C_{dec} from Section 4.6.3 (a Groth16-circuit defined over $\mathbb{F}_1$) is now updated to "absorb" the two new checks of IVC.Verify:

1. dec_io: hash consistency of $\mathbb{U}_{t+1,1}.\boldsymbol{x}$ (unchanged).
2. $\mathsf{dec\_RlxR1CS}_1$: primary Relaxed-R1CS relation for $\mathbb{U}_{\mathsf{acc}}^{(t,1)}$ w.r.t. (A'_1, B'_1, C'_1) (unchanged except the matrices).
3. $\mathsf{dec\_R1CS}_1$: standard R1CS for the primary fresh instance $\mathbb{U}_{t+1,1}$ (unchanged).
4. $\mathsf{dec\_RlxR1CS}_2$: secondary Relaxed-R1CS relation for $\mathbb{U}_{\mathsf{acc}}^{(t,2)}$ w.r.t. (A'_2, B'_2, C'_2).
5. $\mathsf{dec\_R1CS}_2$: standard R1CS for the secondary fresh instance $\mathbb{U}_{t,2}$.

All five checks are encoded as R1CS-constraints inside C_{dec}. Since C_{dec} is defined over $\mathbb{F}_1$, checks 1–3 are native, while checks 4 and 5 involve the secondary Relaxed-R1CS/R1CS-matrices over $\mathbb{F}_2$, and hence require non-native emulation inside C_{dec}. Now, since F'_2 is small and *fixed* ($\approx$ 3,000–5,000 constraints), encoding these non-natively adds only a modest cost to the decider proving time—far less than what encoding the full F' non-natively would have incurred.

We note that the external interface remains unchanged: $\mathsf{Decider.Verify}(vk_{\mathsf{G}}, t, \boldsymbol{\psi}_0, \boldsymbol{\psi}_{t+1}, \pi_t^c)$ still performs a single Groth16 verification (3 pairings), and the proof π_t^c remains $\approx$ 192 bytes. The only change is that the decider's proving time increases slightly (due to $\mathsf{dec\_RlxR1CS}_2$ and $\mathsf{dec\_R1CS}_2$ inside C_{dec}), while the primary IVC per-step proving time decreases substantially.

We provide the full description of the updated decider next:

Decider with CycleFold: Ingredients, Parameters, Notations, and Setting

- NOVA IVC scheme (with CycleFold) with public parameters:
  $\mathsf{pp} = \big(g_1, h_{1,1}, \ldots, h_{m_1,1};\ g_2, h_{1,2}, \ldots, h_{m_2,2};\ A'_1, B'_1, C'_1;\ A'_2, B'_2, C'_2\big)$.
- Non-interactive folding scheme NIFS = (NIFS.Prove, NIFS.Verify) for committed Relaxed-R1CS (cf. Section 4.5.2), used on both the primary and secondary threads.
- Groth16 proof system (Groth16.Setup, Groth16.Prove, Groth16.Verify) (cf. Section 4.3).
- Decider circuit C_{dec} (defined over $\mathbb{F}_1$) with public input $\big(\mathbb{U}'_{\mathsf{acc}1}, \mathbb{U}'_{\mathsf{acc}2}, \mathbb{U}_{t+1,1}\big)$ and witness $\big(\mathbb{W}_{\mathsf{acc}}{}'_1, \mathbb{W}_{\mathsf{acc}}{}'_2\big)$, where $\mathbb{U}'_{\mathsf{acc}i} = (\boldsymbol{x}'_i, u'_i, \overline{E}'_i, \overline{W}'_i)$ and $\mathbb{W}_{\mathsf{acc}}{}'_i = (\boldsymbol{w}'_i, \boldsymbol{E}'_i, r_{W'_i}, r_{E'_i})$. The circuit enforces:

  (Primary Relaxed-R1CS check for $\mathbb{U}'_{\mathsf{acc}1}$ (native over $\mathbb{F}_1$):)
  - $\mathsf{Com}(\boldsymbol{E}'_1; r_{E'_1}) \stackrel{?}{=} \overline{E}'_1$
  - $\mathsf{Com}(\boldsymbol{w}'_1; r_{W'_1}) \stackrel{?}{=} \overline{W}'_1$
  - $(A'_1 \boldsymbol{z}'_1) \circ (B'_1 \boldsymbol{z}'_1) \stackrel{?}{=} u'_1 \cdot (C'_1 \boldsymbol{z}'_1) + \boldsymbol{E}'_1$, where $\boldsymbol{z}'_1 = (u'_1, \boldsymbol{x}'_1, \boldsymbol{w}'_1)$

  (Freshness check on the primary fresh instance (native over $\mathbb{F}_1$):)
  - $(\mathbb{U}_{t+1,1}.\overline{E} \stackrel{?}{=} \mathbb{U}_{\perp,1}.\overline{E}) \wedge (\mathbb{U}_{t+1,1}.u \stackrel{?}{=} 1)$

  (Secondary Relaxed-R1CS check for $\mathbb{U}'_{\mathsf{acc}2}$ (non-native over $\mathbb{F}_1$):)
  - $\mathsf{Com}(\boldsymbol{E}'_2; r_{E'_2}) \stackrel{?}{=} \overline{E}'_2$

- $\mathsf{Com}(\boldsymbol{w}'_2; r_{W'_2}) \stackrel{?}{=} \overline{W}'_2$
- $(A'_2 \boldsymbol{z}'_2) \circ (B'_2 \boldsymbol{z}'_2) \stackrel{?}{=} u'_2 \cdot (C'_2 \boldsymbol{z}'_2) + \boldsymbol{E}'_2, \quad \text{where } \boldsymbol{z}'_2 = (u'_2, \boldsymbol{x}'_2, \boldsymbol{w}'_2)$

Decider with CycleFold: Construction

- $\mathsf{Decider.Setup}(\mathsf{pp}) \to (pk_{\mathsf{G}}, vk_{\mathsf{G}})$:
  - $(A, B, C) := \mathsf{R1CS}(C_{\mathsf{dec}})$
  - $(pk_{\mathsf{G}}, vk_{\mathsf{G}}) := \mathsf{Groth16.Setup}(A, B, C)$
- $\mathsf{Decider.Prove}(pk_{\mathsf{G}}, (t, \boldsymbol{\psi}_0, \boldsymbol{\psi}_{t+1}, \pi_t)) \to \pi_{\mathsf{dec}}$:
  - Parse π_t as $\big(\mathbb{U}_{\mathsf{acc}}^{(t,1)}, \mathbb{W}_{\mathsf{acc}}{}^{(t,1)}, \mathbb{U}_{t+1,1}, \mathbb{W}_{t+1,1}; \mathbb{U}_{\mathsf{acc}}^{(t,2)}, \mathbb{W}_{\mathsf{acc}}{}^{(t,2)}, \mathbb{U}_{t,2}, \mathbb{W}_{t,2}, \overline{T}_{t,2}; r_{t+1}\big)$.
    (Fold the primary accumulator with the primary fresh instance into a single committed Relaxed-R1CS pair; $\overline{T}_1$ is the primary cross-term commitment.)
  - $(\mathbb{U}'_{\mathsf{acc}\,1}, \mathbb{W}_{\mathsf{acc}}{}'_1, \overline{T}_1) := \mathsf{NIFS.Prove}\big(\mathsf{pp}, (\mathbb{U}_{\mathsf{acc}}^{(t,1)}, \mathbb{W}_{\mathsf{acc}}{}^{(t,1)}), (\mathbb{U}_{t+1,1}, \mathbb{W}_{t+1,1})\big)$
    (Fold the secondary accumulator with the secondary fresh instance using the cross-term $\overline{T}_{t,2}$ carried in π_t.)
  - $(\mathbb{U}'_{\mathsf{acc}\,2}, \mathbb{W}_{\mathsf{acc}}{}'_2) := \mathsf{NIFS.Prove}^{\star}\big(\mathsf{pp}, (\mathbb{U}_{\mathsf{acc}}^{(t,2)}, \mathbb{W}_{\mathsf{acc}}{}^{(t,2)}), (\mathbb{U}_{t,2}, \mathbb{W}_{t,2}), \overline{T}_{t,2}\big)$
    (Encode the public input and witness for C_{dec} and produce the Groth16 proof.)
  - $(\boldsymbol{x}, \boldsymbol{w}) := \mathsf{trace}_{\mathsf{R1}}\big(C_{\mathsf{dec}}, (\mathbb{U}'_{\mathsf{acc}\,1}, \mathbb{U}'_{\mathsf{acc}\,2}, \mathbb{U}_{t+1,1}), (\mathbb{W}_{\mathsf{acc}}{}'_1, \mathbb{W}_{\mathsf{acc}}{}'_2)\big)$
  - $\pi_{C_{\mathsf{dec}}} := \mathsf{Groth16.Prove}(pk_{\mathsf{G}}, \boldsymbol{x}, \boldsymbol{w})$
  - Output $\pi_{\mathsf{dec}} := \big(\mathbb{U}_{\mathsf{acc}}^{(t,1)}, \mathbb{U}_{t+1,1}, \mathbb{U}_{\mathsf{acc}}^{(t,2)}, \mathbb{U}_{t,2}, \overline{T}_{t,2}, r_{t+1}, \overline{T}_1, \pi_{C_{\mathsf{dec}}}\big)$.
- $\mathsf{Decider.Verify}(vk_{\mathsf{G}}, (t, \boldsymbol{\psi}_0, \boldsymbol{\psi}_{t+1}, \pi_{\mathsf{dec}})) \to \mathsf{dec\_bit}$:
  - Parse π_{dec} as $\big(\mathbb{U}_{\mathsf{acc}}^{(t,1)}, \mathbb{U}_{t+1,1}, \mathbb{U}_{\mathsf{acc}}^{(t,2)}, \mathbb{U}_{t,2}, \overline{T}_{t,2}, r_{t+1}, \overline{T}_1, \pi_{C_{\mathsf{dec}}}\big)$.
    (Hash binding check (subscript ',1' added on the primary, $\mathbb{U}_{\mathsf{acc}}^{(t,2)}$ now also hashed in):)
  - $\mathsf{dec\_io} := \big(\mathbb{U}_{t+1,1}.\boldsymbol{x} \stackrel{?}{=} \mathcal{H}_{\mathsf{io}}(\mathsf{pp}, t+1, \boldsymbol{\psi}_0, \boldsymbol{\psi}_{t+1}, \mathbb{U}_{\mathsf{acc}}^{(t,1)}, \mathbb{U}_{\mathsf{acc}}^{(t,2)}, r_{t+1})\big)$
    (Freshness checks on the fresh instances:)
  - $\mathsf{dec\_R1CS}_1 := (\mathbb{U}_{t+1,1}.\overline{E} \stackrel{?}{=} \mathbb{U}_{\perp,1}.\overline{E}) \wedge (\mathbb{U}_{t+1,1}.u \stackrel{?}{=} 1)$
  - $\mathsf{dec\_R1CS}_2 := (\mathbb{U}_{t,2}.\overline{E} \stackrel{?}{=} \mathbb{U}_{\perp,2}.\overline{E}) \wedge (\mathbb{U}_{t,2}.u \stackrel{?}{=} 1)$
    (Recompute the folded primary instance using $\overline{T}_1$ from π_{dec}:)
  - $\mathbb{U}'_{\mathsf{acc}\,1} := \mathsf{NIFS.Verify}(\mathsf{pp}, \mathbb{U}_{\mathsf{acc}}^{(t,1)}, \mathbb{U}_{t+1,1}, \overline{T}_1)$
    (Recompute the folded secondary instance using $\overline{T}_{t,2}$ from π_{dec}:)
  - $\mathbb{U}'_{\mathsf{acc}\,2} := \mathsf{NIFS.Verify}(\mathsf{pp}, \mathbb{U}_{\mathsf{acc}}^{(t,2)}, \mathbb{U}_{t,2}, \overline{T}_{t,2})$
    (Public-input encoding of $(\mathbb{U}'_{\mathsf{acc}\,1}, \mathbb{U}'_{\mathsf{acc}\,2}, \mathbb{U}_{t+1,1})$ for C_{dec}, then a single Groth16 verification:)
  - $\boldsymbol{x} := \mathsf{trace}_{\mathsf{R1}}\big(C_{\mathsf{dec}}, (\mathbb{U}'_{\mathsf{acc}\,1}, \mathbb{U}'_{\mathsf{acc}\,2}, \mathbb{U}_{t+1,1})\big)$
  - $\mathsf{dec\_C}_{\mathsf{dec}} := \mathsf{Groth16.Verify}(vk_{\mathsf{G}}, \boldsymbol{x}, \pi_{C_{\mathsf{dec}}})$
  - $\mathsf{dec\_bit} := \mathsf{dec\_io} \wedge \mathsf{dec\_R1CS}_1 \wedge \mathsf{dec\_R1CS}_2 \wedge \mathsf{dec\_C}_{\mathsf{dec}}$

Remark 3 (Curve choice for HieroTSS)**.** Recall that hinTS and the Groth16 decider both rely on BN254 pairings. So, we instantiate the CycleFold pair as $\mathsf{EC}_1 = \text{BN254}$ and $\mathsf{EC}_2 = \text{Grumpkin}$, a 2-cycle widely

supported in existing IVC-implementations (e.g. Sonobe [0xP24], Nova Scotia). Concretely, Grumpkin is defined over $\mathbb{F}_1$ (BN254's scalar field) with group order $|\mathbb{F}_2|$ (BN254's base field prime). The primary Pedersen commitments for the NOVA fold lie in $\mathsf{EC}_1(\mathbb{F}_2) = \text{BN254}$, whose scalar multiplications are native over $\mathbb{F}_2$—precisely the native field of F_2'. The secondary Pedersen commitments lie in $\mathsf{EC}_2(\mathbb{F}_1) = \text{Grumpkin}$, whose scalar multiplications are native over $\mathbb{F}_1$—again, precisely the native field of F_1'. This symmetric 2-cycle realizes the CycleFold delegation and the native secondary fold without incurring any further non-native work.

Reusing BN254 across all three components—hinTS, the Groth16 decider, and the primary Pedersen group—also avoids any cross-curve translation of group elements between the threshold-signing and the key-update parts of HieroTSS, which is essential for keeping the external verifier's check to a single set of BN254-pairing operations.[26]

5 HieroTSS: Putting things together

We now describe the full operational flow of the HieroTSS system, combining hinTS (Section 3), Schnorr multi-signatures (Section 4.1), and the NOVA IVC + Groth16 decider pipeline described earlier.

Address Books An *address book* AB is a ledger-maintained registry that defines the current set of validators authorized to participate in consensus. Concretely, an address book is an ordered list of entries $\mathsf{AB} := [(\mathsf{id}_i, \mathsf{svk}_i, \mathsf{vk}_i, w_i)]_{i\in[n]}$, where id_i is a unique party identifier, svk_i is the party-i's Schnorr verification key (used for key rotation signatures), vk_i is the hinTS verification key (used for threshold signing), and w_i is the weight. The *threshold* $t_{\mathsf{th}} := \lceil \widehat{w}/2 \rceil + 1$ (where $\widehat{w} := \sum_i w_i$) determines the minimum weight required for a valid quorum.

A *participating set* $\mathcal{S} \subseteq [n]$ for a given signing session is the subset of parties from the current address book who are online and willing to sign; a valid signature requires $\sum_{j\in\mathcal{S}} w_j \geqslant t_{\mathsf{th}}$. The address book is *static within an epoch*: all signing sessions during epoch η_t use the same AB_t. When the network decides to update the validator set, a new address book AB_{t+1} is created and the transition from AB_t to AB_{t+1} is authorized by a Schnorr multi-signature from a sufficient quorum of AB_t's validators.

The IVC Step Function The step function F is the computation that gets proved at each IVC step. It takes as input the current IVC state $\boldsymbol{\psi}_t$ and auxiliary (private) input aux_t, and outputs the next state $\boldsymbol{\psi}_{t+1} := F(\boldsymbol{\psi}_t, \mathsf{aux}_t)$. Concretely:

Descripton of IVC step function F

Ingredients: Hash function $\mathcal{H}$.

Inputs:

- IVC **state.** The state encodes the current epoch's identity:
  - $\boldsymbol{\psi}_t := (\mathcal{H}(\mathsf{AB}_t),\ \mathcal{H}(\mathsf{VK}_t))$ — hash of the current address book and hinTS verification key.
- **Auxiliary input.** The auxiliary input aux_t contains the private data needed to verify the transition:
  - AB_t — the current address book (in full, not just its hash).
  - AB_{t+1} — the next address book.
  - σ_t — the Schnorr multi-signature authorizing the rotation.
  - $\mathcal{S}$ — the participating set (quorum) that signed the rotation.
  - $\boldsymbol{b}$ — the signer vector (binary indicator of which parties signed).

[26] Grumpkin is not pairing-friendly, but the NOVA fold uses only group-additions and scalar multiplications, so a pairing on EC_2 is never needed. Pairings are required only on $\mathsf{EC}_1 = \text{BN254}$, for hinTS verification and the final Groth16 check.

Description of $F(\boldsymbol{\psi}_t, \mathsf{aux}_t) \to \boldsymbol{\psi}_{t+1}$

- *Input consistency:* If $\mathcal{H}(\mathsf{AB}_t)$ matches the first component of the input state $\boldsymbol{\psi}_t$, set $\mathsf{dec\_ip} = 1$, else set $\mathsf{dec\_ip} = 0$.
- *Signature verification:* $\mathsf{dec\_sig} := (\mathsf{Mult.Schnorr.Verify}(\{\mathsf{svk}_j\}_{j\in\mathcal{S}}, \mathsf{msg}_t, \sigma_t) = 1)$, where the Schnorr verification keys svk_j are extracted from AB_t and the rotation message is $\mathsf{msg}_t := \mathcal{H}(\mathsf{VK}_t \| \mathsf{AB}_{t+1})$.
- *Threshold:* $\mathsf{dec\_thresh} := \left(\sum_{j\in\mathcal{S}} w_j \geqslant t_{\mathsf{th}}\right)$, where t_{th} is the threshold derived from AB_t.
- Set output

$$\boldsymbol{\psi}_{t+1} := \begin{cases} (\mathcal{H}(\mathsf{AB}_{t+1}),\ \mathcal{H}(\mathsf{VK}_{t+1})) & \text{if } \left(\mathsf{dec\_ip} \wedge \mathsf{dec\_sig} \wedge \mathsf{dec\_thresh}\right) = 1 \\ \bot & \text{otherwise} \end{cases}$$

Note that, the Schnorr multi-signature verification, the hash computations, and the weight check are all encoded as R1CS constraints inside F. The auxiliary input aux_t is never revealed to the IVC verifier—it is consumed by F inside the augmented circuit F'.

5.1 End-to-End Operational Flow

The HieroTSS system operates in *epochs*. Each epoch η_t is associated with an address book AB_t that dnotes the active validator set. Within an epoch, the hinTS threshold signature scheme is used to sign messages (e.g., consensus blocks). When the network decides to rotate the validator set from AB_t to AB_{t+1}, it transitions to a new epoch via a multi-step process: first, the outgoing validators authorize the new address book with a Schnorr multi-signature; then, the NOVA IVC prover folds this authorization into the running accumulator; finally, the Groth16 decider compresses the accumulated proof into a constant-size proof suitable for external verification.

The *key invariant* is that the IVC state $\boldsymbol{\psi}_t = (\mathcal{H}(\mathsf{AB}_t), \mathcal{H}(\mathsf{VK}_t))$ is maintained across epochs, with each transition provably authorized by the previous epoch's quorum. Any external verifier needs to trust only the genesis state $\boldsymbol{\psi}_0$ in order to verify the entire chain of rotations from epoch η_0 to epoch η_t by checking a single Groth16 proof—without seeing any intermediate address books, signatures, or quorum information.

We now describe each phase in detail. In addition to different phases below, there is an initial one-time **system setup** that generates the public parameters for hinTS, the IVC scheme, and the decider. The one-time system setup is done using a **ceremony** following [KMSV21].

HieroTSS: System Setup (Ceremony)

- **Parameters, Ingredients, Notations**:
  - **hinTS:** Parameters AllPar
  - Hash function $\mathcal{H}$
- hinTS **setup:** $\mathsf{SRS} \leftarrow \mathsf{hinTS.Setup}(\mathsf{AllPar})$.

  (The SRS is generated once via a trusted setup ceremony (or MPC) and shared by all parties across all epochs.)
- IVC **setup:**
  - $\mathsf{pp} \leftarrow \mathsf{IVC.Setup}(F)$.
  - $F' := \mathsf{Augment}(F)$
  - $(A', B', C') := \mathsf{R1CS}(F')$
- **Decider setup:** $(pk_{\mathsf{G}}, vk_{\mathsf{G}}) \leftarrow \mathsf{Decider.Setup}(\mathsf{pp})$.

(Constructs the decider circuit C_{dec} (encoding IVC.Verify) and runs Groth16 setup.)

Key generation and registration: Each initial party P_i generates and registers its cryptographic keys:

- *Schnorr key pair:* $(\mathsf{ssk}_i, \mathsf{svk}_i) \leftarrow\!\!\leftarrow \mathsf{Mult.Schnorr.KGen}(i)$. The verification key svk_i is registered (used for key-rotation signatures).
- hinTS *key pair:* $(\mathsf{sk}_i, \mathsf{vk}_i) \leftarrow \mathsf{hinTS.KGen}(\mathsf{AllPar}, i)$. The verification key vk_i is registered (used for threshold signing).

Both key pairs are generated *before* the genesis block. The public keys $(\mathsf{svk}_i, \mathsf{vk}_i)$ become part of the genesis address book. Each party also performs a proof of possession for the Schnorr key (cf. Section 4.1) to prevent rogue-key attacks.

Genesis:

1. **Genesis address book:** The genesis address book is assembled from the registered keys and governance-assigned weights:
$$\mathsf{AB}_0 := [(\mathsf{id}_i, \mathsf{svk}_i, \mathsf{vk}_i, w_i)]_{i \in \mathsf{AB}_0}$$
where w_i is the consensus weight assigned to party P_i by the founding governance process.
2. hinTS **hint generation:** Each party $P_i \in \mathsf{AB}_0$ generates and broadcasts its hint:
   - *Key-independent local setup:* $\mathsf{loc\_st}_i \leftarrow \mathsf{hinTS.PreHintGen}(\mathsf{SRS}, i)$
   - *Hints generation* $\mathsf{hint}_i \leftarrow \mathsf{hinTS.HintGen}(\mathsf{sk}_i, \mathsf{loc\_st}_i)$

   Every party in AB_0 runs $(\mathsf{AK}_0, \mathsf{VK}_0) \leftarrow \mathsf{hinTS.PreProcess}(\mathsf{SRS}, \{\mathsf{hint}_i, \mathsf{vk}_i\}_{i \in [n_0]})$. Recall that, hinTS.PreProcess internally verifies each hint via hinTS.PartVerify and discards invalid ones.
3. **Root-of-trust:** The root-of-trust IVC state is:
$$\psi_0 := (\mathcal{H}(\mathsf{AB}_0),\ \mathcal{H}(\mathsf{VK}_0))$$
   Initialize the IVC proof: $\pi_0 := \perp$. Note that, the genesis block is the root of trust for the entire system. Its contents—in particular $\mathcal{H}(\mathsf{AB}_0)$—are published and assumed authentic by all verifiers. No cryptographic proof is needed for this step; it works as an axiom for all practical purpose.

After genesis, the **system enters epoch** η_0: parties can immediately produce hinTS threshold signatures using $(\mathsf{AK}_0, \mathsf{VK}_0)$, and can initiate address book rotations. The first rotation triggers the transition to epoch η_1.

Epoch η_t $(t \geq 0)$**:** Now we describe the per-epoch operational flow including the address book rotations from the previous epoch. For $t = 0$ the hints from genesis are used and step 1 (Hints generation) below is skipped; for $t \geqslant 1$ new parties generate hints as described in step 1.

1. **Hints generation:** This is executed (only for $t \geqslant 1$; epoch 0 uses the hints from genesis) once per epoch, when the address book is rotated.
   (a) Each *new* party P_i joining AB_t (who was not in AB_{t-1}) generates keys and hints:
   - $(\mathsf{ssk}_i, \mathsf{svk}_i) \leftarrow \mathsf{Schnorr.KGen}(1^\kappa)$; $\quad (\mathsf{sk}_i, \mathsf{vk}_i) \leftarrow \mathsf{hinTS.KGen}(\mathsf{AllPar}, i)$
   - $\mathsf{loc\_st}_i \leftarrow \mathsf{hinTS.PreHintGen}(\mathsf{SRS}, i)$; $\quad \mathsf{hint}_i \leftarrow \mathsf{hinTS.HintGen}(\mathsf{sk}_i, \mathsf{loc\_st}_i)$

   (b) Parties already in AB_{t-1} who remain in AB_t may reuse their existing keys and hints.
   (c) All parties broadcast $(\mathsf{hint}_i, \mathsf{vk}_i)$.

(d) The aggregator (or any party) runs $(\mathsf{AK}_t, \mathsf{VK}_t) \leftarrow \mathsf{hinTS.PreProcess}(\mathsf{SRS}, \{\mathsf{hint}_i, \mathsf{vk}_i\}_{i\in[n_t]})$. hinTS.PreProcess internally verifies each hint via hinTS.PartVerify and discards invalid ones.[27]

(e) The aggregation key AK_t and verification key VK_t for epoch η_t are broadcasted.

2. hinTS **threshold signing:** Executed for every message that needs to be signed during epoch η_t.

(a) A quorum $\mathcal{S} \subseteq \mathsf{AB}_t$ with $\sum_{j\in\mathcal{S}} w_j \geqslant t_{\mathsf{th}}$ decides to sign message msg.

(b) Each party $P_i \in \mathcal{S}$ produces a partial BLS signature: $\sigma_i \leftarrow \mathsf{hinTS.PartSign}(\mathsf{sk}_i, \mathsf{msg})$.

(c) The aggregator (which can be anyone from or outside the quorum) runs:

$$\mathsf{ASIG} \leftarrow \mathsf{hinTS.SignAggr}(\mathsf{SRS}, \mathsf{AK}, \mathsf{msg}, \{i, \sigma_i\}_{i\in\mathcal{S}})$$

Recall that, $\mathsf{ASIG} = (\sigma, \mathsf{AVK}, \mathbf{\Pi})$.

(d) ASIG is broadcasted. Any verifier (can be a smart-contract) checks $\mathsf{dec\_bit} \leftarrow \mathsf{hinTS.Verify}(\mathsf{SRS}, \mathsf{msg}, \mathsf{ASIG}, t, \mathsf{VK})$.

3. **Address book rotation (Key-update):** Executed when the network decides to transition from AB_t to AB_{t+1}.

(a) A quorum $\mathcal{S} \subseteq \mathsf{AB}_t$ with $\sum_{j\in\mathcal{S}} w_j \geqslant t_{\mathsf{th}}$ authorizes the rotation by signing the rotation message $\mathsf{msg}_t := \mathcal{H}(\mathsf{VK}_t \| \mathsf{AB}_{t+1})$ using **Schnorr multi-signature**:

$$\sigma_t \leftarrow \mathsf{Mult.Schnorr.Sign}(\{\mathsf{ssk}_j\}_{j\in\mathcal{S}}, \mathsf{msg}_t)$$

The quorum belongs to the *current* address book AB_t. The rotation message binds the current hinTS verification key and the next address book, preventing replay and substitution attacks.

(b) The prover assembles the auxiliary input $\mathsf{aux}_t := (\mathsf{AB}_t, \mathsf{AB}_{t+1}, \sigma_t, \mathcal{S}, \boldsymbol{b})$ and runs **NOVA IVC proof**:

$$\pi_t := \mathsf{IVC.Prove}(\mathsf{pp}, t, \boldsymbol{\psi}_0, \boldsymbol{\psi}_t, \pi_{t-1})$$

where π_{t-1} is the IVC proof from the previous epoch (or $\perp$ if $t = 1$). Internally, IVC.Prove folds the previous fresh instance into the accumulator (via NIFS.Prove), then runs F' via trace. The augmented circuit F' receives aux_t as part of its input, invokes F to verify the Schnorr multi-signature σ_t under AB_t and to compute $\boldsymbol{\psi}_{t+1}$, and performs the NIFS folding verification. The resulting $\pi_t = (\mathbb{U}_{\mathsf{acc}}^{(t)}, \mathbb{W}_{\mathsf{acc}}{}^{(t)}, \mathbb{U}_t, \mathbb{W}_t)$ is maintained only by the prover.

(c) Use Groth16 to compress the IVC proof as a **decider**:

$$\pi_t^{\mathsf{c}} := \mathsf{Decider.Prove}(pk_{\mathsf{G}}, t, \boldsymbol{\psi}_0, \boldsymbol{\psi}_t, \pi_t)$$

Compresses the full IVC proof into a constant-size Groth16 proof ($\approx$ 192 bytes), suitable for on-chain or external verification.

4. **External Verification:** Any external verifier, given only the genesis hash and the current state, can verify the entire chain of rotations.

(a) Inputs: genesis state $\boldsymbol{\psi}_0$, current state $\boldsymbol{\psi}_t$, step t, compressed proof π_t^{c}, decider verification key vk_{G}.

(b) Computes $\mathsf{dec\_bit} := \mathsf{Decider.Verify}(vk_{\mathsf{G}}, t, \boldsymbol{\psi}_0, \boldsymbol{\psi}_t, \pi_t^{\mathsf{c}})$ using a single Groth16 verification (3 pairings). No knowledge of intermediate address books, participating sets, hinTS keys, or Schnorr signatures is required.

Now, for further exposition we provide an illustrative toy example below:

[27] In our current deployment, every party performs this verification; in general, any party, including an external party, such as a dedciated aggregator, may run it independently.

5.1.1 Toy Example: 5-Party Network

We illustrate the operational flow with a concrete example. Consider 5 initial parties with the following weights:

Party	Schnorr key	hinTS key	Weight
P_1	svk_1	vk_1	1
P_2	svk_2	vk_2	2
P_3	svk_3	vk_3	3
P_4	svk_4	vk_4	1
P_5	svk_5	vk_5	2

Total weight $\widehat{w} = 9$. Threshold $t_{\mathsf{th}} = \lceil 9/2 \rceil + 1 = 6$.

System setup. The network runs $\mathsf{SRS} \leftarrow \mathsf{hinTS.Setup}(1^\kappa)$, $\mathsf{pp} \leftarrow \mathsf{IVC.Setup}(F)$, and $(pk_{\mathsf{G}}, vk_{\mathsf{G}}) \leftarrow \mathsf{Decider.Setup}(\mathsf{pp})$. Each of the 5 parties generates $(\mathsf{ssk}_i, \mathsf{svk}_i) \leftarrow \mathsf{Mult.Schnorr.KGen}(i)$ and $(\mathsf{sk}_i, \mathsf{vk}_i) \leftarrow \mathsf{hinTS.KGen}(\mathsf{AllPar}, i)$, and registers the public keys.

Genesis. The genesis address book $\mathsf{AB}_0 = [(P_1, \mathsf{svk}_1, \mathsf{vk}_1, 1), \ldots, (P_5, \mathsf{svk}_5, \mathsf{vk}_5, 2)]$ is assembled. All 5 parties generate hints ($\mathsf{loc\_st}_i \leftarrow \mathsf{hinTS.PreHintGen}(\mathsf{SRS}, i)$, $\mathsf{hint}_i \leftarrow \mathsf{hinTS.HintGen}(\mathsf{sk}_i, \mathsf{loc\_st}_i)$) and broadcast $(\mathsf{hint}_i, \mathsf{vk}_i)$. Every party runs $(\mathsf{AK}_0, \mathsf{VK}_0) \leftarrow \mathsf{hinTS.PreProcess}(\mathsf{SRS}, \{\mathsf{hint}_i, \mathsf{vk}_i\}_{i\in[5]})$. The root-of-trust is $\boldsymbol{\psi}_0 = (\mathcal{H}(\mathsf{AB}_0), \mathcal{H}(\mathsf{VK}_0))$, with $\pi_0 = \perp$. The system now enters epoch η_0.

Epoch 0: Signing (step 2). With $(\mathsf{AK}_0, \mathsf{VK}_0)$ in hand, parties can produce hinTS threshold signatures. Any quorum with weight $\geqslant 6$ suffices:

- $\mathcal{S} = \{P_2, P_3, P_5\}$: weight $2 + 3 + 2 = 7 \geqslant 6$. ✓
- $\mathcal{S} = \{P_1, P_3, P_5\}$: weight $1 + 3 + 2 = 6 \geqslant 6$. ✓
- $\mathcal{S} = \{P_2, P_3\}$: weight $2 + 3 = 5 < 6$. ×

Suppose the quorum $\mathcal{S} = \{P_2, P_3, P_5\}$ signs a consensus message msg:

1. Each signer produces a partial BLS signature: $\sigma_i \leftarrow \mathsf{hinTS.PartSign}(\mathsf{sk}_i, \mathsf{msg})$ for $i \in \{2, 3, 5\}$.
2. The aggregator computes $\mathsf{ASIG} \leftarrow \mathsf{hinTS.SignAggr}(\mathsf{SRS}, \mathsf{AK}_0, \mathsf{msg}, \{(2, \sigma_2), (3, \sigma_3), (5, \sigma_5)\})$. Recall $\mathsf{ASIG} = (\sigma, \mathsf{AVK}, \boldsymbol{\Pi})$.
3. Any verifier (e.g., a smart contract) checks $\mathsf{dec\_bit} \leftarrow \mathsf{hinTS.Verify}(\mathsf{SRS}, \mathsf{msg}, \mathsf{ASIG}, t_{\mathsf{th}}, \mathsf{VK}_0)$ and accepts.

Epoch 0: Rotation to epoch 1 (step 3). Suppose P_4 leaves and a new party P_6 (weight 2) joins, giving $\mathsf{AB}_1 = [(P_1, \mathsf{svk}_1, \mathsf{vk}_1, 1), (P_2, \mathsf{svk}_2, \mathsf{vk}_2, 2), (P_3, \mathsf{svk}_3, \mathsf{vk}_3, 3), (P_5, \mathsf{svk}_5, \mathsf{vk}_5, 2), (P_6, \mathsf{svk}_6, \mathsf{vk}_6, 2)]$.

1. **Schnorr multi-sig (step 3a).** A quorum from AB_0, say $\mathcal{S} = \{P_2, P_3, P_5\}$ (weight $7 \geqslant 6$), signs $\mathsf{msg}_0 = \mathcal{H}(\mathsf{VK}_0 \| \mathsf{AB}_1)$:
$$\sigma_0 \leftarrow \mathsf{Mult.Schnorr.Sign}(\{\mathsf{ssk}_2, \mathsf{ssk}_3, \mathsf{ssk}_5\}, \mathsf{msg}_0)$$
2. **NOVA IVC step (step 3b).** The prover assembles $\mathsf{aux}_0 = (\mathsf{AB}_0, \mathsf{AB}_1, \sigma_0, \{P_2, P_3, P_5\}, \boldsymbol{b})$ and runs:
$$\pi_1 \leftarrow \mathsf{IVC.Prove}(\mathsf{pp}, 1, \boldsymbol{\psi}_0, \boldsymbol{\psi}_1, \perp)$$
Since $\pi_0 = \perp$ (first rotation), $\mathsf{IVC.Prove}$ initializes $\mathbb{U}_{\mathsf{acc}}^{(0)} = \mathbb{U}_\perp$, runs F' via trace—F' verifies σ_0 against AB_0, checks quorum weight $7 \geqslant 6$, computes $\boldsymbol{\psi}_1 = (\mathcal{H}(\mathsf{AB}_1), \mathcal{H}(\mathsf{VK}_1))$—and folds to produce $\pi_1 = (\mathbb{U}_{\mathsf{acc}}^{(1)}, \mathbb{W}_{\mathsf{acc}}{}^{(1)}, \mathbb{U}_1, \mathbb{W}_1)$.
3. **Decider (step 3c).** $\pi_1^{\mathsf{c}} \leftarrow \mathsf{Decider.Prove}(pk_{\mathsf{G}}, 1, \boldsymbol{\psi}_0, \boldsymbol{\psi}_1, \pi_1)$. Output: ≈ 192 bytes.

Epoch 1: Hint generation (step 1). The new validator set AB_1 has total weight $1+2+3+2+2=10$ and threshold $t_{\mathsf{th}} = \lceil 10/2 \rceil + 1 = 6$. The new party P_6 generates fresh keys and hints; parties P_1, P_2, P_3, P_5 reuse theirs. The aggregator runs $(\mathsf{AK}_1, \mathsf{VK}_1) \leftarrow \mathsf{hinTS.PreProcess}(\mathsf{SRS}, \{\mathsf{hint}_i, \mathsf{vk}_i\}_{i \in \mathsf{AB}_1})$.

Epoch 1: Signing (step 2). Signing proceeds with threshold 6 using the new keys $(\mathsf{AK}_1, \mathsf{VK}_1)$. For instance, $\mathcal{S} = \{P_1, P_3, P_6\}$ (weight $1+3+2=6 \geqslant 6$) signs a message msg': each signer produces $\sigma_i \leftarrow \mathsf{hinTS.PartSign}(\mathsf{sk}_i, \mathsf{msg}')$, the aggregator computes $\mathsf{ASIG}' \leftarrow \mathsf{hinTS.SignAggr}(\mathsf{SRS}, \mathsf{AK}_1, \mathsf{msg}', \{(1,\sigma_1),(3,\sigma_3),(6,\sigma_6)\})$, and broadcasts $\mathsf{ASIG}' = (\sigma', \mathsf{AVK}', \mathbf{\Pi}')$.

External verification (step 4). An external verifier (e.g., a light client or smart contract) trusts the genesis state $\boldsymbol{\psi}_0 = (\mathcal{H}(\mathsf{AB}_0), \mathcal{H}(\mathsf{VK}_0))$. Since VK_0 is part of the root-of-trust, no additional proof is needed to verify epoch-0 signatures: the verifier simply runs $\mathsf{dec\_bit} \leftarrow \mathsf{hinTS.Verify}(\mathsf{SRS}, \mathsf{msg}, \mathsf{ASIG}, t_{\mathsf{th}}, \mathsf{VK}_0)$ and accepts.

However, the epoch-1 signature ASIG' uses a new verification key VK_1 that the verifier does not yet trust. The decider proof becomes necessary: the verifier must first confirm the rotation chain before trusting VK_1. To verify the epoch-1 signature on msg', the verifier performs:

1. **Decider proof check:** $\mathsf{Decider.Verify}(vk_{\mathsf{G}}, 1, \boldsymbol{\psi}_0, \boldsymbol{\psi}_1, \pi_1^{\mathsf{c}})$—a single pairing check. This establishes that $\boldsymbol{\psi}_1 = (\mathcal{H}(\mathsf{AB}_1), \mathcal{H}(\mathsf{VK}_1))$ is the legitimate successor of $\boldsymbol{\psi}_0$, and hence VK_1 is trustworthy.
2. **hinTS signature check:** $\mathsf{hinTS.Verify}(\mathsf{SRS}, \mathsf{msg}', \mathsf{ASIG}', t_{\mathsf{th}}, \mathsf{VK}_1)$. Now that VK_1 is authenticated via the decider proof, this confirms that a sufficient quorum from AB_1 signed msg'.

Epoch 1: Rotation to epoch 2 (step 3). When the network decides to rotate to AB_2, a quorum from AB_1 signs the new rotation message. The IVC prover runs $\pi_2 \leftarrow \mathsf{IVC.Prove}(\mathsf{pp}, 2, \boldsymbol{\psi}_0, \boldsymbol{\psi}_2, \pi_1)$—this time IVC.Prove takes the recursive branch, folding $\mathbb{U}_1$ into the accumulator. The accumulated instance $\mathbb{U}_{\mathsf{acc}}^{(2)}$ now encodes both rotations ($\mathsf{AB}_0 \to \mathsf{AB}_1$ and $\mathsf{AB}_1 \to \mathsf{AB}_2$). The decider compresses to $\pi_2^{\mathsf{c}} \approx 192$ bytes. At epoch 2, an external verifier follows the same pattern: first $\mathsf{Decider.Verify}(vk_{\mathsf{G}}, 2, \boldsymbol{\psi}_0, \boldsymbol{\psi}_2, \pi_2^{\mathsf{c}})$ to authenticate VK_2, then $\mathsf{hinTS.Verify}(\mathsf{SRS}, \mathsf{msg}'', \mathsf{ASIG}'', t_{\mathsf{th}}, \mathsf{VK}_2)$ for any epoch-2 signature—each is a constant-time operation.

5.2 Security Intuition

The end-to-end security of the HieroTSS key-update mechanism rests on:

1. **Genesis hash authenticity.** The verifier trusts $\mathcal{H}(\mathsf{AB}_0)$ as the root of trust.
2. **Schnorr multi-signature unforgeability.** At each rotation step, only a quorum from AB_t meeting the weight threshold t_{th} can produce a valid signature on the rotation message. With proof of possession (cf. Section 4.1), rogue key attacks are prevented.
3. **NOVA folding soundness.** A satisfying accumulated instance implies all folded instances were satisfying, with soundness error $\leqslant 2/|\mathbb{F}|$ per step (Schwartz-Zippel). Over T steps, the total error $\leqslant 2T/|\mathbb{F}| = \mathsf{negl}(\kappa)$.
4. **Groth16 soundness.** The decider proof is computationally sound under the q-type knowledge assumptions in the bilinear group.
5. **Pedersen commitment binding.** Commitments $\overline{W}, \overline{E}$ are computationally binding under the DLP assumption.

5.3 Resource Requirements

Parameter	Value
IVC proof size (uncompressed)	$O(m+n)$ — proportional to circuit size
Decider proof size (compressed)	3 group elements ≈ 192 bytes
Per-step prover cost	2 MSMs of size $\approx \lvert F' \rvert$ (no FFTs)
Decider prover cost	Full Groth16 proof of C_{dec}
Verification cost (Decider.Verify)	1 MSM over public inputs + 3 pairings

6 Conclusion

In this paper, we describe the HieroTSS weighted threshold signing system in detail. Specifically, we present the full construction of hinTS silent threshold signatures—including the KZG-based hint mechanism, the Plonkish zero-test and sumcheck arguments, and the optimized aggregation and verification procedures. Then we provide the building blocks for committee rotation and key updates: Schnorr multi-signatures with proof of possession, Rank-1 Constraint Systems, Groth16, Pedersen commitments, and the NOVA folding scheme with its IVC. Finally, we assemble these into the complete HieroTSS protocol and provide security intuitions for the composed system. The central guarantee is that an external verifier, who trust only the genesis state ψ_0, can verify the legitimacy of any epoch's threshold signature using a constant number of pairing operations, regardless of the number of validators or the length of the rotation history. For full formalization, including definition and security analysis, we refer to the respective research work.

References

[0xP24] 0xPARC and Privacy & Scaling Explorations. Sonobe: Experimental folding schemes library. https://github.com/privacy-scaling-explorations/sonobe, 2024. Accessed: 2026-05-13.

[ark22] arkworks contributors. arkworks zkSNARK ecosystem. https://arkworks.rs, 2022. Accessed March 2026.

[BLS01] Dan Boneh, Ben Lynn, and Hovav Shacham. Short signatures from the Weil pairing. pages 514–532, 2001.

[GJM+24] Sanjam Garg, Abhishek Jain, Pratyay Mukherjee, Rohit Sinha, Mingyuan Wang, and Yinuo Zhang. hinTS: Threshold signatures with silent setup. In *Proceedings of the 2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2024.

[Gro16] Jens Groth. On the size of pairing-based non-interactive arguments. pages 305–326, 2016.

[GWC19] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. PLONK: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. Cryptology ePrint Archive, Paper 2019/953, 2019.

[KMSV21] Markulf Kohlweiss, Mary Maller, Janno Siim, and Mikhail Volkhov. Snarky ceremonies. pages 98–127, 2021.

[KS23] Abhiram Kothapalli and Srinath Setty. CycleFold: Folding-scheme-based recursive arguments over a cycle of elliptic curves. In *Cryptology ePrint Archive, Paper 2023/1192*, 2023.

[KST22] Abhiram Kothapalli, Srinath Setty, and Ioanna Tzialla. Nova: Recursive zero-knowledge arguments from folding schemes. pages 359–388, 2022.

[KZG10] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. pages 177–194, 2010.

[nInT+23] Marta Bellés-Mu noz, Miguel Isabel, Jose Luis Mu noz Tapia, Albert Rubio, and Jordi Baylina. Circom: A circuit description language for building zero-knowledge applications. Cryptology ePrint Archive, Paper 2023/681, 2023.

[RZ21] Carla Ràfols and Arantxa Zapico. An algebraic framework for universal and updatable SNARKs. pages 774–804, 2021.

[Sch91] Claus-Peter Schnorr. Efficient signature generation by smart cards. *Journal of Cryptology*, 4(3):161–174, 1991.